Laboratorio DevSecOps

Pipeline CI/CD con la seguridad integrada desde el primer commit
Jira · Docker · Jenkins · SonarQube · Trivy · Gitleaks · GitHub Actions

Fran Álvarez · TheMagicFrank
github.com/TheMagicFrank/devsecops-lab
Octubre de 2026

Índice

Resumen 3

1. Entorno y requisitos previos 5

2. Planificación en Jira 12

3. Sprint 1 · DSL-5: API REST de usuarios 15

4. Sprint 1 · DSL-6: contenerizar la aplicación 19

5. Cierre del Sprint 1 y lección de trazabilidad 23

6. Sprint 2 · DSL-7 y DSL-8: Jenkins y SonarQube 25

7. Sprint 3 · DSL-14: vulnerabilidad real de mass assignment 34

8. DSL-9: Trivy, imágenes fijadas y Dependabot 39

9. DSL-10: detección de secretos con Gitleaks 46

10. DSL-11: inyección SQL introducida, detectada y corregida 49

11. Sprint 4 · DSL-12: el mismo pipeline en GitHub Actions 55

12. DSL-13: documentación 64

13. Riesgos conocidos y mejoras pendientes 65

Resumen

Este laboratorio monta desde cero un pipeline de integración continua con la seguridad integrada en cada paso (DevSecOps), aplicado a una API REST en Java. Todo el trabajo se ha organizado en sprints de Jira, con una rama y una Pull Request revisada por tarea o bloque de trabajo, y se ha documentado con capturas reales, incluidos los errores y cómo se resolvieron.

El resultado: cada cambio pasa automáticamente por detección de secretos, compilación y tests, análisis de dependencias y de configuración, análisis estático del código y un quality gate. Si algo falla, el pipeline se para y el código no se integra. El mismo pipeline funciona en Jenkins (local) y en GitHub Actions (nube).

Pipeline

#	Etapa	Qué controla
1	Instalar Gitleaks (verificado)	Versión fijada y SHA-256 comprobado antes de ejecutar nada
2	Gitleaks: secretos en el historial	Ningún secreto en ningún commit; si aparece, se para antes de compilar
3	Build y tests	Compilación con Maven y tests (servicio, controlador y regresión de seguridad)
4	Instalar Trivy (verificado)	Misma cadena de confianza que Gitleaks
5	Trivy: dependencias (SCA)	CVE HIGH/CRITICAL con parche disponible en las librerías
6	Trivy: configuración (IaC)	Malas configuraciones en el Dockerfile
7	Análisis SonarQube (SAST)	Vulnerabilidades, bugs y code smells en el código
8	Quality Gate	Si el código no cumple, el pipeline falla

Resultados

Hallazgo	Herramienta	Resultado
Mass assignment (High) en POST /api/usuarios	SonarQube	Explotado, corregido con un DTO y verificado (DSL-14)
Inyección SQL introducida a propósito	SonarQube + quality gate	Pipeline en rojo; corregida con consulta parametrizada (DSL-11)
8 CVE (3 CRITICAL en Tomcat, 5 HIGH en Jackson)	Trivy SCA	Versiones corregidas fijadas en el pom (DSL-9)
Token falso en el historial (demostración)	Gitleaks	Detectado aunque se borró en el commit siguiente (DSL-10)
2 CVE transitivas (JAXB, Logback)	SonarQube Cloud SCA	No aplicables: aceptadas con justificación (DSL-12)

Principios aplicados

- **Mínimo privilegio:** contenedor sin root, .jar propiedad de root, tokens limitados a un proyecto y con caducidad, permisos de solo lectura en Actions.

- **Exponer lo mínimo:** servicios publicados solo en 127.0.0.1, base de datos sin puerto, consola H2 desactivada.
- **Secretos fuera del código:** .env ignorado por Git, credenciales cifradas en Jenkins y secrets en GitHub.
- **Cadena de suministro:** herramientas con versión fijada y SHA-256 verificado, imágenes fijadas por digest, checkout por SHA y Dependabot revisado a mano.
- **Trazabilidad:** cada commit lleva la clave de Jira y cada cambio entra por una Pull Request con descripción y revisión.

1. Entorno y requisitos previos

Todo el laboratorio se ha montado en un portátil con Windows: Java 21 (Eclipse Temurin), Git, Docker Desktop con WSL 2 y una cuenta de GitHub. Solo la instalación de WSL requiere permisos de administrador; todo lo demás se hace con PowerShell **sin privilegios** y fuera de las carpetas del sistema: es el principio de mínimo privilegio aplicado al propio puesto de trabajo.

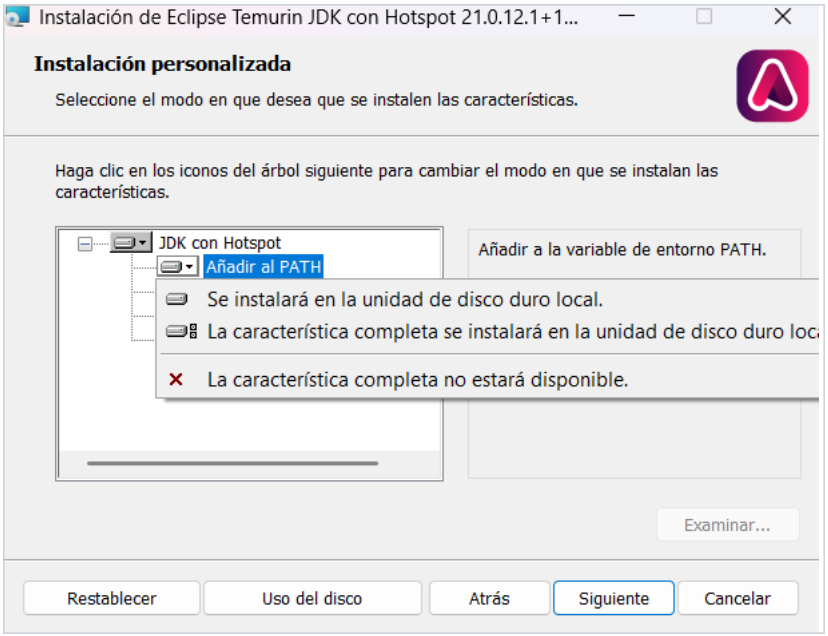


Figura 1. *Instalación de Eclipse Temurin JDK 21 con «Añadir al PATH», para poder usar java desde cualquier terminal.*

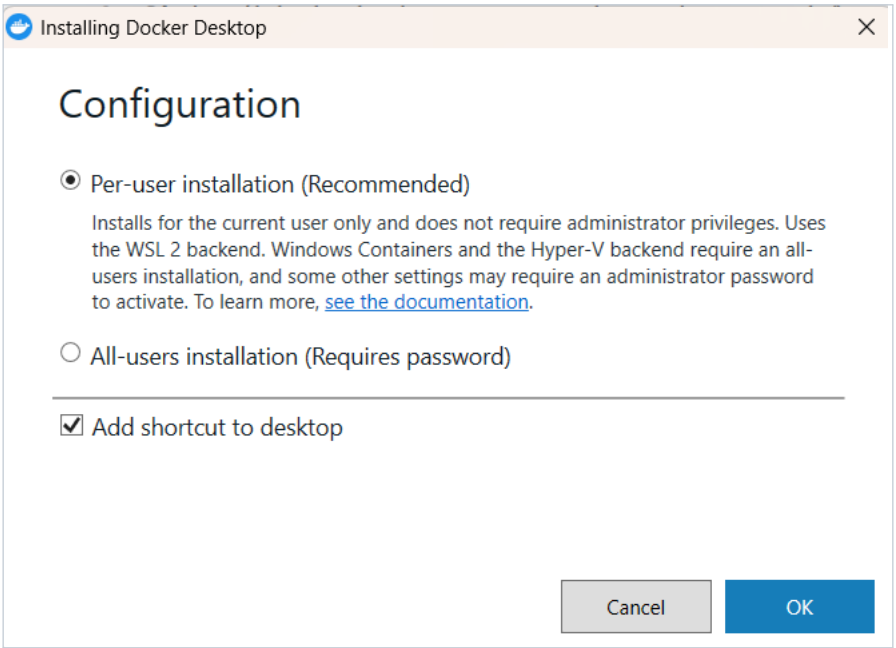


Figura 2. *Docker Desktop en modo Per-user (recomendado): no necesita privilegios de administrador y usa el backend WSL 2.*

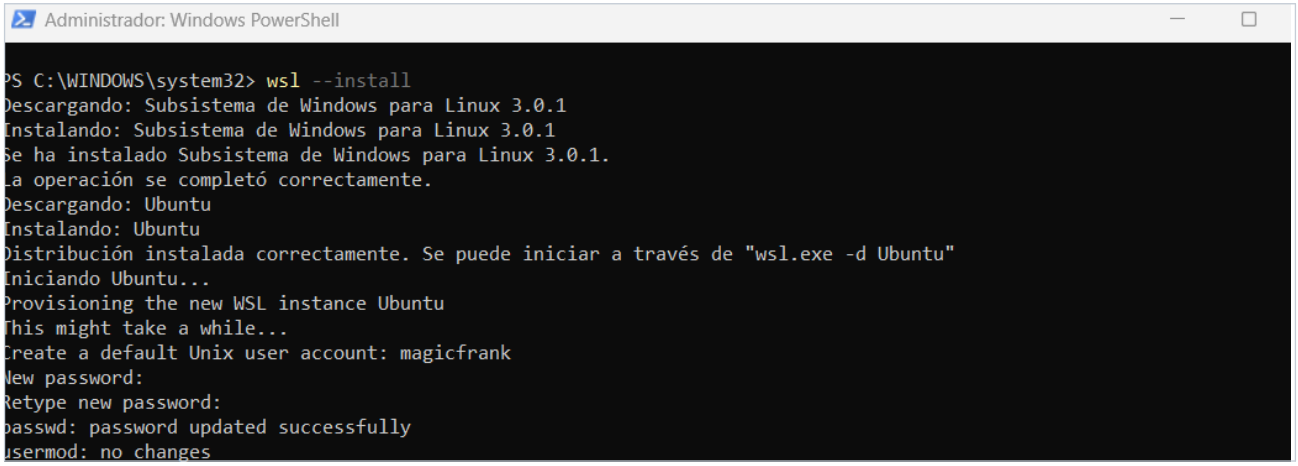


Figura 3. *Instalación de WSL con Ubuntu y creación de la cuenta Unix del laboratorio.*

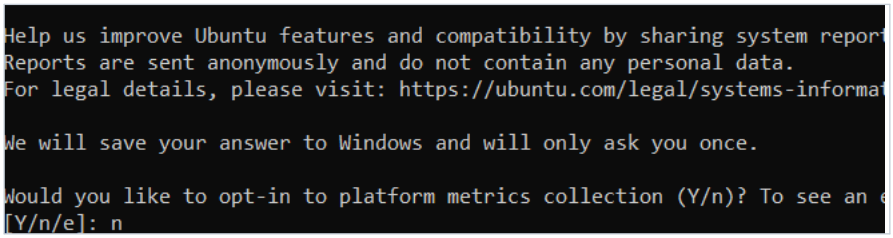


Figura 4. *Envío de métricas de uso a Canonical rechazado: no se comparte más información de la necesaria.*

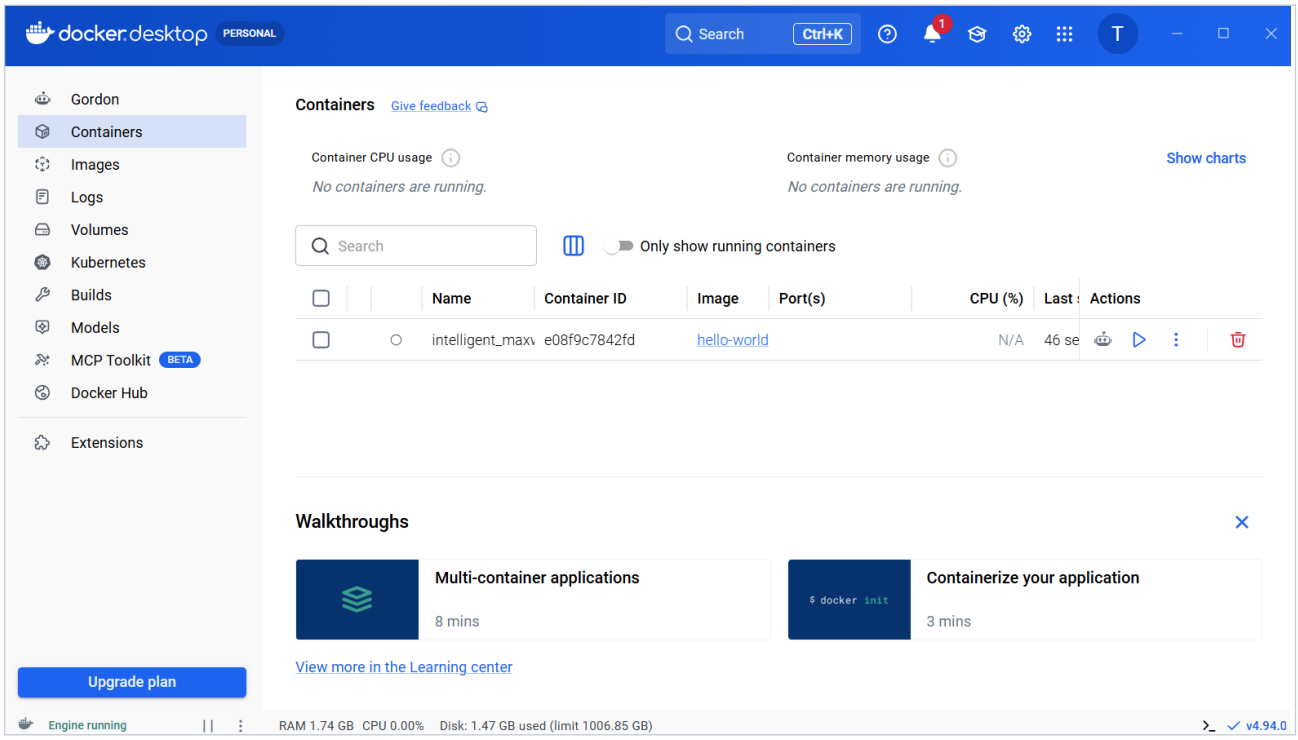


Figura 5. *Docker Desktop con el motor en marcha y el contenedor de prueba hello-world.*

```
PS [redacted] java -version
openjdk version "21.0.12.1" 2026-08-18 LTS
OpenJDK Runtime Environment Temurin-21.0.12.1+1 (build 21.0.12.1+1-LTS)
OpenJDK 64-Bit Server VM Temurin-21.0.12.1+1 (build 21.0.12.1+1-LTS, mixed mode, sharing)
PS [redacted] git --version
git version 2.56.0.windows.2
PS [redacted] docker --version
Docker version 29.8.2, build 7fc2dff
```

Figura 6. *Versiones instaladas: Java 21, Git y Docker.*

```
PS [redacted] git config --global --list
user.name=TheMagicFrank
user.email=339875747+TheMagicFrank@users.noreply.github.com
init.defaultbranch=main
```

Figura 7. *Git configurado con la dirección noreply de GitHub, para que el correo personal no quede en el historial de commits.*

```
PS [redacted] docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

Figura 8. *Primera prueba de Docker: «Hello from Docker!».*

Repositorio y aplicación base

El repositorio público **TheMagicFrank/devsecops-lab** se crea con README, licencia MIT y la plantilla .gitignore de Maven. La aplicación es una API REST en Spring Boot 4.1.1 con Java 21, generada con Spring Initializr.

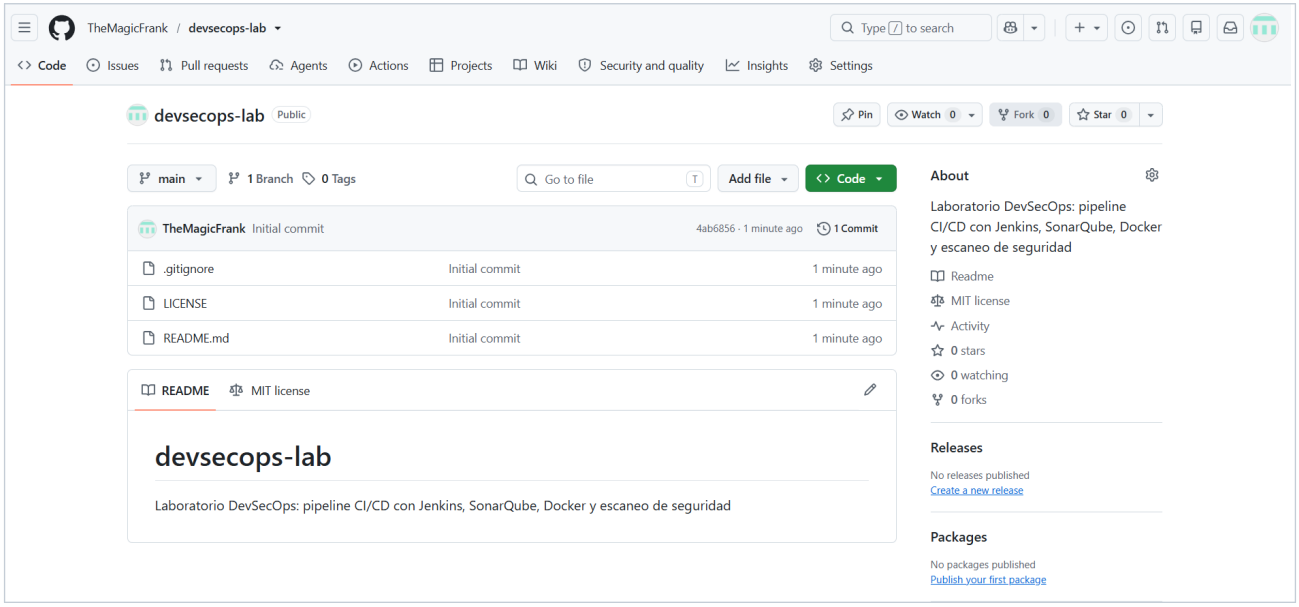


Figura 9. *Repositorio recién creado en GitHub.*

```
PS [redacted] mkdir proyectos


    Directorio: [redacted]


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        10/10/2026     22:38                proyectos


PS [redacted] cd proyectos
PS [redacted] git clone https://github.com/TheMagicFrank/devsecops-lab.git
Cloning into 'devsecops-lab'...
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (5/5), done.
PS [redacted] cd devsecops-lab
PS [redacted]\devsecops-lab> dir


    Directorio: [redacted]


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        10/10/2026     22:38            407 .gitignore
-a----        10/10/2026     22:38           1091 LICENSE
-a----        10/10/2026     22:38            110 README.md


PS [redacted]\devsecops-lab>
```

Figura 10. *Clonado del repositorio en local.*

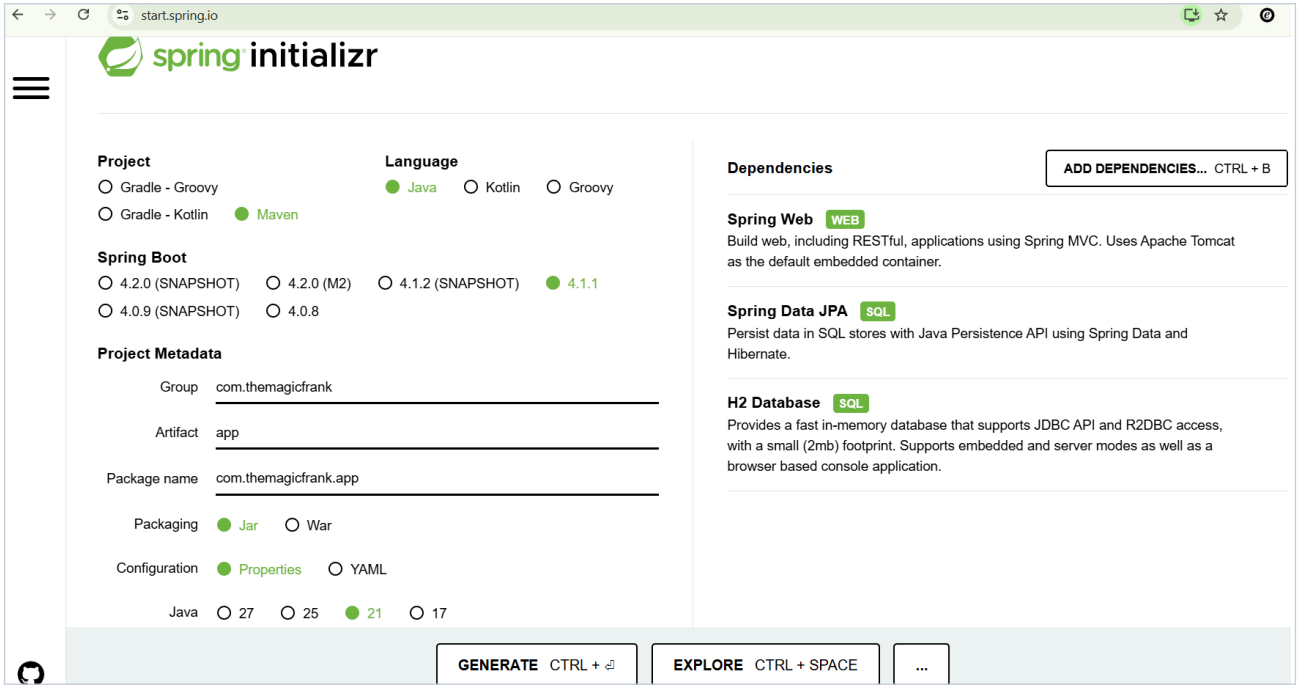


Figura 11. *Spring Initializr: Maven, Java 21, Spring Boot 4.1.1 y las dependencias Spring Web, Spring Data JPA y H2.*

```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 9.174 s --
nTests
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  36.777 s
```

Figura 12. *Primera compilación con Maven: BUILD SUCCESS.*

> **Problema resuelto.** La ventana de autorización de Git Credential Manager no permitía completar el acceso. Se cambió a autenticación por **clave SSH ed25519 con passphrase**, comprobando antes la huella oficial del servidor de GitHub. La clave privada no aparece en ninguna captura.

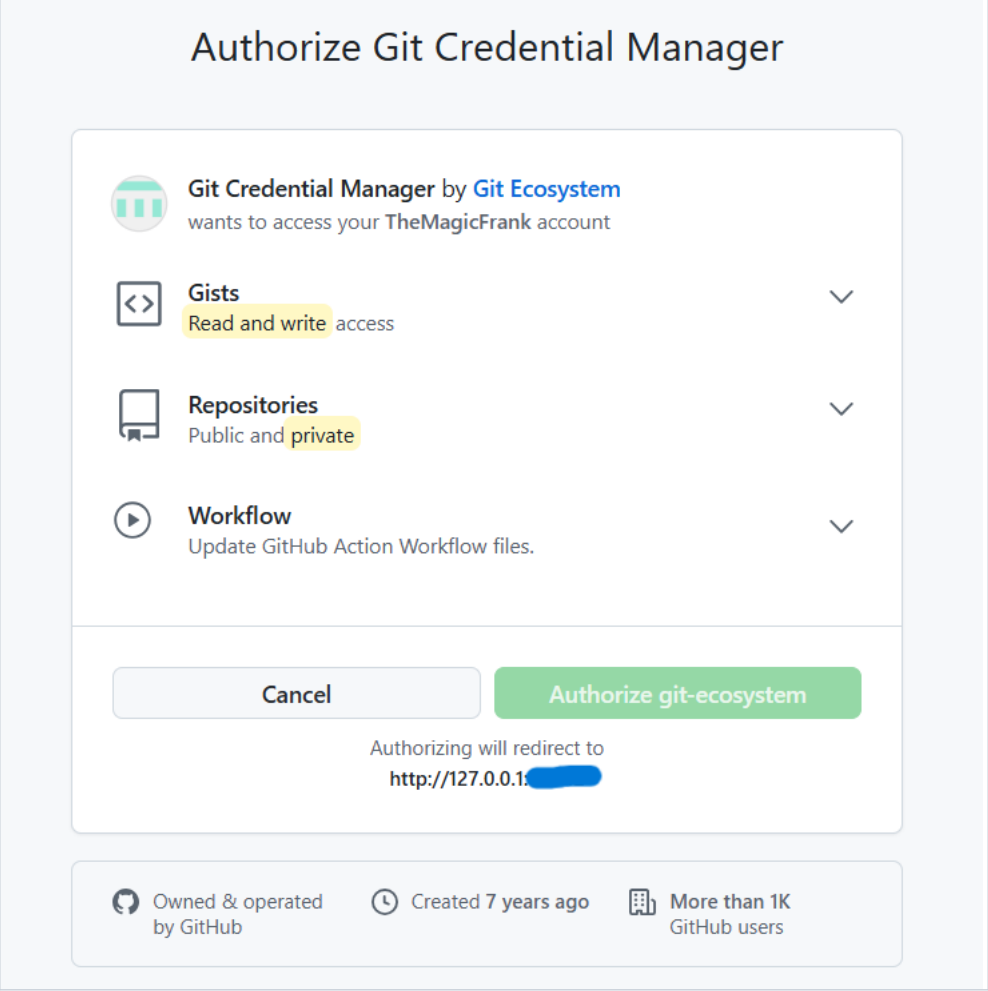


Figura 13. *Ventana de Git Credential Manager que no dejaba autorizar el acceso.*

> **Problema resuelto.** El primer push fue rechazado («fetch first») porque la LICENSE se había editado desde la web. Se integró el cambio remoto con `git pull --rebase` y se volvió a subir.

```
To github.com:TheMagicFrank/devsecops-lab.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'github.com:TheMagicFrank/devsecops-lab.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
```

Figura 14. *Push rechazado por cambios en el remoto que no estaban en local.*

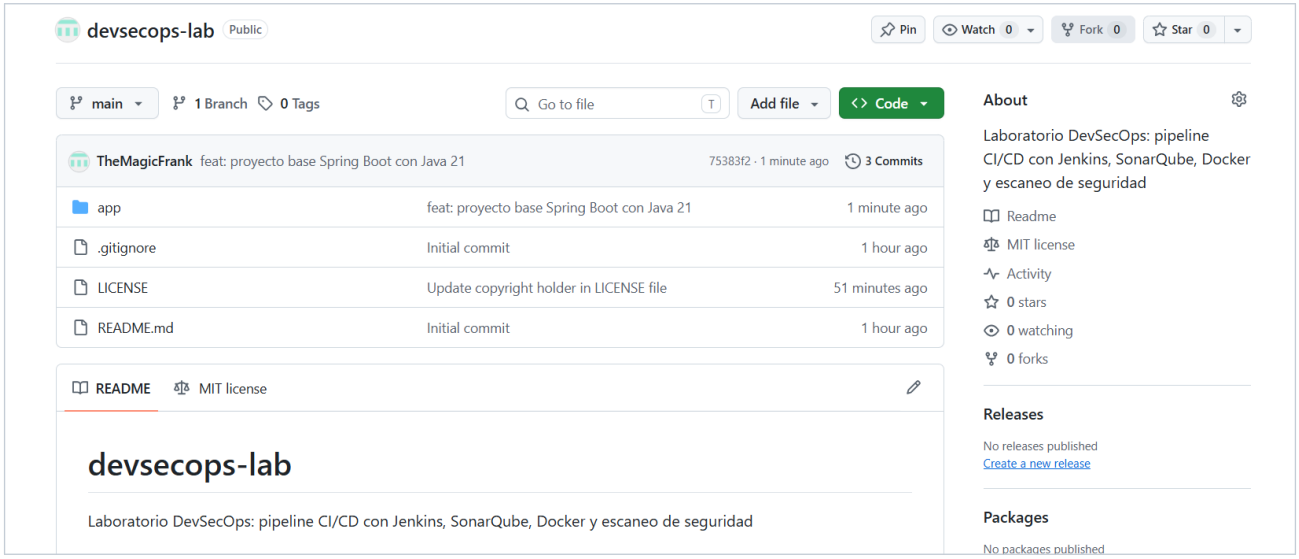


Figura 15. *Repositorio con la carpeta app ya subida.*

2. Planificación en Jira

El trabajo se organiza en un espacio Scrum de Jira, **DevSecOps Lab** (clave DSL), con los estados To Do, In Progress, In Review y Done. Cada tarea o bloque de trabajo tiene su rama, sus commits llevan la clave de Jira y se cierra con una Pull Request revisada, como en un equipo real.

> **Problemas resueltos.** Al cambiar la clave del espacio de SCRUM a DSL, Jira devolvió un error 404 en el tablero. Además creó por su cuenta un sprint activo con el prefijo antiguo y otro duplicado: se eliminó el duplicado y se reutilizó el activo. Las tareas empiezan en DSL-5 porque Jira no reutiliza números.

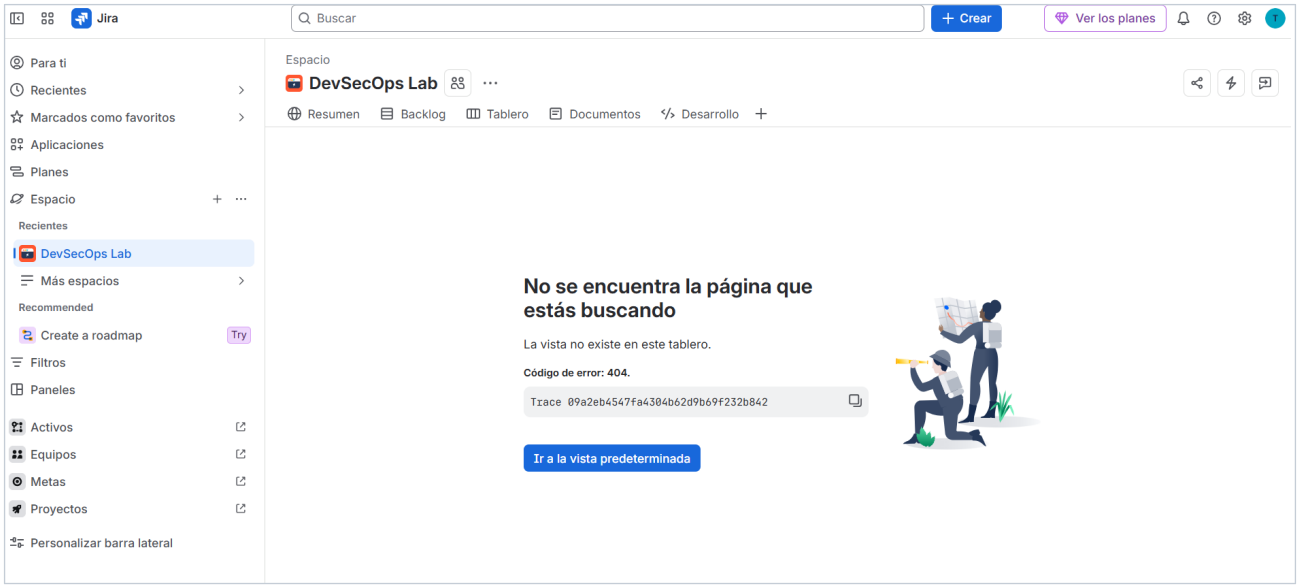


Figura 16. *Error 404 tras cambiar la clave del espacio.*

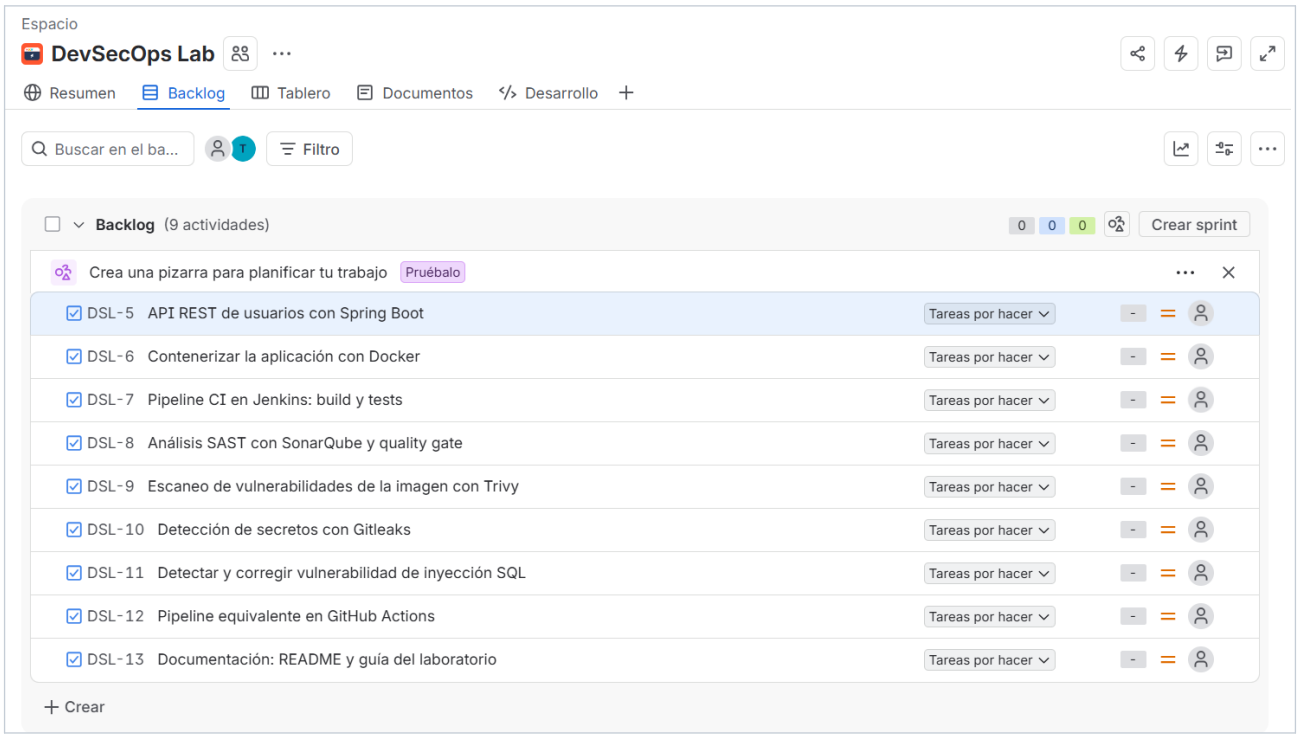


Figura 17. *Backlog con las 9 tareas del laboratorio, de DSL-5 a DSL-13.*

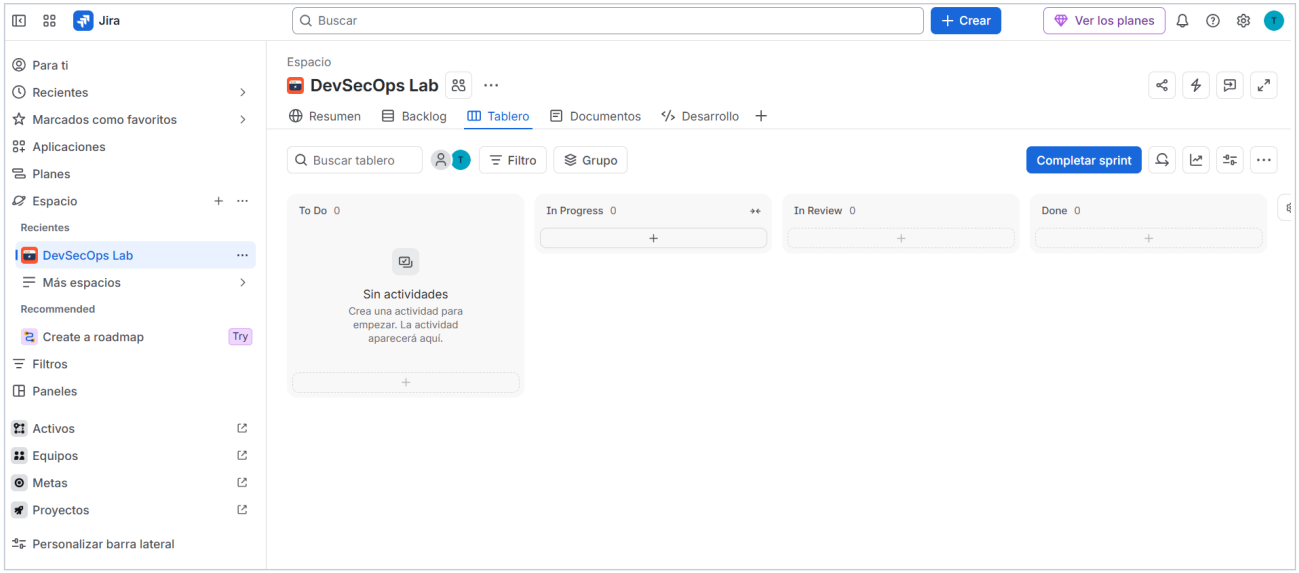


Figura 18. *Espacio creado con las cuatro columnas, incluida In Review.*

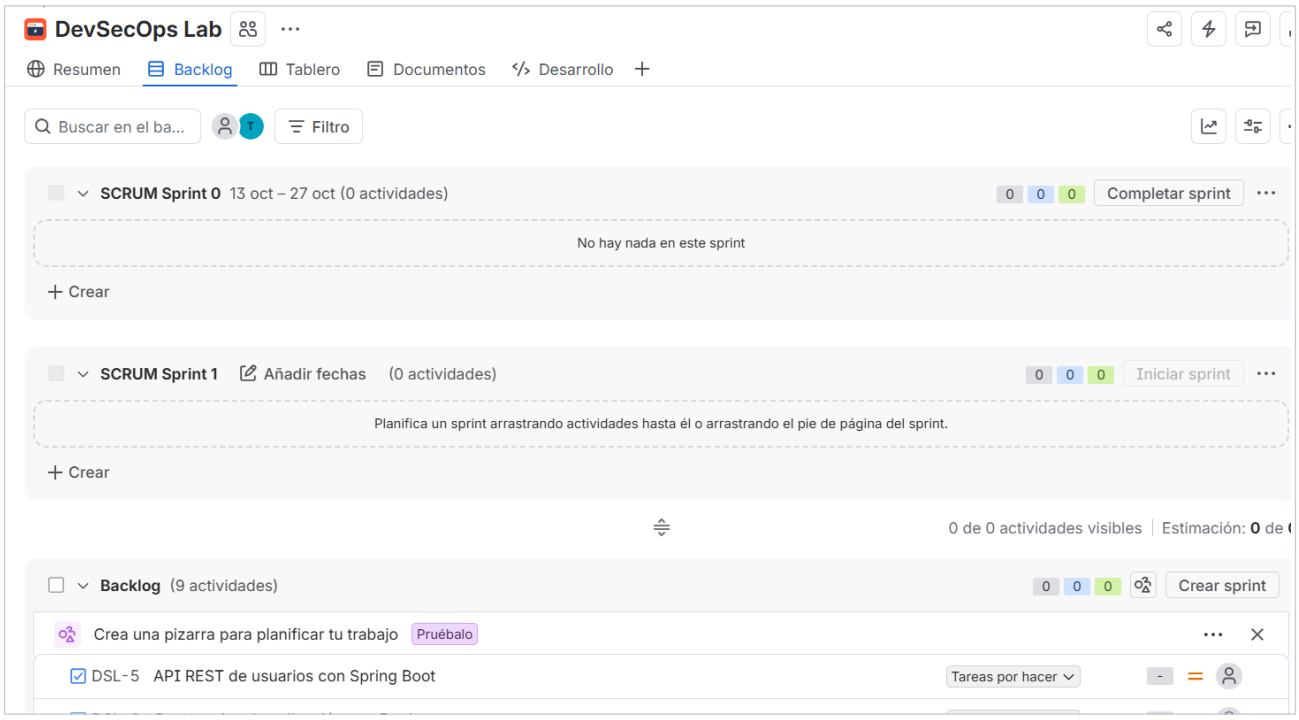


Figura 19. *Sprints duplicados creados automáticamente por Jira.*

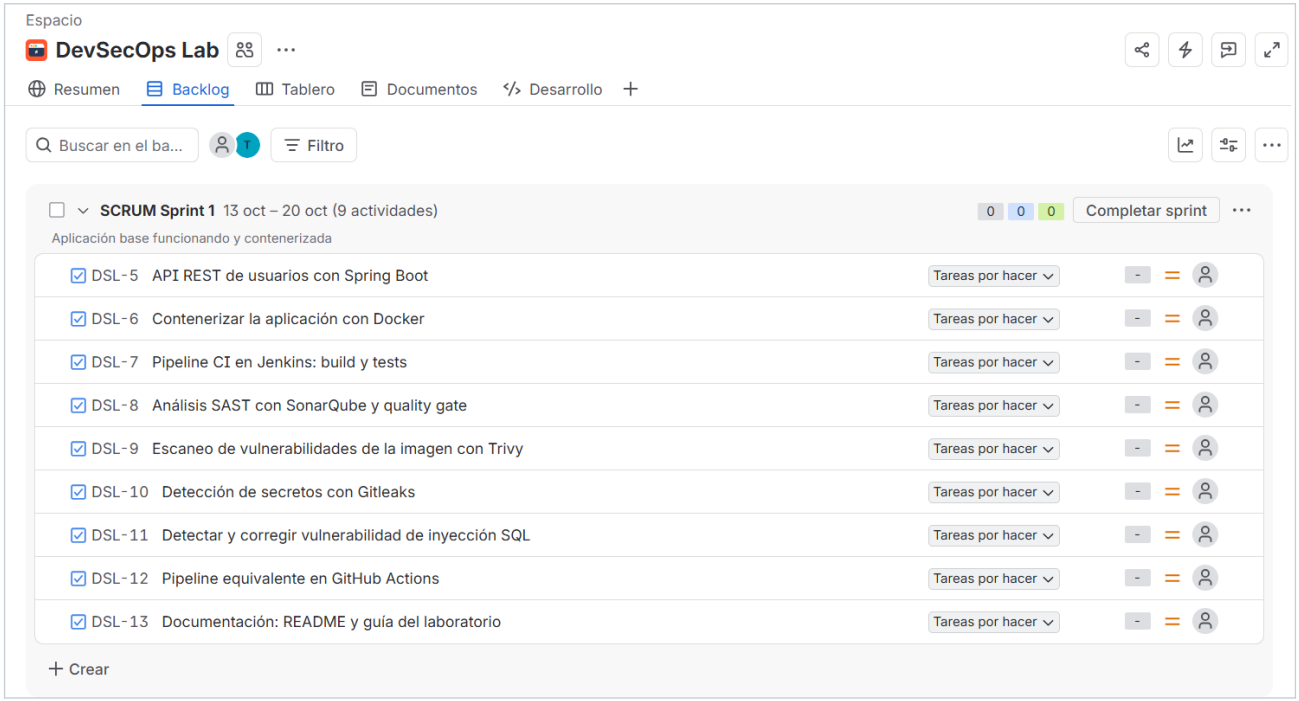


Figura 20. *Las 9 tareas metidas por error en el Sprint 1. Se corrigió dejando solo DSL-5 y DSL-6 (ver la lección de trazabilidad en el apartado 5).*

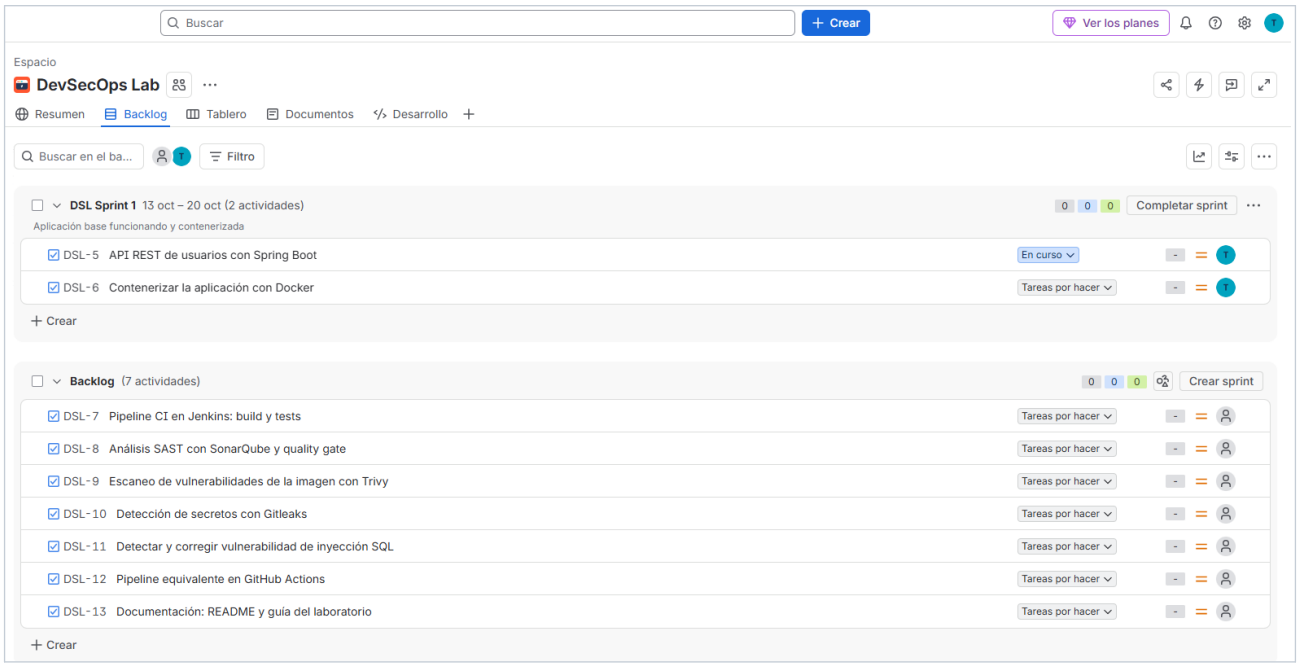


Figura 21. *Sprint 1 con su objetivo: «Aplicación base funcionando y contenerizada».*

3. Sprint 1 · DSL-5: API REST de usuarios

Primera funcionalidad: una API de usuarios con endpoints GET, POST y DELETE en /api/usuarios, persistencia JPA sobre H2 en memoria y tests del servicio. El prompt de PowerShell se acorta a PS> para que las capturas no muestren rutas del equipo.

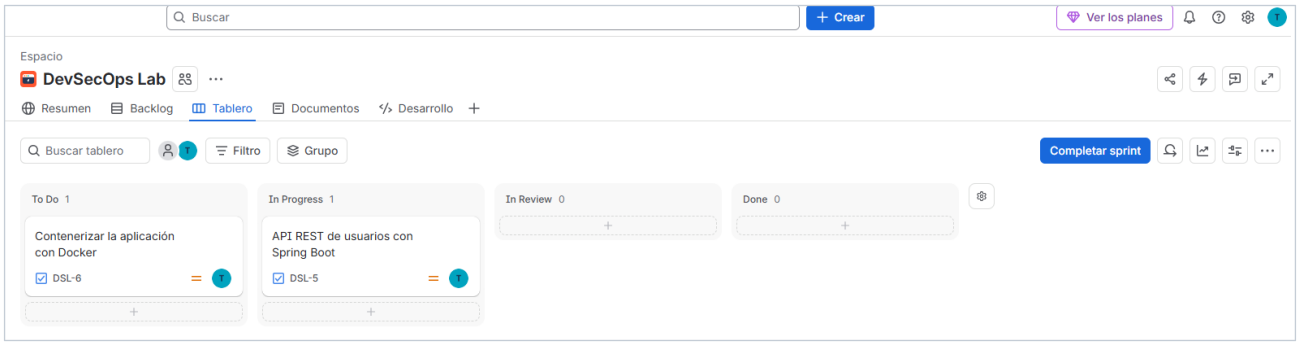


Figura 22. *Tablero con DSL-5 en curso.*

```
PS> function prompt { "PS> " }
PS> cd $HOME\proyectos\devsecops-lab
PS> git checkout -b feature/DSL-5-api-usuarios
Switched to a new branch 'feature/DSL-5-api-usuarios'
PS>
```

Figura 23. *Rama feature/DSL-5-api-usuarios.*

```
[INFO] Results:
[INFO]
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  16.553 s
```

Figura 24. *5 tests en verde.*

> **Decisión de seguridad.** Al arrancar la aplicación, el firewall de Windows pidió permiso de red para Java. Se canceló: la API solo se prueba desde localhost y no tiene por qué ser accesible desde otras máquinas de la red.

```
icfrank.app.AppApplication      : Started AppApplication in 3.064 seconds (proces
s running for 3.394)
```

Figura 25. *Aplicación arrancada.*

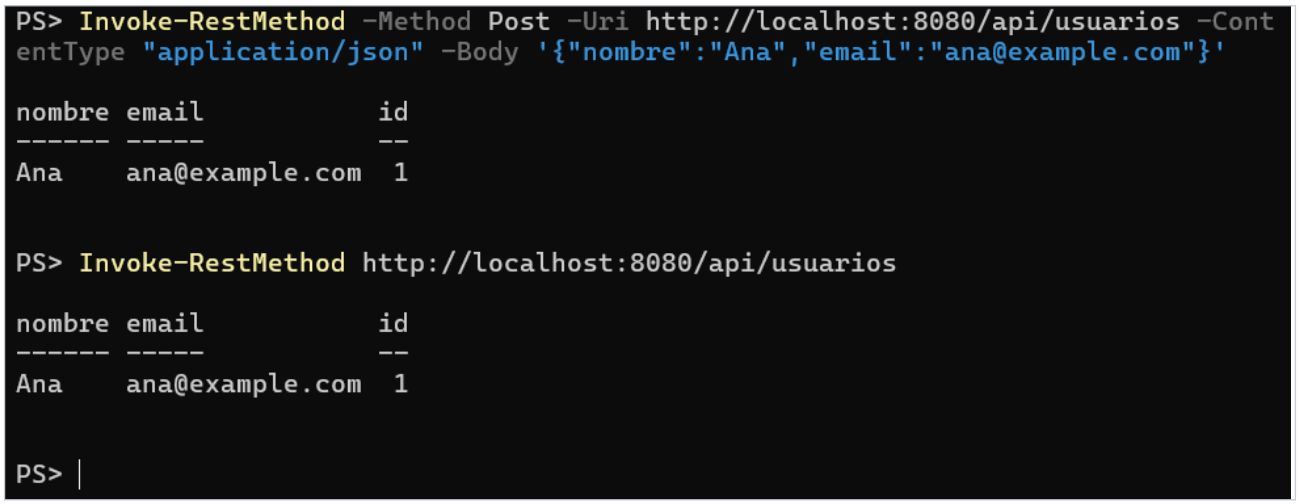


Figura 26. *API probada con Invoke-RestMethod: alta de un usuario y listado.*

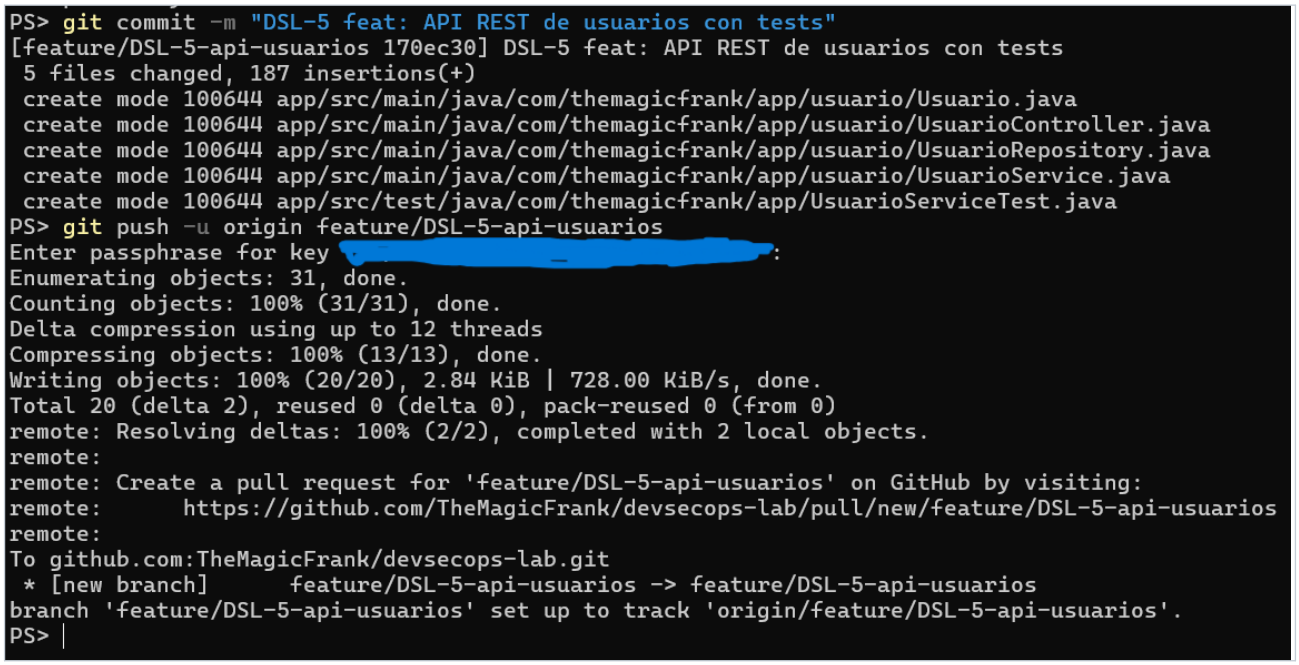


Figura 27. *Commit con la clave DSL-5 y push de la rama. Al revisar esta salida se detectó un test fuera de su paquete, que se movió con git mv.*

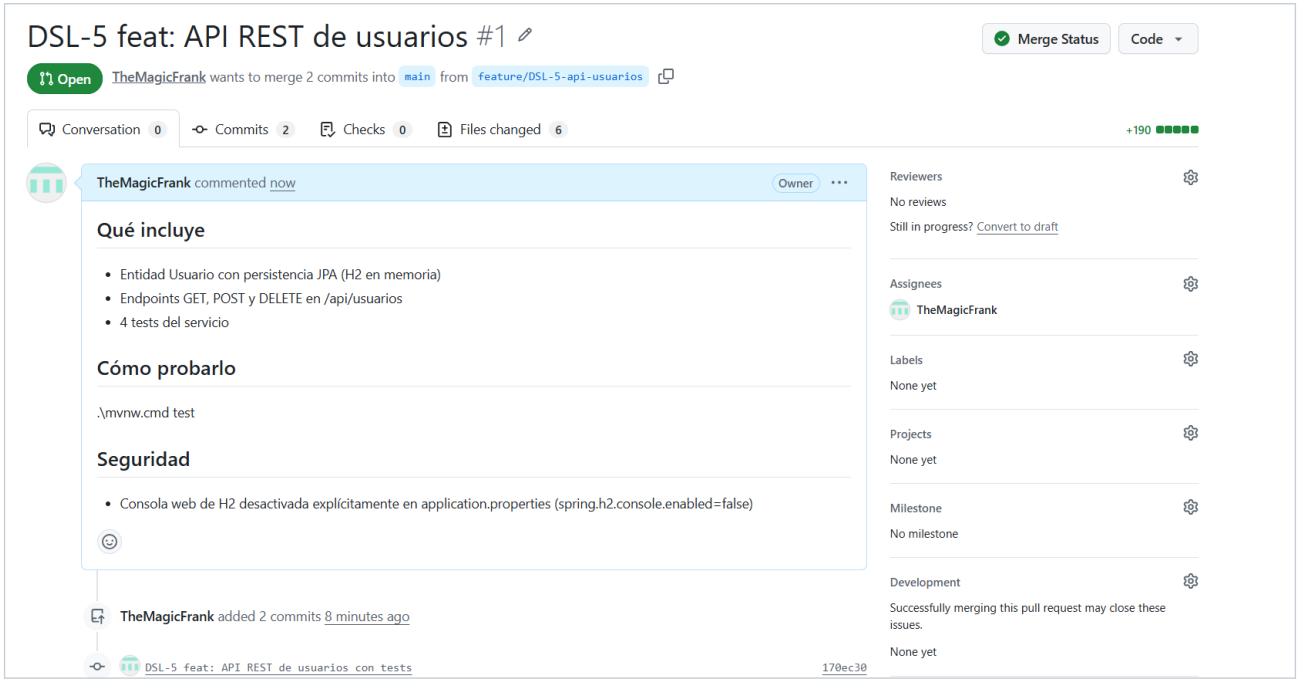


Figura 28. *Pull Request #1 abierta, con descripción y asignada.*

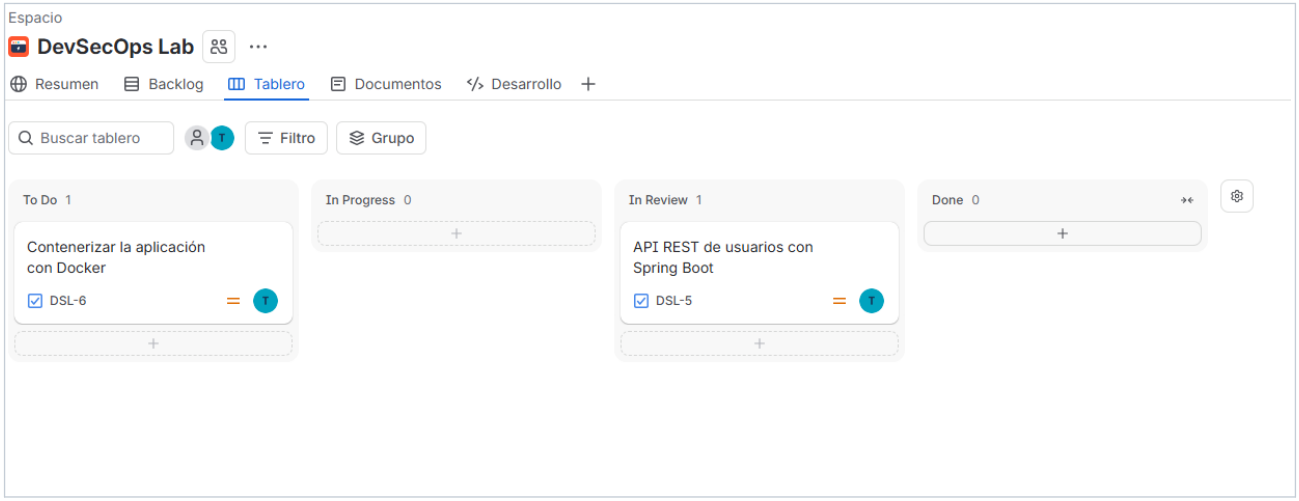


Figura 29. *DSL-5 en In Review.*

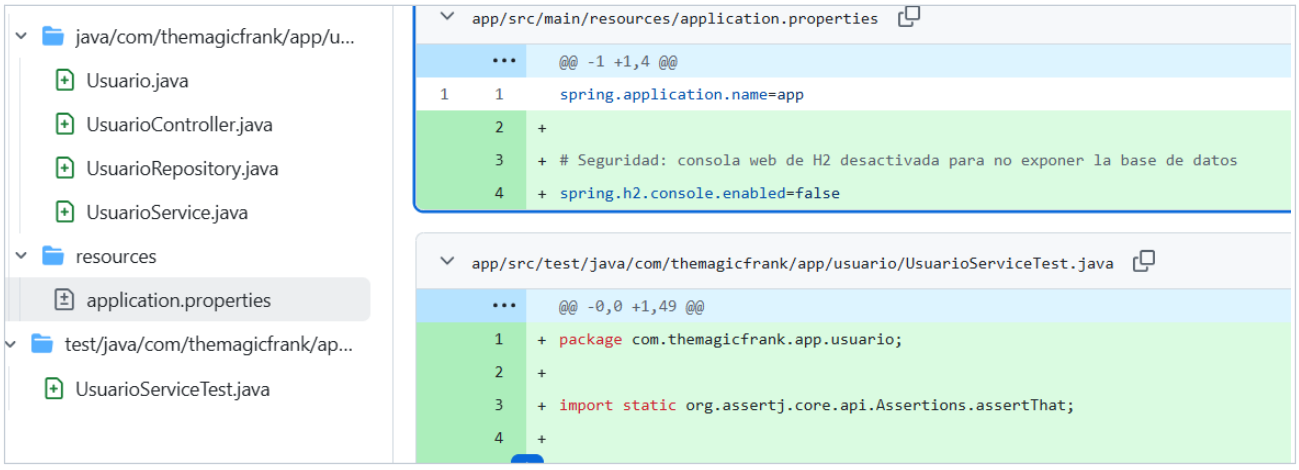


Figura 30. *Revisión de los cambios: la consola web de H2 queda desactivada de forma explícita y el test ya está en su paquete.*

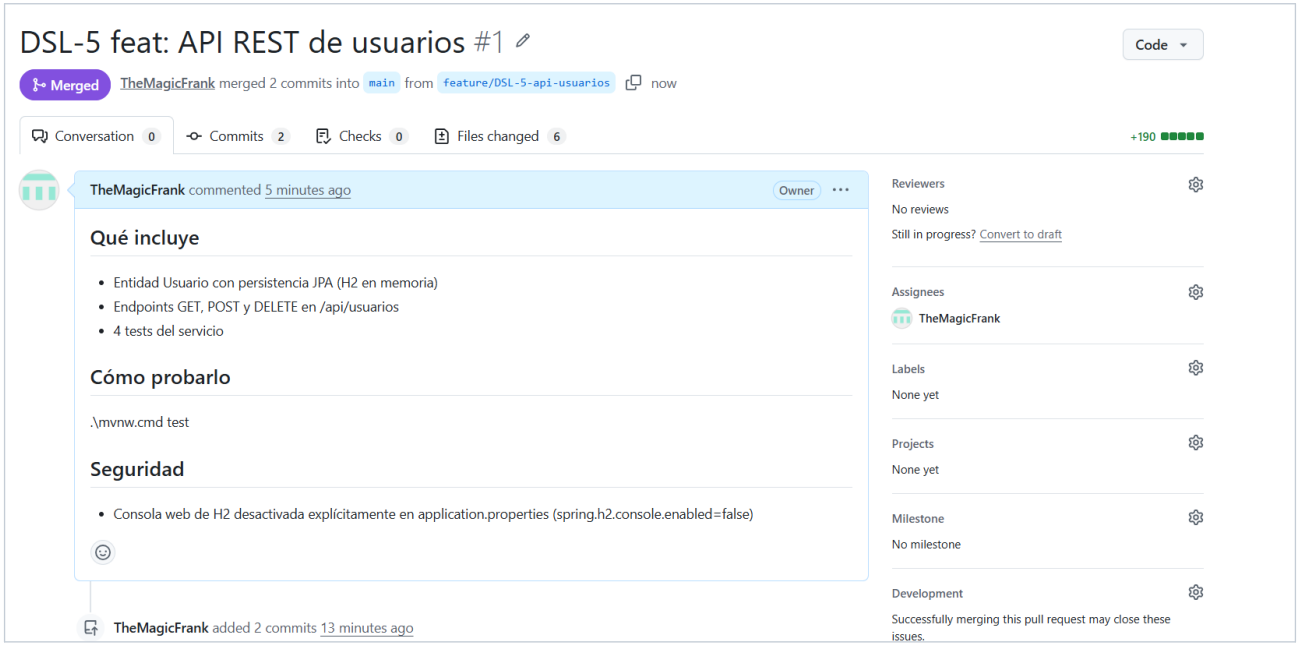


Figura 31. *PR #1 fusionada.*

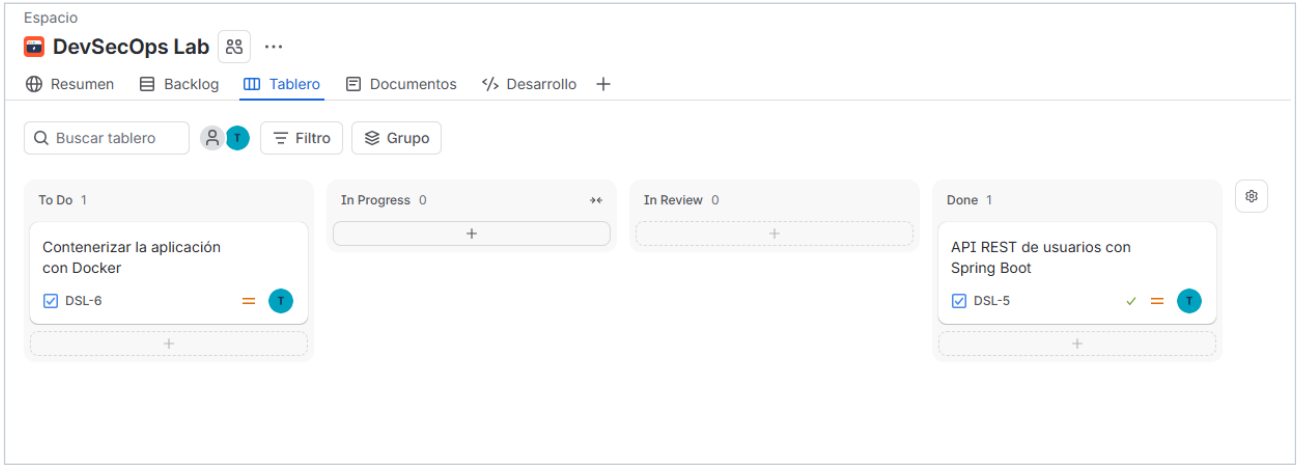


Figura 32. *DSL-5 en Done.*

4. Sprint 1 · DSL-6: contenerizar la aplicación

El Dockerfile es **multi-stage**: compila con la imagen de Maven y ejecuta solo con el JRE 21 sobre Alpine, sin compilador ni Maven en la imagen final. La aplicación corre con un **usuario sin privilegios** (app) y el puerto se publica únicamente en 127.0.0.1, porque Docker se salta el firewall de Windows.

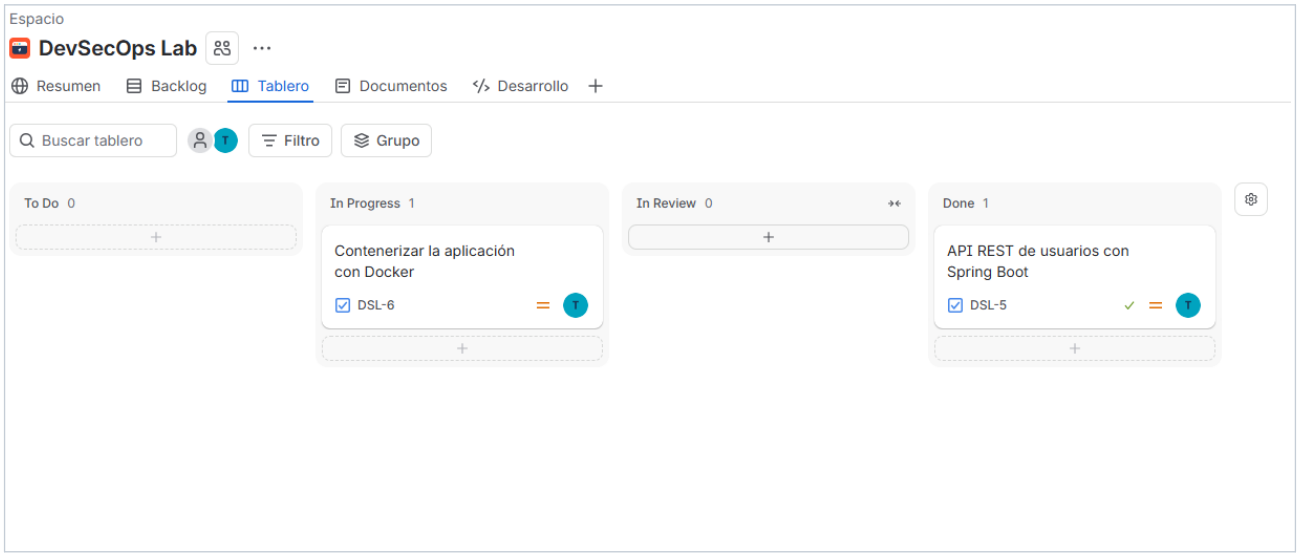


Figura 33. *DSL-6 en curso.*

```
PS> dir -Force app


    Directorio: [redacted]\devsecops-lab\app


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        10/10/2026     23:31                .mvn
d-----        10/10/2026     23:31                src
d-----        10/10/2026     22:55                target
-a----        13/10/2026     18:13             78 .dockerignore
-a----        10/10/2026     23:31             40 .gitattributes
-a----        10/10/2026     23:31            427 .gitignore
-a----        13/10/2026     18:13            902 Dockerfile
-a----        10/10/2026     22:52           1435 HELP.md
-a----        10/10/2026     23:31          11790 mvnw
-a----        10/10/2026     23:31           8481 mvnw.cmd
-a----        10/10/2026     23:31           1977 pom.xml


PS>
```

Figura 34. *Comprobación de que Dockerfile y .dockerignore tienen el nombre exacto.*

```
 => => naming to docker.io/library/devsecops-lab-app:0.1
 => => unpacking to docker.io/library/devsecops-lab-app:0.1
PS>
```

Figura 35. *Imagen devsecops-lab-app:0.1 construida.*

```
PS> docker run -d --name devsecops-app -p 127.0.0.1:8080:8080 devsecops-lab-app:0.1
fef3492ab5ac9809c95373e1c6f821257511177e9bb109fe365145b600611ef2
PS> docker ps
CONTAINER ID   IMAGE                  COMMAND               CREATED          STATUS         PORTS
           NAMES
fef3492ab5ac   devsecops-lab-app:0.1   "java -jar app.jar"   10 seconds ago   Up 9 seconds   127.0.0.1:8080-
>8080/tcp   devsecops-app
PS>
```

Figura 36. *Contenedor en marcha, publicado solo en 127.0.0.1:8080.*

```
PS> Invoke-RestMethod -Method Post -Uri http://localhost:8080/api/usuarios -ContentType "application/json" -
Body '{"nombre":"Luis","email":"luis@example.com"}'

nombre email           id
------ -----           --
Luis   luis@example.com  1


PS> Invoke-RestMethod http://localhost:8080/api/usuarios

nombre email           id
------ -----           --
Luis   luis@example.com  1


PS> docker exec devsecops-app whoami
app
PS>
```

Figura 37. *La API responde dentro del contenedor y whoami devuelve app, no root.*

```
PS> docker images devsecops-lab-app
                                                                    i Info →  U In Use
IMAGE                   ID             DISK USAGE   CONTENT SIZE   EXTRA
devsecops-lab-app:0.1   a62ea649919d       486MB          169MB   U
PS>
```

Figura 38. *Tamaño de la imagen final.*

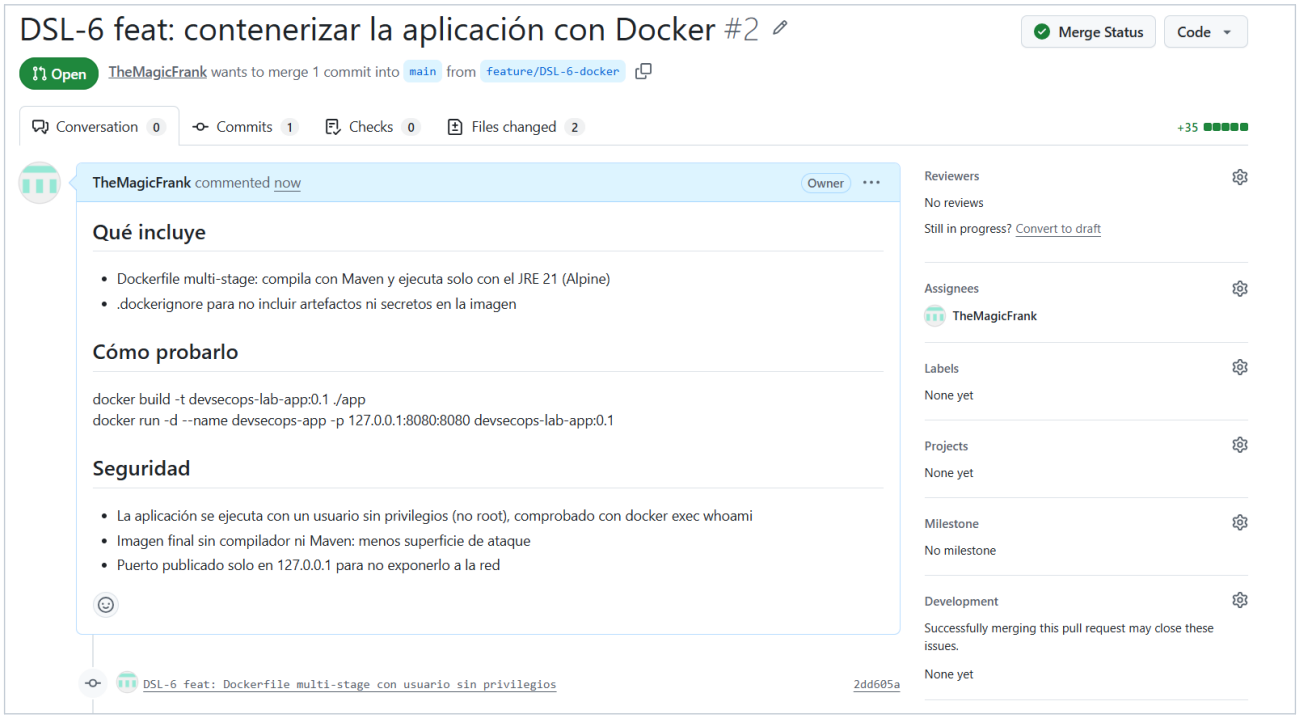


Figura 39. *Pull Request #2 abierta.*

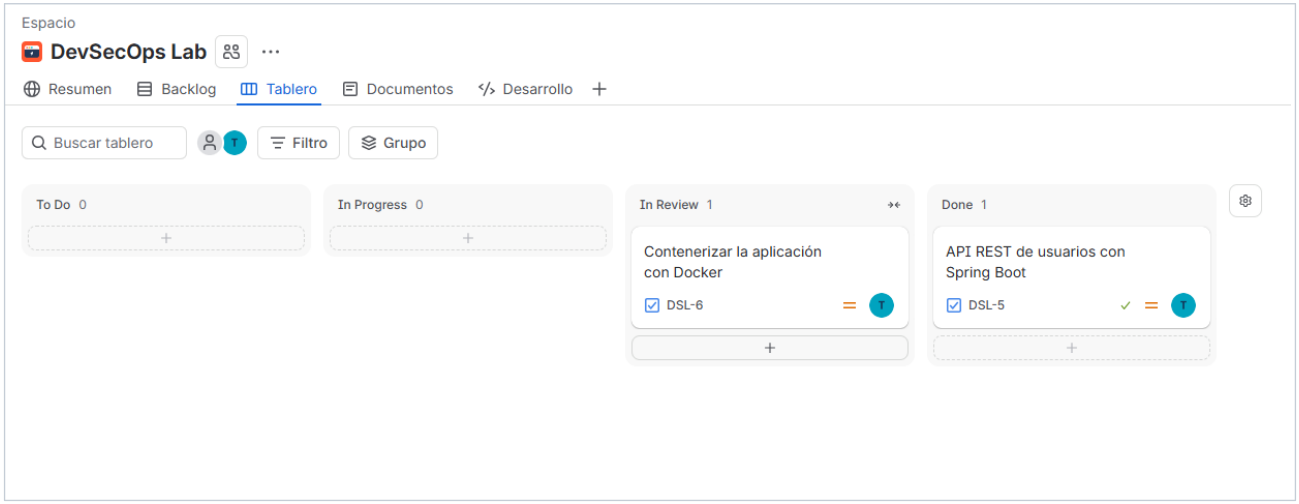


Figura 40. *DSL-6 en In Review.*

> **Hallazgo en la revisión.** El Dockerfile hacía al usuario app dueño de su propio .jar, así que podría modificarlo si alguien explotara la aplicación. Se eliminó el chown: el .jar pertenece a root y el usuario app solo puede leerlo y ejecutarlo.

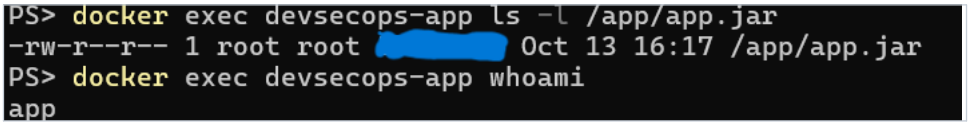


Figura 41. *El .jar pertenece a root y la aplicación sigue ejecutándose como app.*

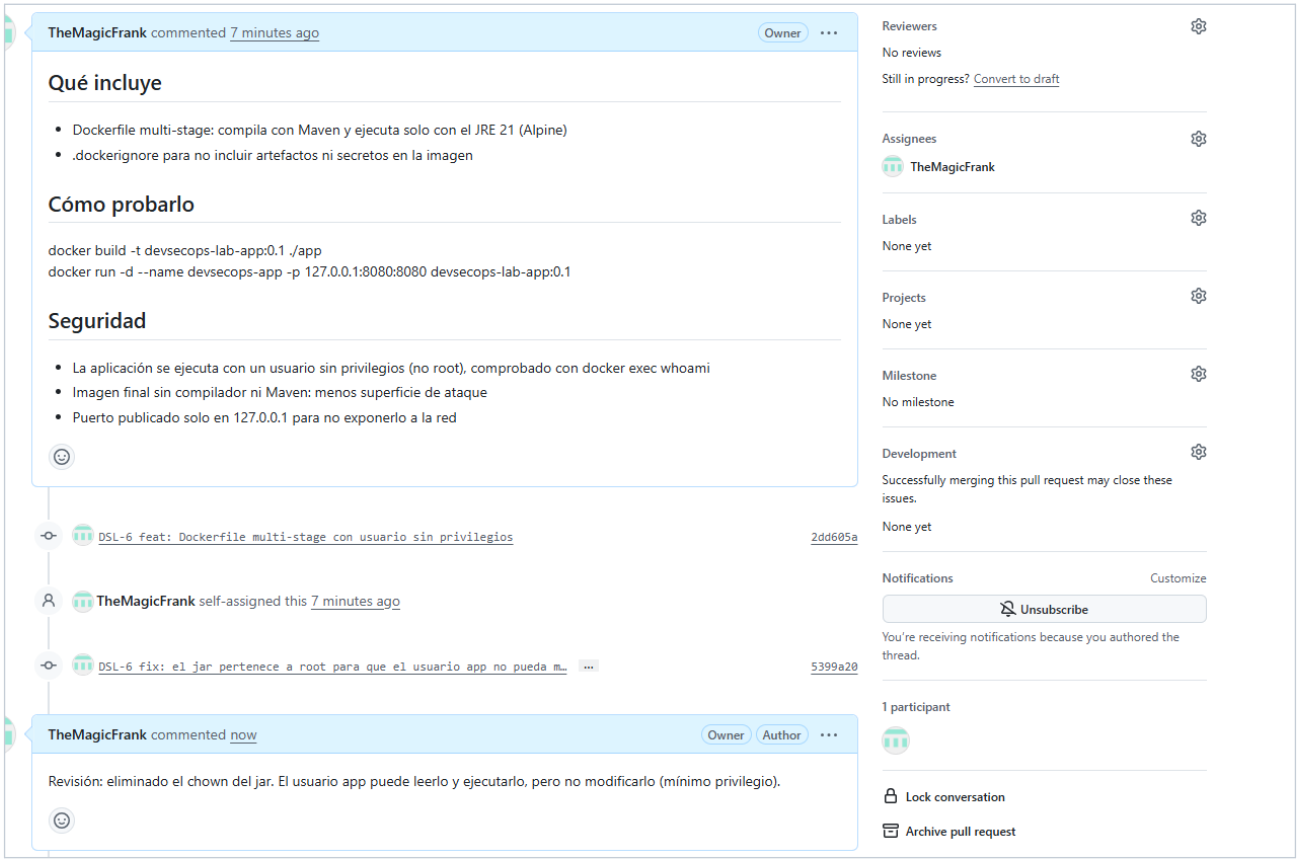


Figura 42. *Comentario de revisión en la PR explicando la corrección.*

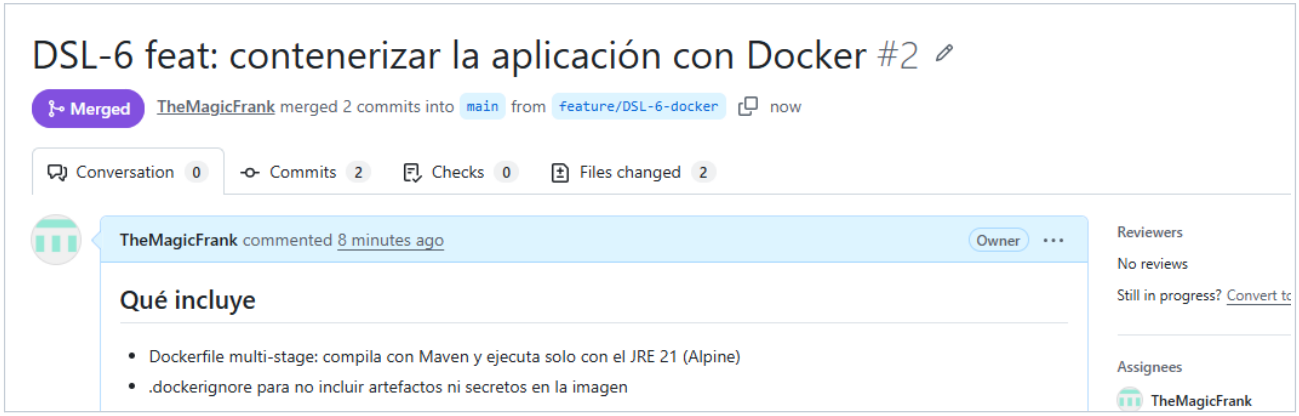


Figura 43. *PR #2 fusionada.*

5. Cierre del Sprint 1 y lección de trazabilidad

Al planificar el Sprint 1 se metieron por error las 9 tareas y después se sacaron 7. Lejos de ocultarlo, se deja como ejemplo: el burndown muestra el pico hasta 9 y el registro de cambios de alcance recoge cada movimiento. Es justo para lo que sirve la trazabilidad de Jira.

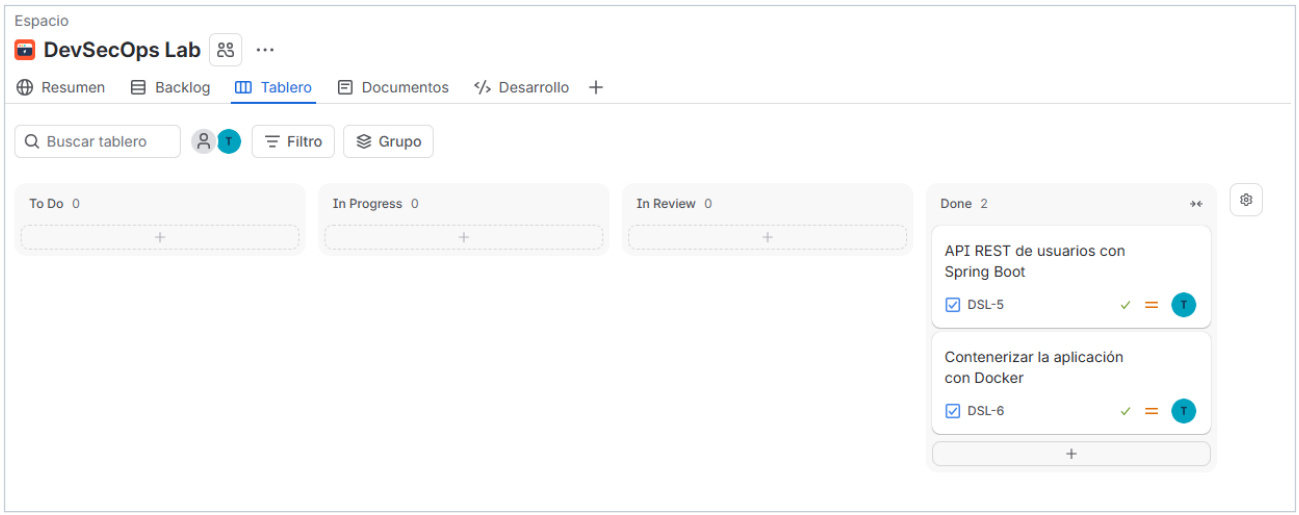


Figura 44. *Sprint 1 con DSL-5 y DSL-6 en Done.*

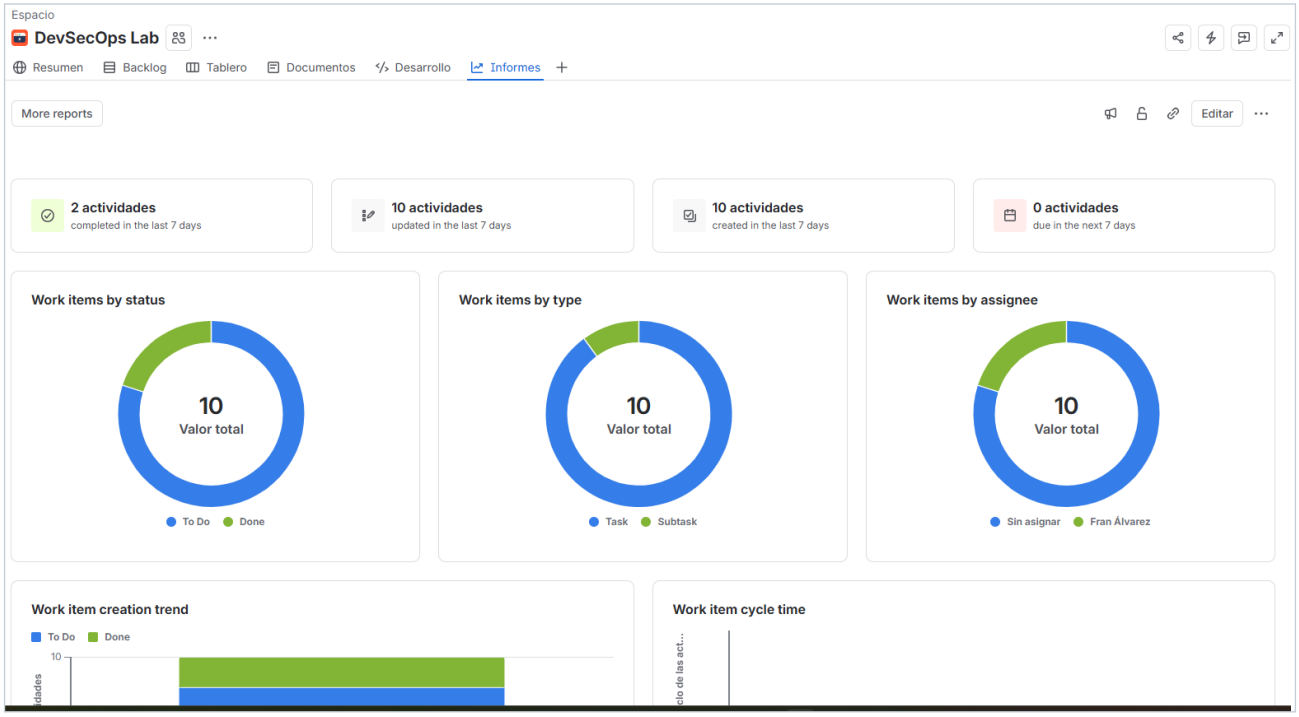


Figura 45. *Panel de informes del espacio.*

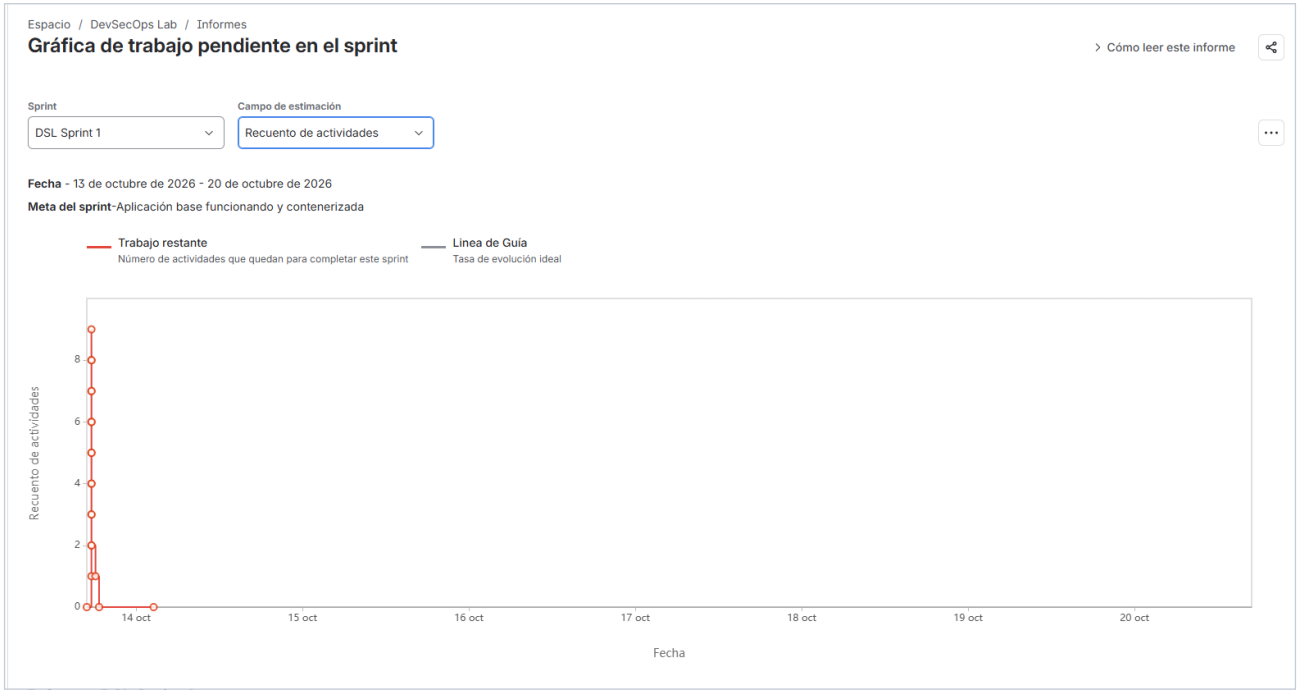


Figura 46. *Burndown del Sprint 1, con el pico causado por el cambio de alcance.*

Informe: DSL Sprint 1

* Actividad añadida tras el inicio del sprint

Registro de cambios del alcance

Ver en el navegador de todas las actividades

Fecha	Clave	Resumen	Tipo de actividad	Epic	Detalles del cambio de alcance	Cambio en la estimación
2026-10-13	DSL-5	API REST de usuarios con Spring Boot	Tarea		Actividad añadida al sprint	1
2026-10-13	DSL-7	Pipeline CI en Jenkins: build y tests	Tarea		Actividad añadida al sprint	1
2026-10-13	DSL-6	Contenerizar la aplicación con Docker	Tarea		Actividad añadida al sprint	1
2026-10-13	DSL-8	Análisis SAST con SonarQube y quality gate	Tarea		Actividad añadida al sprint	1
2026-10-13	DSL-10	Detección de secretos con Gitleaks	Tarea		Actividad añadida al sprint	1
2026-10-13	DSL-9	Escaneo de vulnerabilidades de la imagen con Trivy	Tarea		Actividad añadida al sprint	1
2026-10-13	DSL-13	Documentación: README y guía del laboratorio	Tarea		Actividad añadida al sprint	1
2026-10-13	DSL-11	Detectar y corregir vulnerabilidad de inyección SQL	Tarea		Actividad añadida al sprint	1
2026-10-13	DSL-12	Pipeline equivalente en GitHub Actions	Tarea		Actividad añadida al sprint	1
2026-10-13	DSL-7	Pipeline CI en Jenkins: build y tests	Tarea		Actividad eliminada del sprint	-1
2026-10-13	DSL-10	Detección de secretos con Gitleaks	Tarea		Actividad eliminada del sprint	-1
2026-10-13	DSL-9	Escaneo de vulnerabilidades de la imagen con Trivy	Tarea		Actividad eliminada del sprint	-1
2026-10-13	DSL-8	Análisis SAST con SonarQube y quality gate	Tarea		Actividad eliminada del sprint	-1
2026-10-13	DSL-11	Detectar y corregir vulnerabilidad de inyección SQL	Tarea		Actividad eliminada del sprint	-1

Figura 47. *Registro de cambios de alcance del Sprint 1.*

6. Sprint 2 · DSL-7 y DSL-8: Jenkins y SonarQube

La infraestructura de CI se levanta con **docker-compose**: Jenkins (LTS con JDK 21), SonarQube Community y PostgreSQL. Las contraseñas no están en el código: se leen de un archivo `.env` que Git ignora, y al repositorio solo sube la plantilla `.env.example`. Jenkins y SonarQube se publican solo en 127.0.0.1 y la base de datos no expone ningún puerto.

```
PS> cd $HOME\proyectos\devsecops-lab
PS> @"
>> # Copia este archivo como .env y pon tus propios valores.
>> # El archivo .env NUNCA se sube a Git.
>> SONAR_DB_USER=sonar
>> SONAR_DB_PASSWORD=cambia_esto_por_una_contrasena_larga
>> "@ | Set-Content .env.example
PS> Get-Content .env.example
# Copia este archivo como .env y pon tus propios valores.
# El archivo .env NUNCA se sube a Git.
SONAR_DB_USER=sonar
SONAR_DB_PASSWORD=cambia_esto_por_una_contrasena_larga
PS>
```

Figura 48. *Plantilla .env.example con valores de ejemplo.*

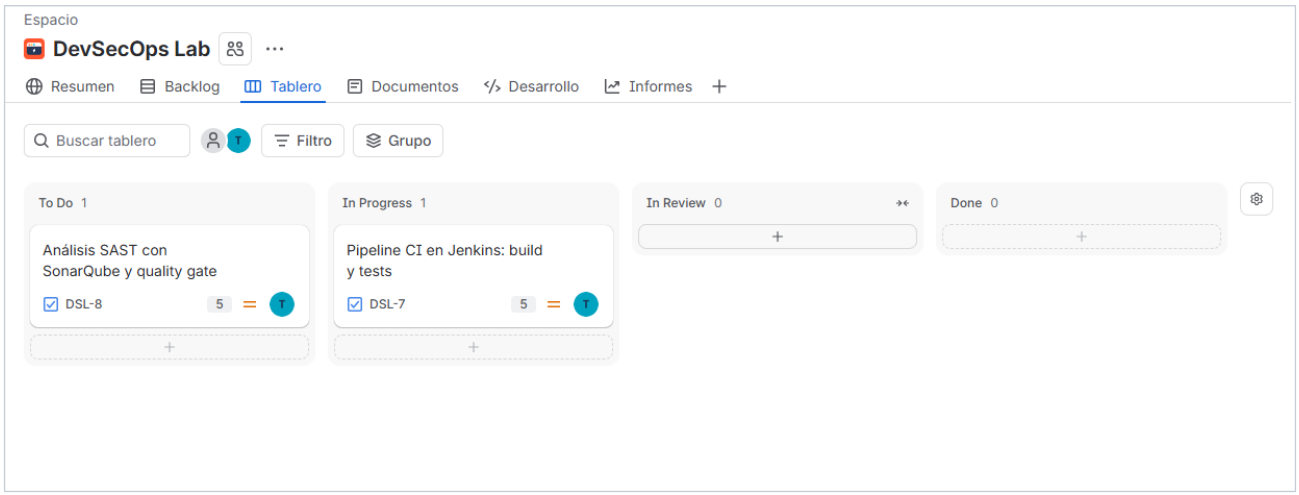


Figura 49. *Sprint 2 con DSL-7 en curso; 5 puntos de historia por tarea.*

```
PS> function prompt { "PS> " }
PS> cd $HOME\proyectos\devsecops-lab
PS> git checkout main
Already on 'main'
Your branch is up to date with 'origin/main'.
PS> git pull
Enter passphrase for key [redacted]:
Already up to date.
PS> git checkout -b feature/DSL-7-infra-ci
Switched to a new branch 'feature/DSL-7-infra-ci'
PS> Move-Item $HOME\Downloads\docker-compose.yml .
PS> dir


    Directorio: [redacted]\devsecops-lab


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        13/10/2026     18:42                app
-a----        14/10/2026      2:45            173 .env.example
-a----        10/10/2026     22:38            407 .gitignore
-a----        14/10/2026      2:43           1545 docker-compose.yml
-a----        10/10/2026     23:31           1091 LICENSE
-a----        10/10/2026     22:38            110 README.md


PS>
```

Figura 50. *Rama feature/DSL-7-infra-ci y docker-compose.yml en la raíz.*

```
PS> Add-Content .gitignore "`n# Secretos locales: nunca se suben al repo`n.env"
PS> Copy-Item .env.example .env
PS> $pass = -join ((48..57)+(65..90)+(97..122) | Get-Random -Count 32 | ForEach-Object {[char]$_})
PS> (Get-Content .env) -replace 'cambia_esto.*', $pass | Set-Content .env
PS> git check-ignore -v .env
.gitignore:20:.env      .env
PS> git status
On branch feature/DSL-7-infra-ci
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   .gitignore

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .env.example
        docker-compose.yml

no changes added to commit (use "git add" and/or "git commit -a")
PS>
```

Figura 51. *Prueba de que el .env no llega al repositorio: git check-ignore y git status.*

```
PS> docker compose ps
NAME           IMAGE                       COMMAND                  SERVICE     CREATED              STATUS
                PORTS
jenkins        jenkins/jenkins:lts-jdk21   "/usr/bin/tini -- /u…"   jenkins     About a minute ago   Up About a
 minute             127.0.0.1:8081->8080/tcp
sonarqube      sonarqube:community         "/opt/sonarqube/dock…"   sonarqube   About a minute ago   Up About a
 minute             127.0.0.1:9000->9000/tcp
sonarqube-db   postgres:16-alpine          "docker-entrypoint.s…"   db          About a minute ago   Up About a
 minute (healthy)   5432/tcp
PS>
```

Figura 52. *Los tres servicios en marcha; PostgreSQL sin puerto publicado.*

SonarQube

Primer acceso cambiando la contraseña por defecto admin/admin. El proyecto se crea en local en vez de importarlo desde GitHub, para no darle a SonarQube acceso a la cuenta. El token para Jenkins es de tipo **Project Analysis**, limitado a este proyecto y con caducidad de 90 días.

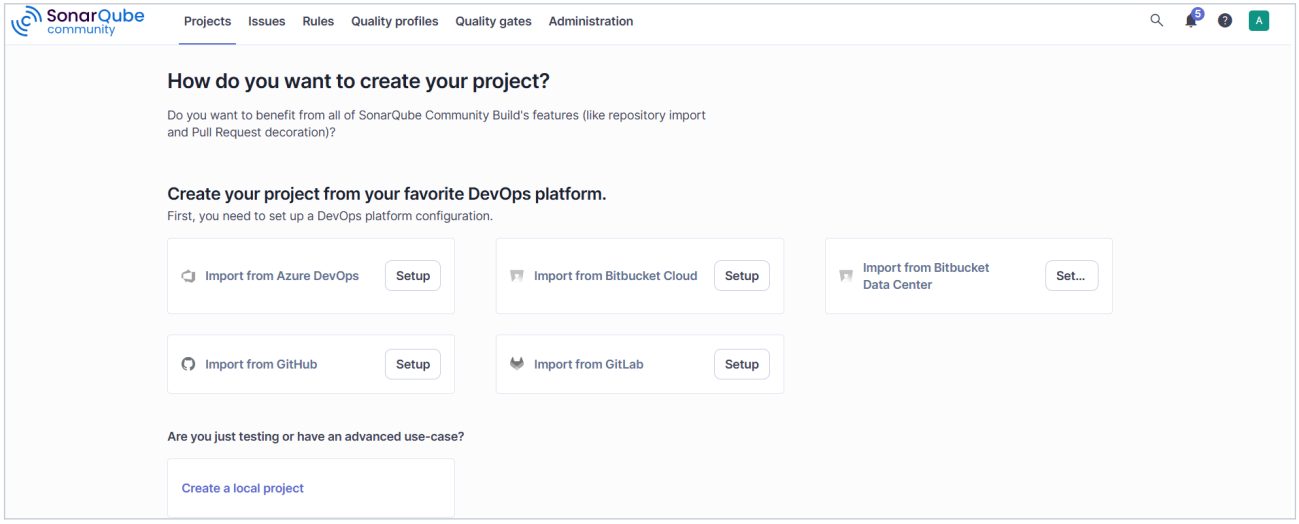


Figura 53. *SonarQube tras el primer acceso.*

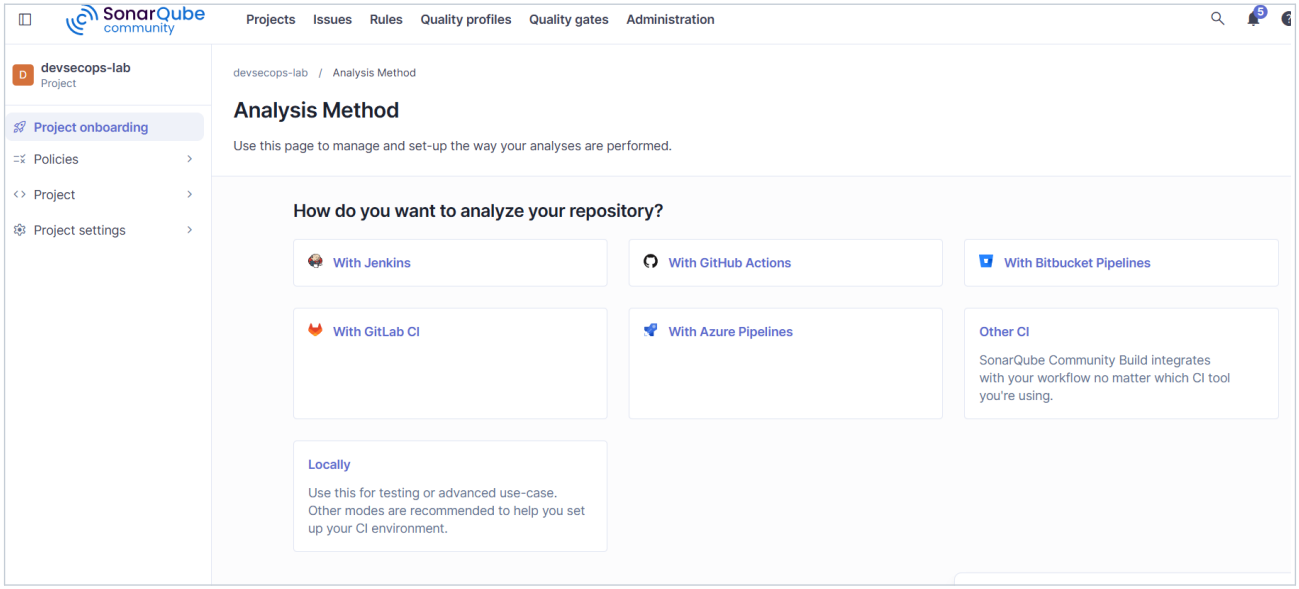


Figura 54. *Proyecto devsecops-lab creado en local.*

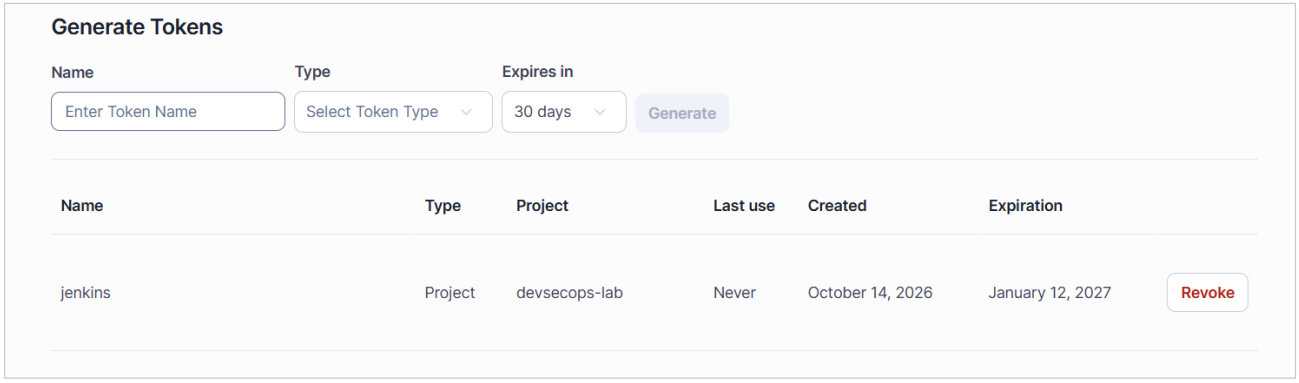


Figura 55. *Token «jenkins» de tipo Project, sin el valor visible.*

Jenkins

El usuario administrador no se llama «admin», que es lo primero que prueba un atacante. El token de SonarQube se guarda como credencial cifrada de Jenkins; después se borraron las copias locales y el historial del portapapeles. Los avisos de Jenkins se documentan como riesgos conocidos (apartado 13).

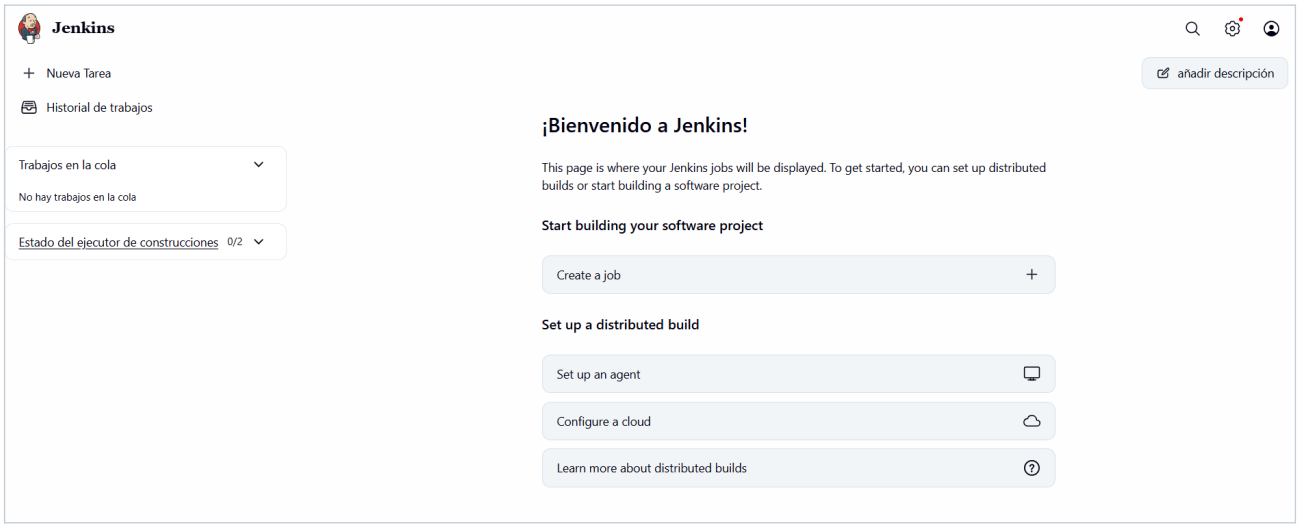


Figura 56. *Jenkins listo tras la configuración inicial.*

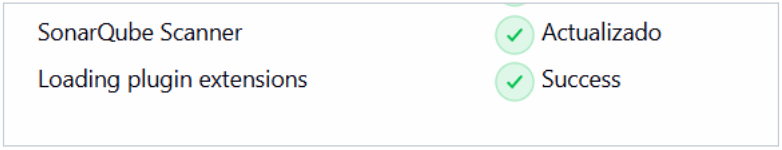


Figura 57. *Plugin SonarQube Scanner instalado.*

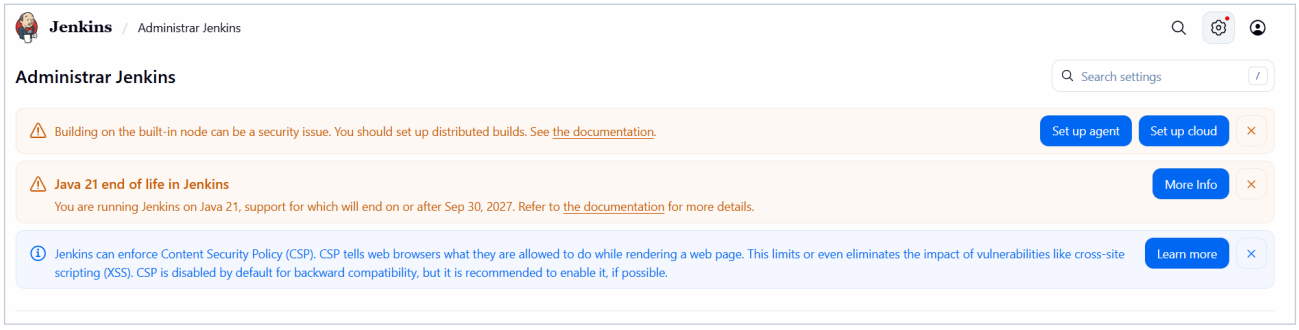


Figura 58. *Avisos de Jenkins: ejecución en el nodo principal, fin de soporte de Java 21 y CSP desactivado.*

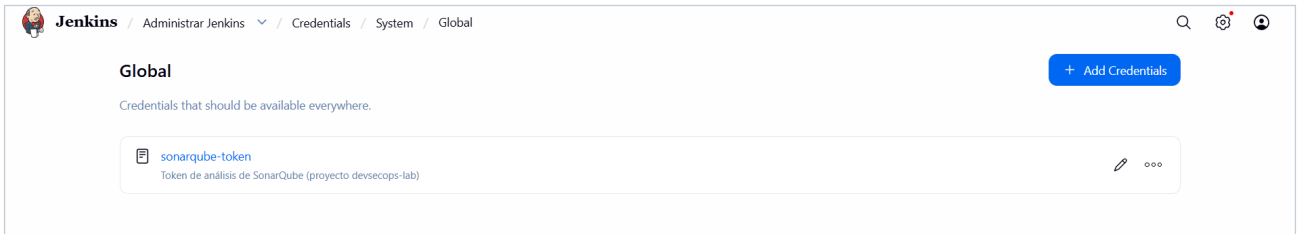


Figura 59. *Credencial sonarqube-token, con el secreto oculto.*

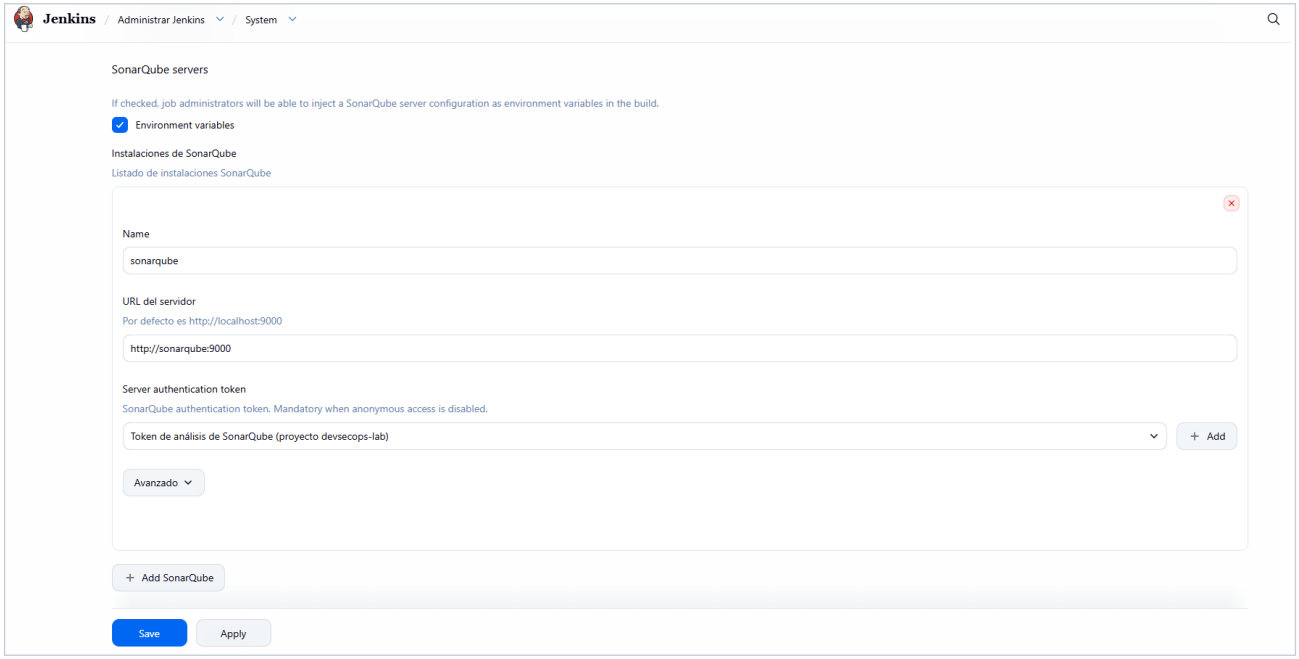


Figura 60. *Servidor SonarQube configurado con el nombre interno del servicio, http://sonarqube:9000.*

Bloqueo de seguridad y control compensatorio. SonarQube rechazó el webhook hacia Jenkins por su protección anti-SSRF, que impide enviar peticiones a direcciones de red internas. Como Jenkins y SonarQube se comunican por la red interna de Docker, se desactivó esa validación y se compensó con un **secreto HMAC**: SonarQube firma cada aviso y Jenkins solo acepta los que llevan la firma correcta.

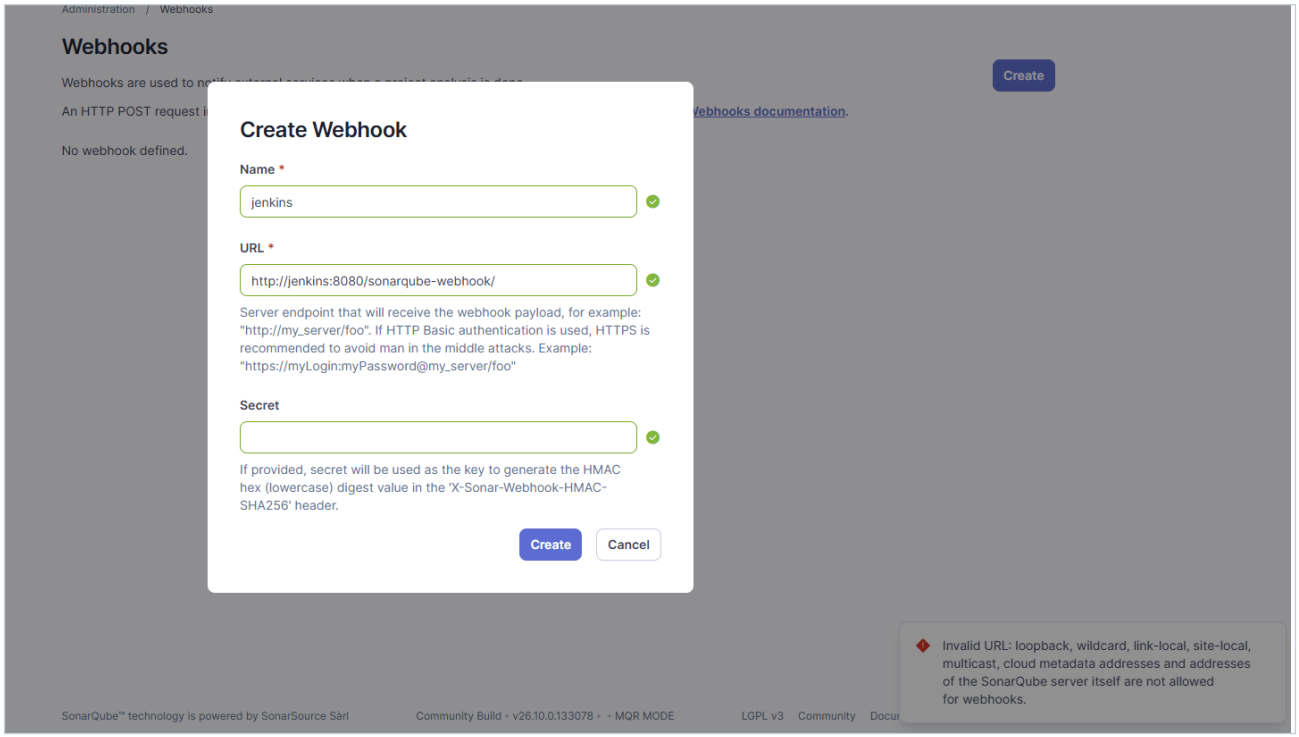


Figura 61. *Webhook rechazado por la protección anti-SSRF.*

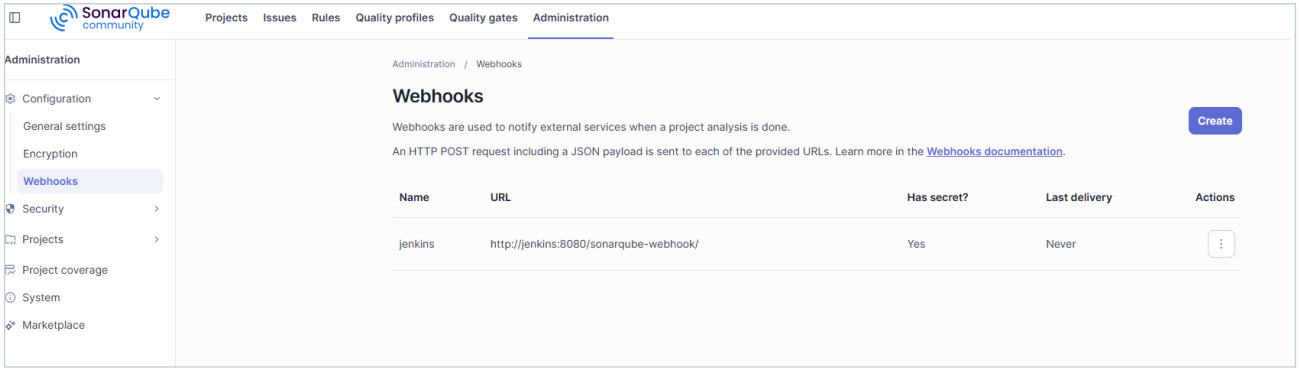


Figura 62. *Webhook creado con secreto.*

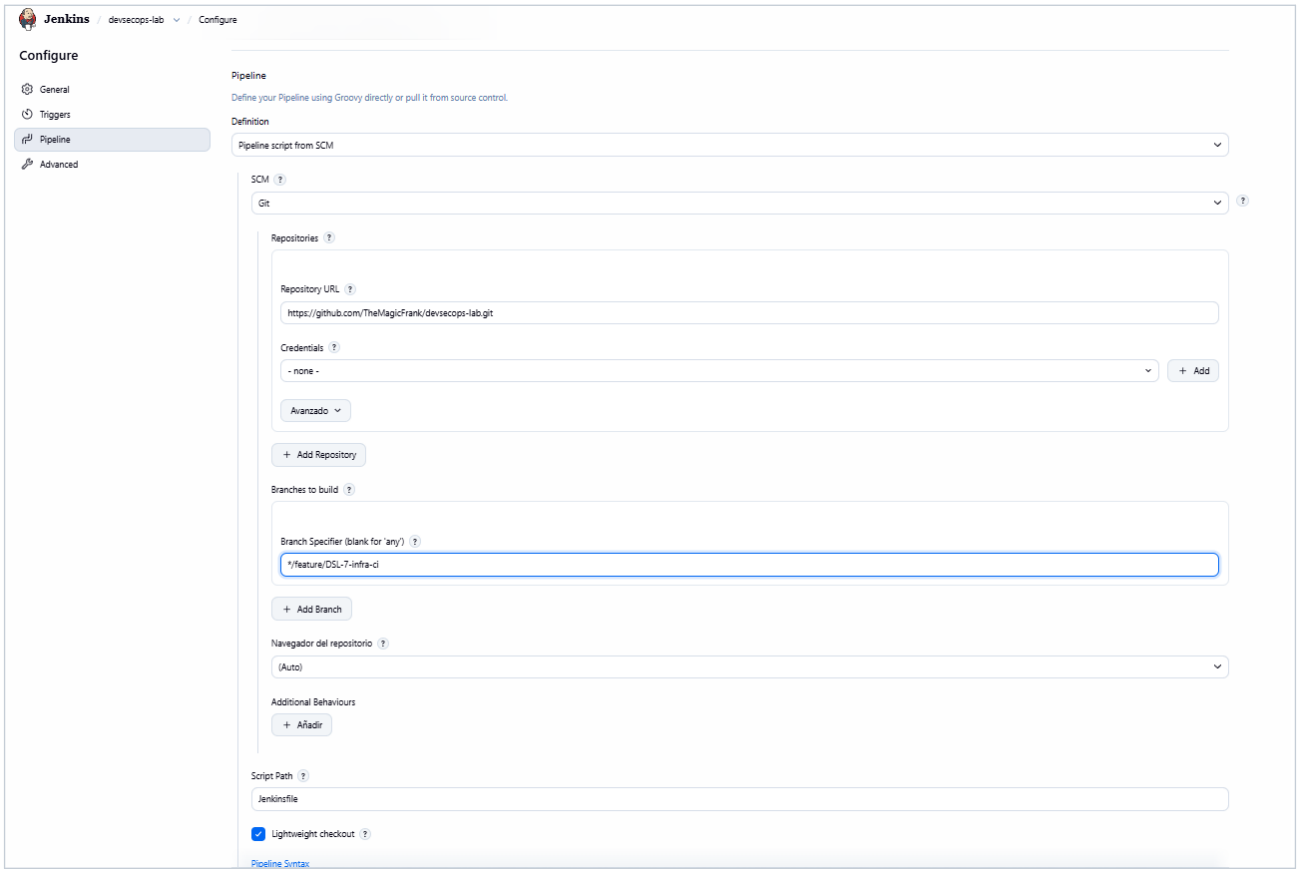


Figura 63. *Trabajo Pipeline leyendo el Jenkinsfile del repositorio público, sin credenciales.*

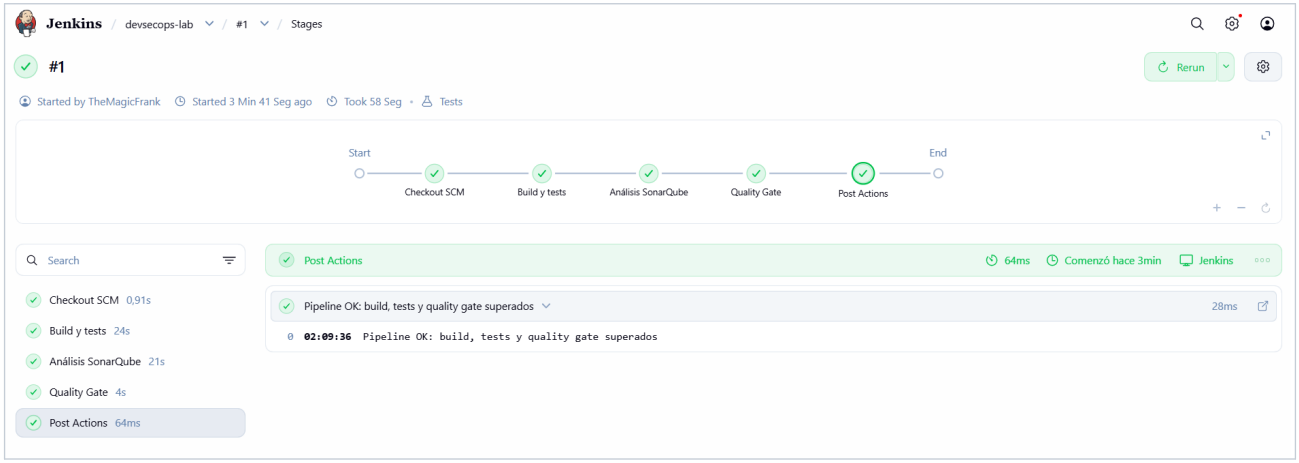


Figura 64. *Primera ejecución en verde: build y tests, análisis SonarQube y quality gate.*

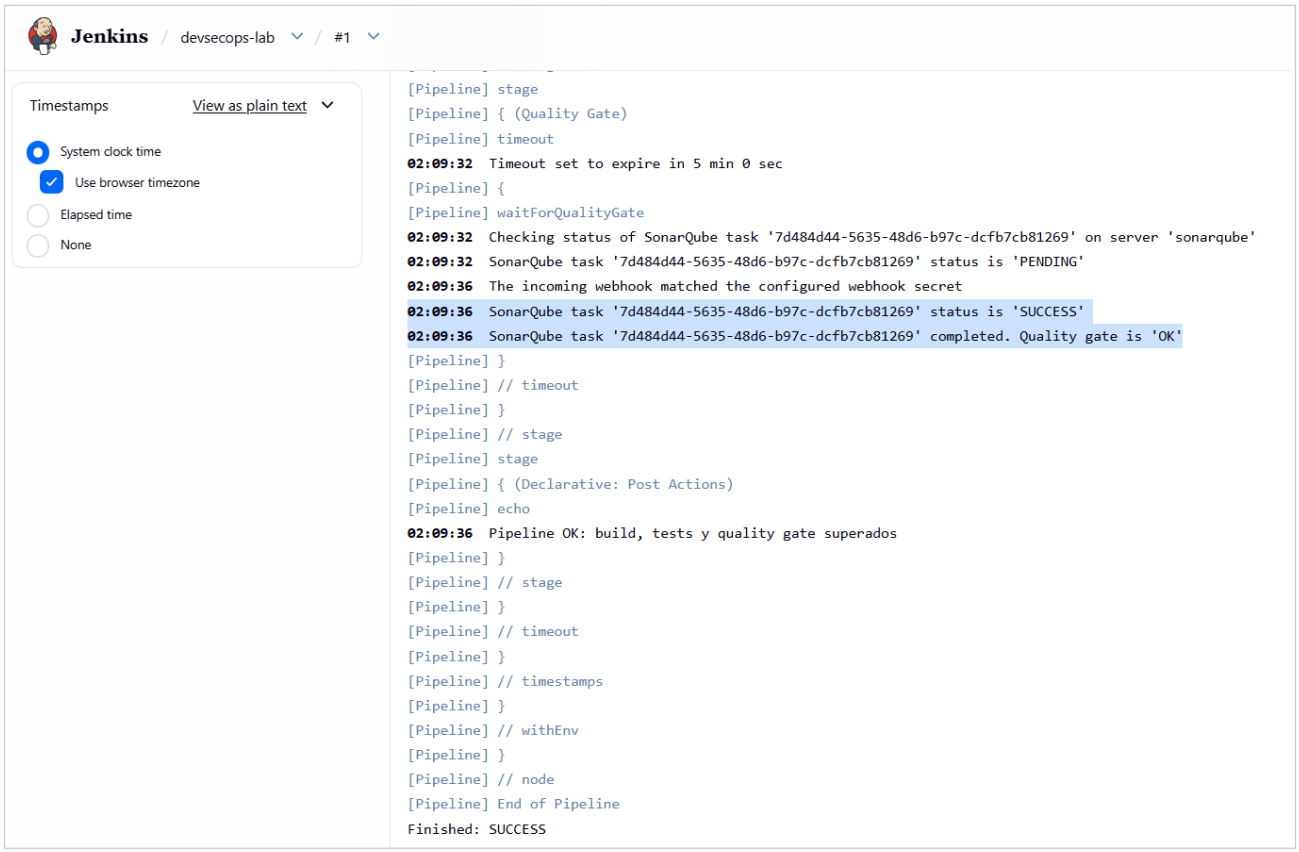


Figura 65. *El webhook firmado funciona: «The incoming webhook matched the configured webhook secret» y quality gate OK.*

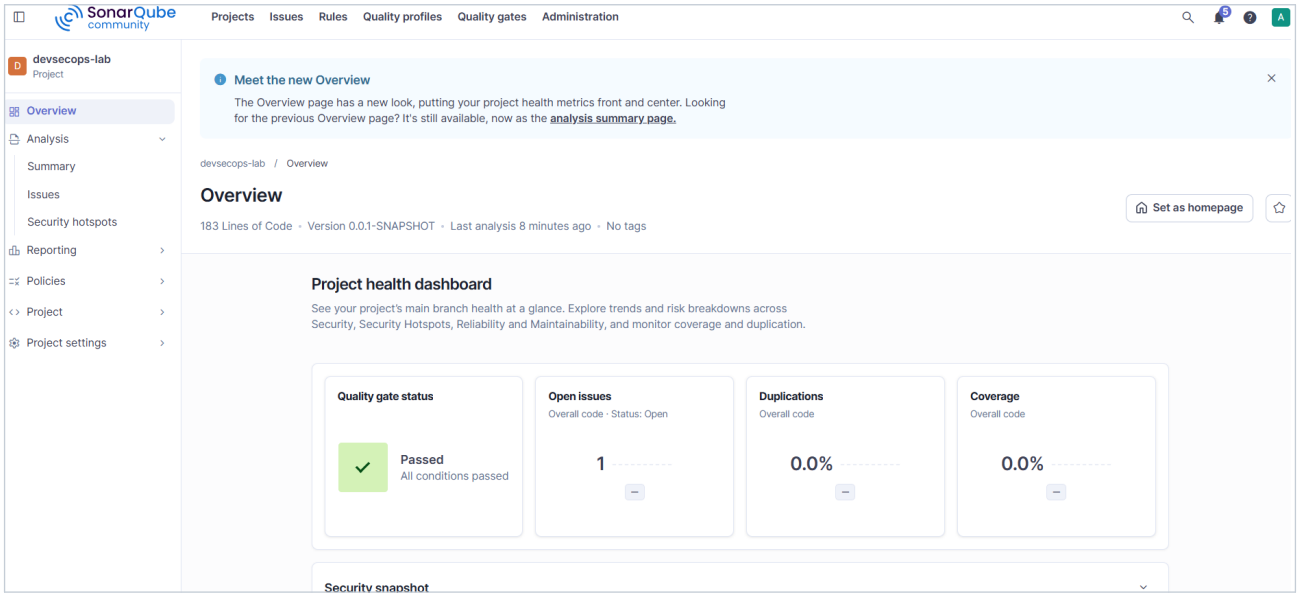


Figura 66. *Primer análisis en SonarQube: quality gate superado.*

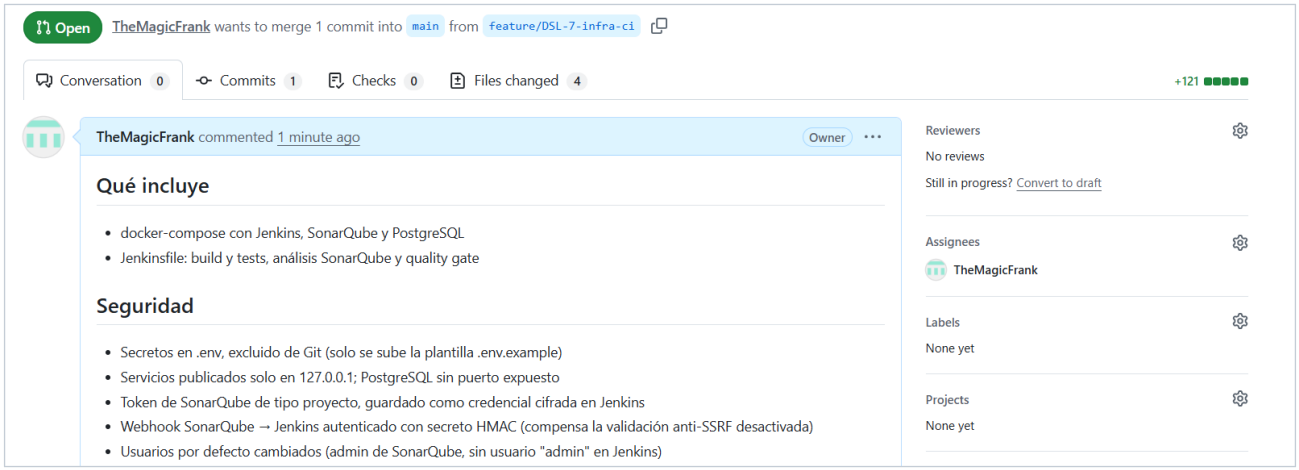


Figura 67. *Pull Request #3 abierta (DSL-7 y DSL-8).*



Figura 68. *PR #3 fusionada.*

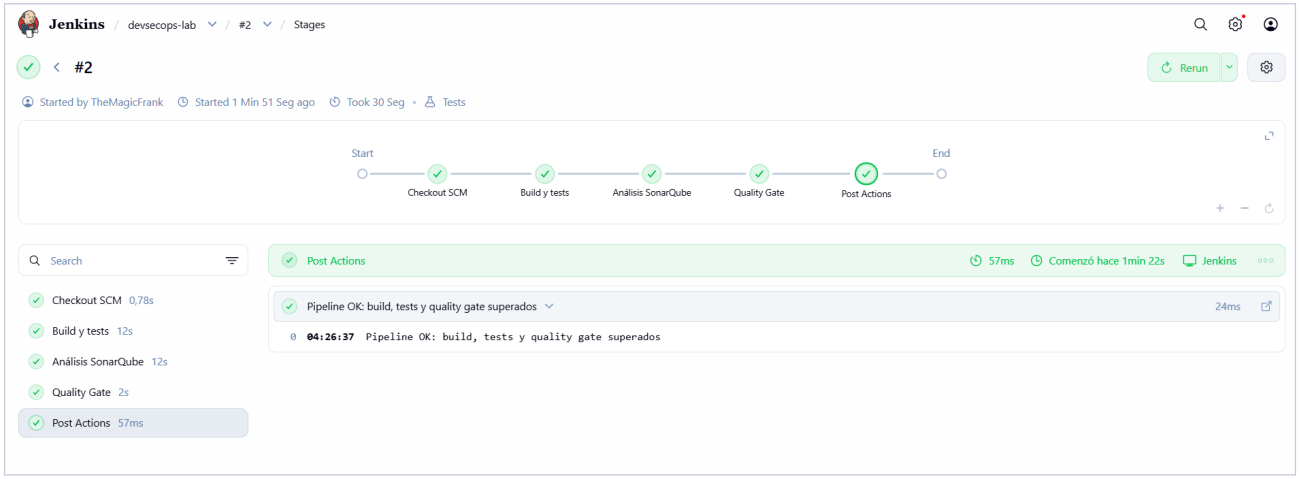


Figura 69. *Pipeline en verde desde main.*

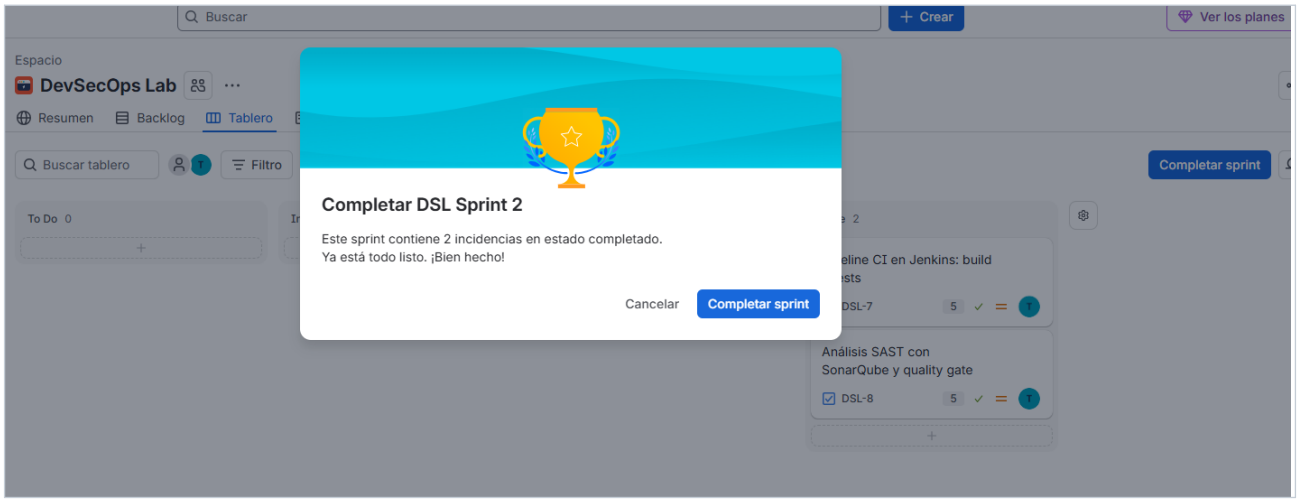


Figura 70. *Cierre del Sprint 2.*

7. Sprint 3 · DSL-14: vulnerabilidad real de mass assignment

El primer análisis de SonarQube encontró una vulnerabilidad **High real**: el controlador recibía la entidad JPA directamente, así que un cliente podía enviar un "id" y sobrescribir otro usuario. El quality gate pasó igualmente porque por defecto solo evalúa el código nuevo, y en el primer análisis todo es código existente. Se registró como bug, se explotó en local para validarla, se corrigió y se verificó.

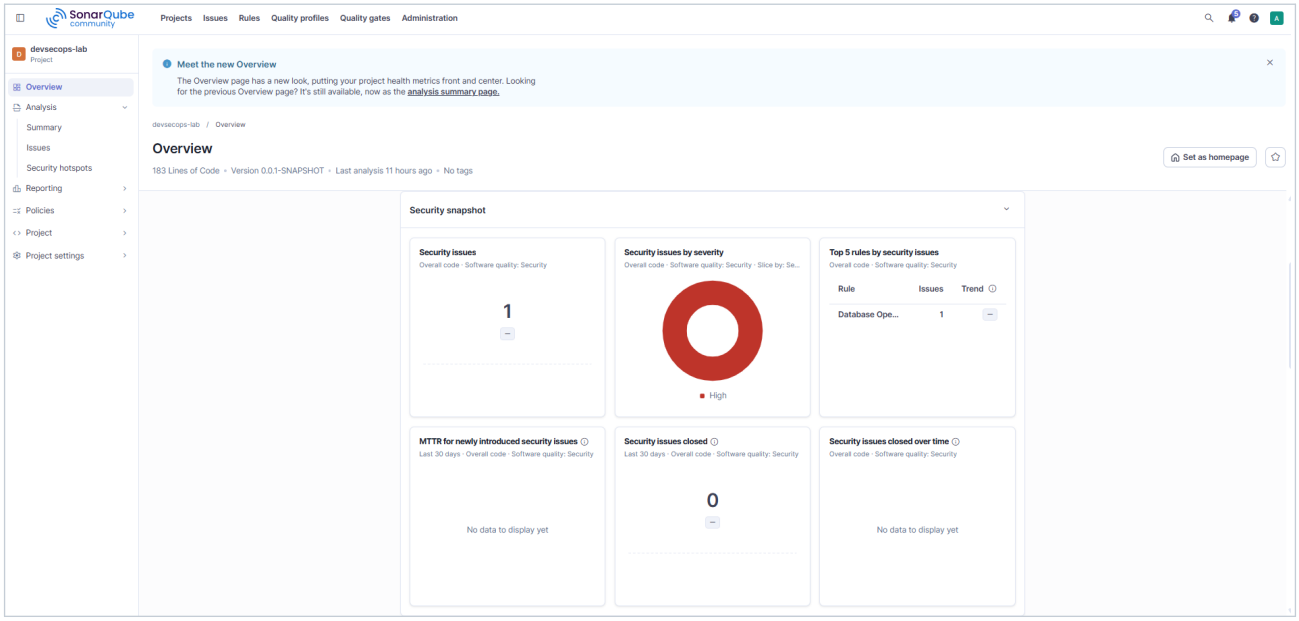


Figura 71. *SonarQube detecta 1 vulnerabilidad de severidad High.*



Figura 72. *Registro del bug DSL-14 con tipo Error.*

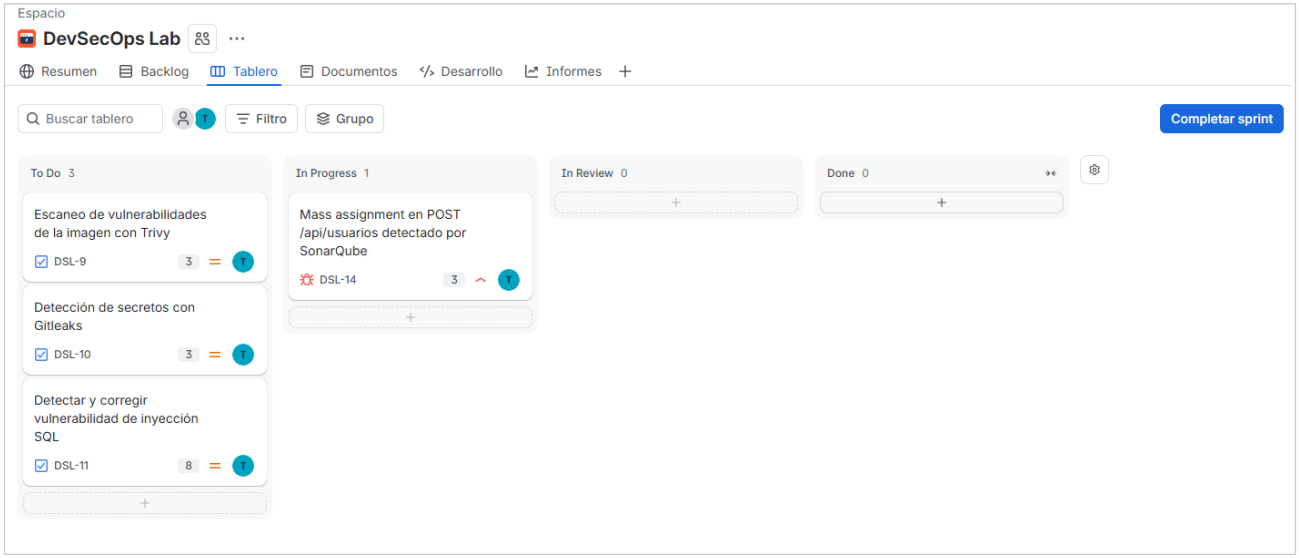


Figura 73. *Sprint 3 con DSL-14 en curso.*

```
PS> Invoke-RestMethod -Method Post -Uri http://localhost:8080/api/usuarios -C
ontentType "application/json" -Body '{"nombre":"Ana","email":"ana@example.com
"}'

nombre email           id
------ -----           --
Ana    ana@example.com  1


PS> Invoke-RestMethod -Method Post -Uri http://localhost:8080/api/usuarios -C
ontentType "application/json" -Body '{"nombre":"Luis","email":"luis@example.c
om"}'

nombre email           id
------ -----           --
Luis   luis@example.com  2


PS> Invoke-RestMethod http://localhost:8080/api/usuarios

nombre email           id
------ -----           --
Ana    ana@example.com   1
Luis   luis@example.com  2
```

Figura 74. *Estado inicial: Ana (id 1) y Luis (id 2).*

```
PS> Invoke-RestMethod -Method Post -Uri http://localhost:8080/api/usuarios -C
ontentType "application/json" -Body '{"id":1,"nombre":"Atacante","email":"ata
cante@example.com"}'

nombre   email               id
------   -----               --
Atacante atacante@example.com  1


PS> Invoke-RestMethod http://localhost:8080/api/usuarios

nombre   email               id
------   -----               --
Atacante atacante@example.com  1
Luis     luis@example.com      2


PS>
```

Figura 75. *Exploit: un POST con "id":1 sustituye a Ana por «Atacante».*

```
[INFO] Results:
[INFO]
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------
--
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------
--
[INFO] Total time:  18.022 s
```

Figura 76. *Corrección con un DTO (UsuarioRequest) y test de regresión: 6 tests en verde.*

```
PS> Invoke-RestMethod -Method Post -Uri http://localhost:8080/api/usuarios -C
ontentType "application/json" -Body '{"id":1,"nombre":"Atacante","email":"ata
cante@example.com"}'

nombre   email               id
------   -----               --
Atacante atacante@example.com  3


PS> Invoke-RestMethod http://localhost:8080/api/usuarios

nombre   email               id
------   -----               --
Ana      ana@example.com       1
Luis     luis@example.com      2
Atacante atacante@example.com  3


PS>
```

Figura 77. *El mismo ataque tras la corrección crea un usuario nuevo (id 3) y Ana sigue intacta.*

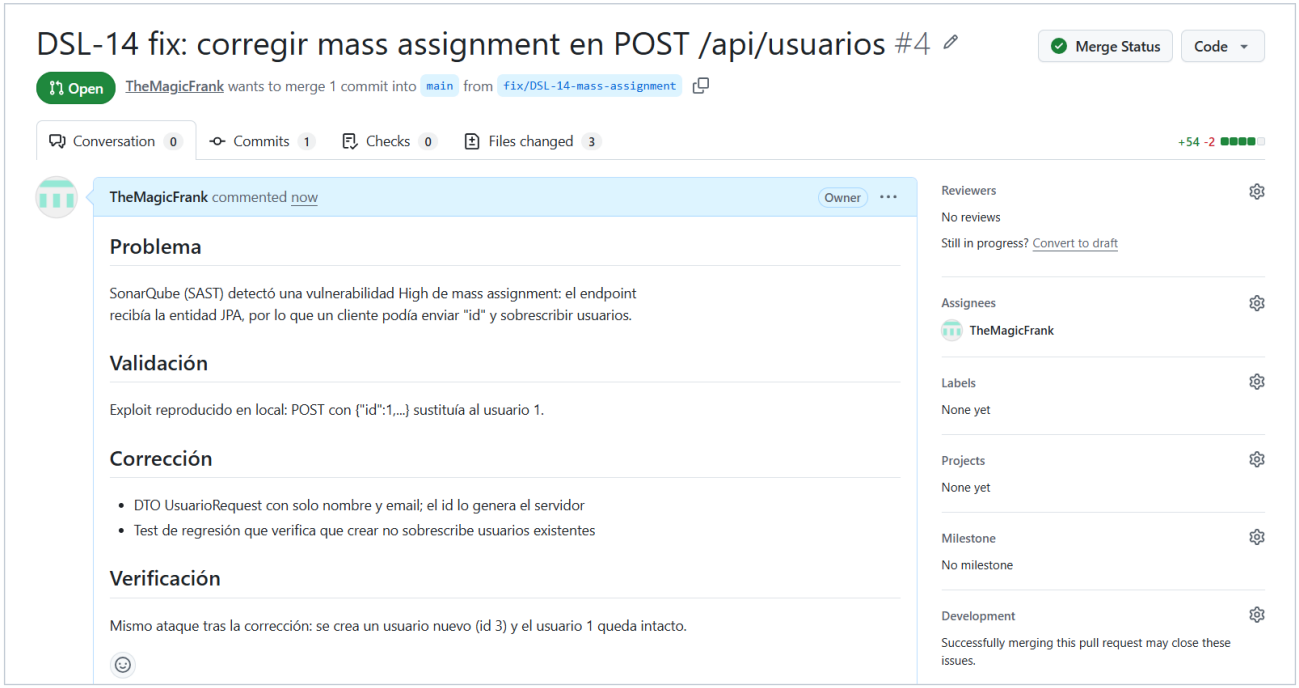


Figura 78. *Pull Request #4 con problema, validación, corrección y verificación.*

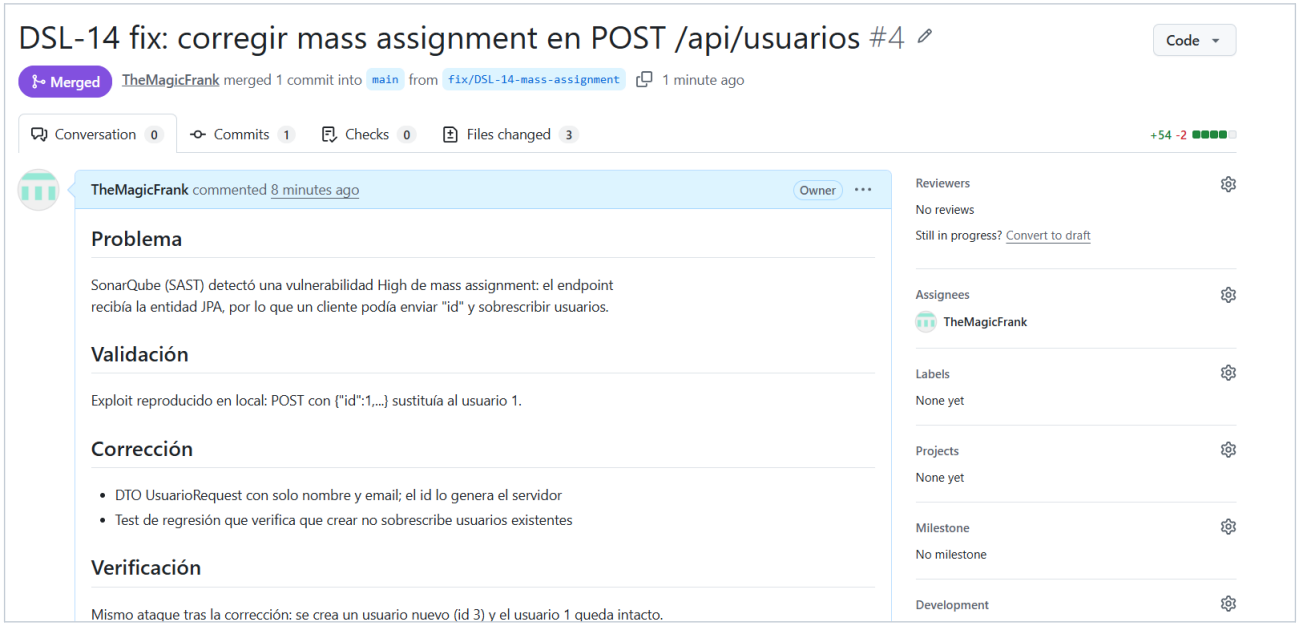


Figura 79. *PR #4 fusionada.*

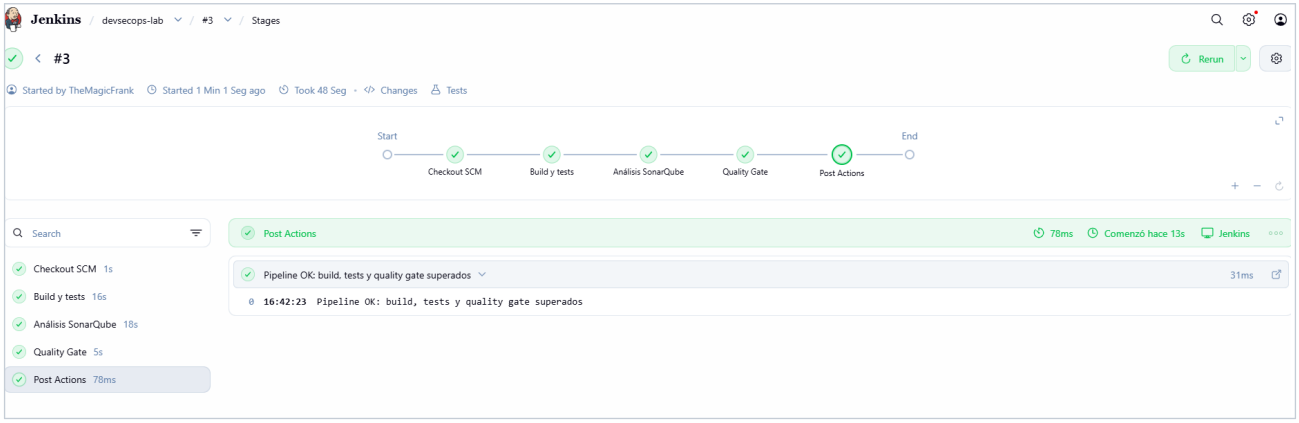


Figura 80. *Pipeline en verde con la corrección.*

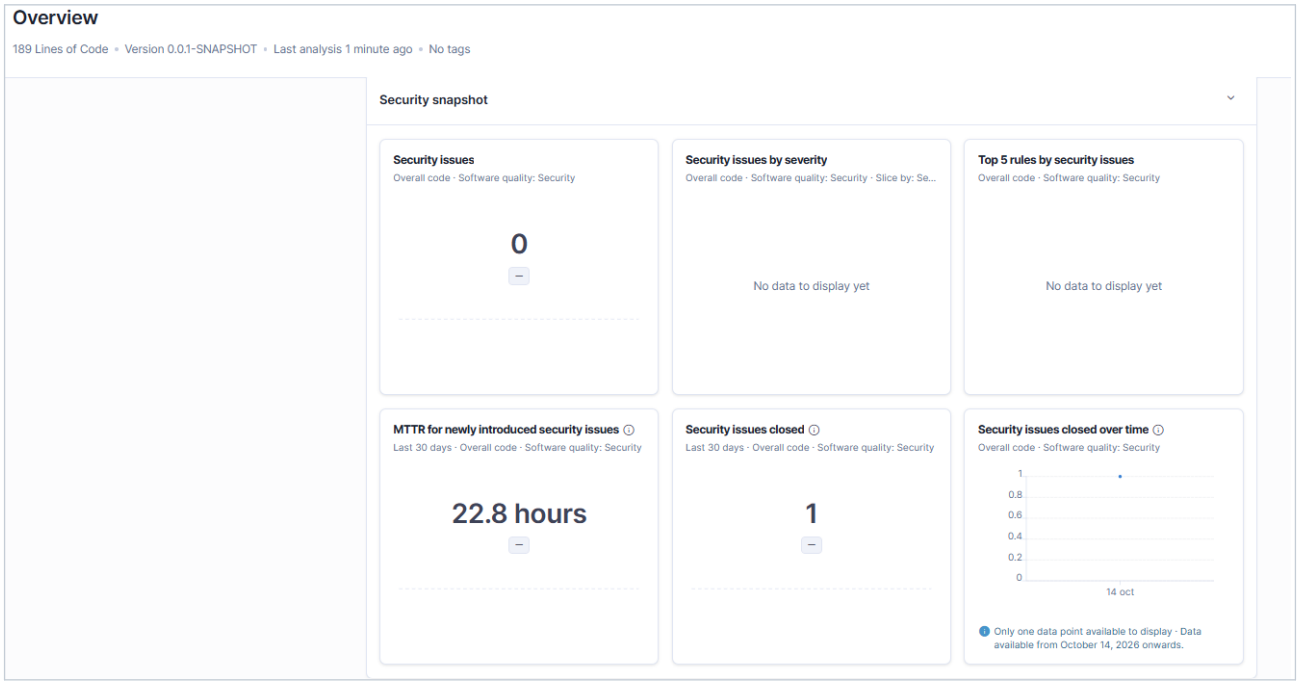


Figura 81. *SonarQube: 0 issues de seguridad y 1 cerrada.*

8. DSL-9: Trivy, imágenes fijadas y Dependabot

En marzo de 2026 Trivy sufrió un ataque a su cadena de suministro con versiones maliciosas publicadas. Por eso se aplica una **cadena de confianza**: versión fijada (0.75.0), huella SHA-256 del archivo de checksums escrita en el Jenkinsfile y, con ese archivo validado, comprobación del binario. La huella se verificó con tres fuentes: la página oficial de la release, el archivo descargado y el Jenkinsfile.



Figura 82. *Huella del archivo de checksums en la release oficial de Trivy.*

```
PS> Get-FileHash .\trivy_0.75.0_checksums.txt -Algorithm SHA256

Algorithm       Hash
---------       ----
SHA256          43A87485D39B60DAE55CB1A14725DDDD612040C0B1EC4CAA88948C07D...


PS> (Get-FileHash .\trivy_0.75.0_checksums.txt -Algorithm SHA256).Hash
43A87485D39B60DAE55CB1A14725DDDD612040C0B1EC4CAA88948C07D6C06219
PS>
```

Figura 83. *La misma huella calculada en local con Get-FileHash.*

> **Trivy bloqueó el pipeline.** El análisis SCA encontró 8 CVE: 3 CRITICAL en Tomcat 11.0.24 y 5 HIGH de denegación de servicio en Jackson 3.1.5. Spring Boot 4.1.1 ya era la última versión estable, así que se fijaron en el pom las versiones corregidas (Tomcat 11.0.25 y Jackson 3.1.7) y se comprobó con dependency:tree.

```
17:21:35  Total: 8 (HIGH: 5, CRITICAL: 3)
17:21:35
17:21:35
┌──────────────────────────────────────┬────────────────┬──────────┬────────┬───────────────────┬──────────────────────┬──────────────────────────────────────
```

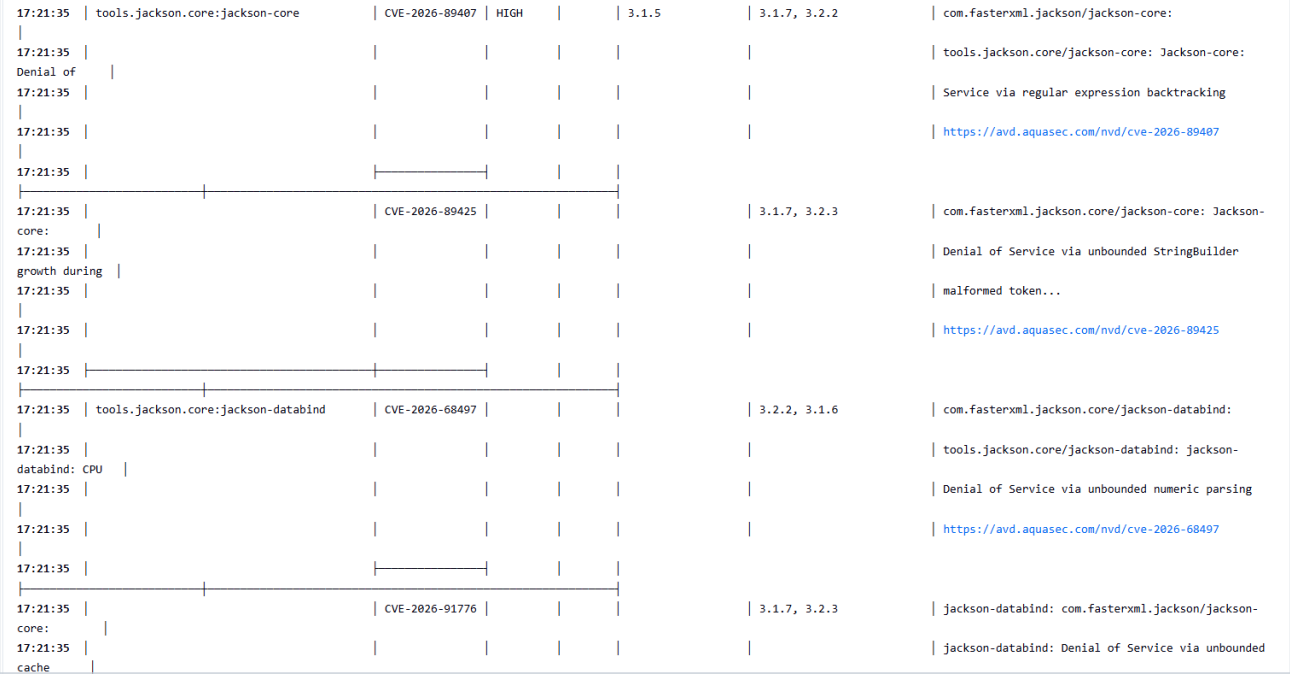


Figura 85. *Detalle de las CVE de Jackson.*

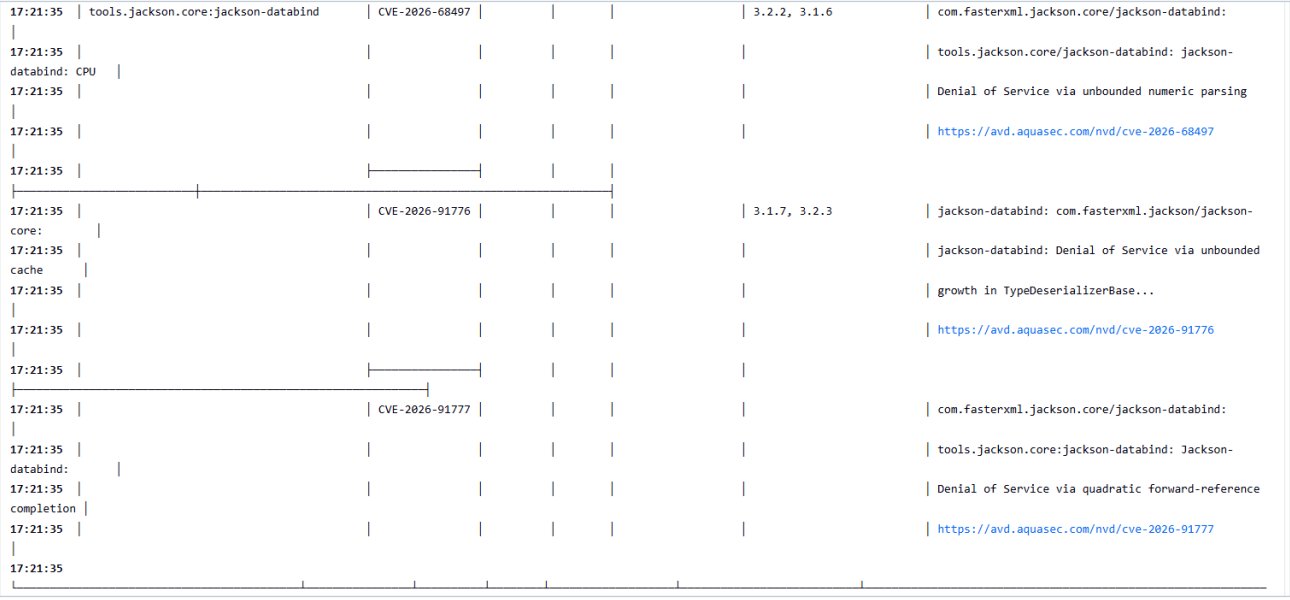


Figura 86. *Detalle de las CVE de Jackson (continuación).*

```
17:21:08  + curl -fsSLO https://github.com/aquasecurity/trivy/releases/download/v0.75.0/trivy_0.75.0_checksums.txt
17:21:08  + echo 43a87485d39b60dae55cb1a14725dddd612040c0b1ec4caa88948c07d6c06219  trivy_0.75.0_checksums.txt
17:21:08  + sha256sum -c -
17:21:08  trivy_0.75.0_checksums.txt: OK
17:21:08  + curl -fsSLO https://github.com/aquasecurity/trivy/releases/download/v0.75.0/trivy_0.75.0_Linux-64bit.tar.gz
17:21:12  + grep  trivy_0.75.0_Linux-64bit.tar.gz$ trivy_0.75.0_checksums.txt
17:21:12  + sha256sum -c -
17:21:12  trivy_0.75.0_Linux-64bit.tar.gz: OK
17:21:12  + tar -xzf trivy_0.75.0_Linux-64bit.tar.gz trivy
17:21:13  + rm -f trivy_0.75.0_Linux-64bit.tar.gz
17:21:13  + /var/jenkins_home/tools/trivy-0.75.0/trivy --version
17:21:13  Version: 0.75.0
```

Figura 87. *Instalación verificada en Jenkins: checksums OK, binario OK, versión 0.75.0.*

```
[INFO] com.themagicfrank:app:jar:0.0.1-SNAPSHOT
[INFO] \- org.springframework.boot:spring-boot-starter-webmvc:jar:4.1.1:compile
[INFO]    +- org.springframework.boot:spring-boot-starter-jackson:jar:4.1.1:compile
[INFO]    |  \- org.springframework.boot:spring-boot-jackson:jar:4.1.1:compile
[INFO]    |     \- tools.jackson.core:jackson-databind:jar:3.1.7:compile
[INFO]    |        \- tools.jackson.core:jackson-core:jar:3.1.7:compile
[INFO]    \- org.springframework.boot:spring-boot-starter-tomcat:jar:4.1.1:compile
[INFO]       \- org.springframework.boot:spring-boot-starter-tomcat-runtime:jar:4.1.1:compile
[INFO]          \- org.apache.tomcat.embed:tomcat-embed-core:jar:11.0.25:compile
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  8.623 s
```

Figura 88. *dependency:tree con Tomcat 11.0.25 y Jackson 3.1.7.*

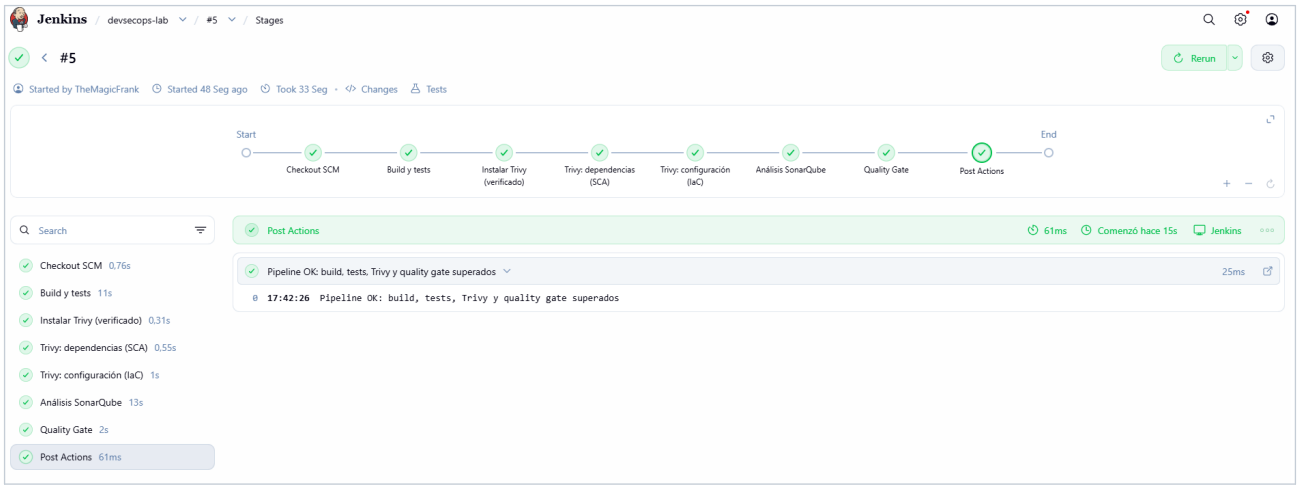


Figura 89. *Pipeline en verde con las etapas de Trivy (SCA e IaC).*

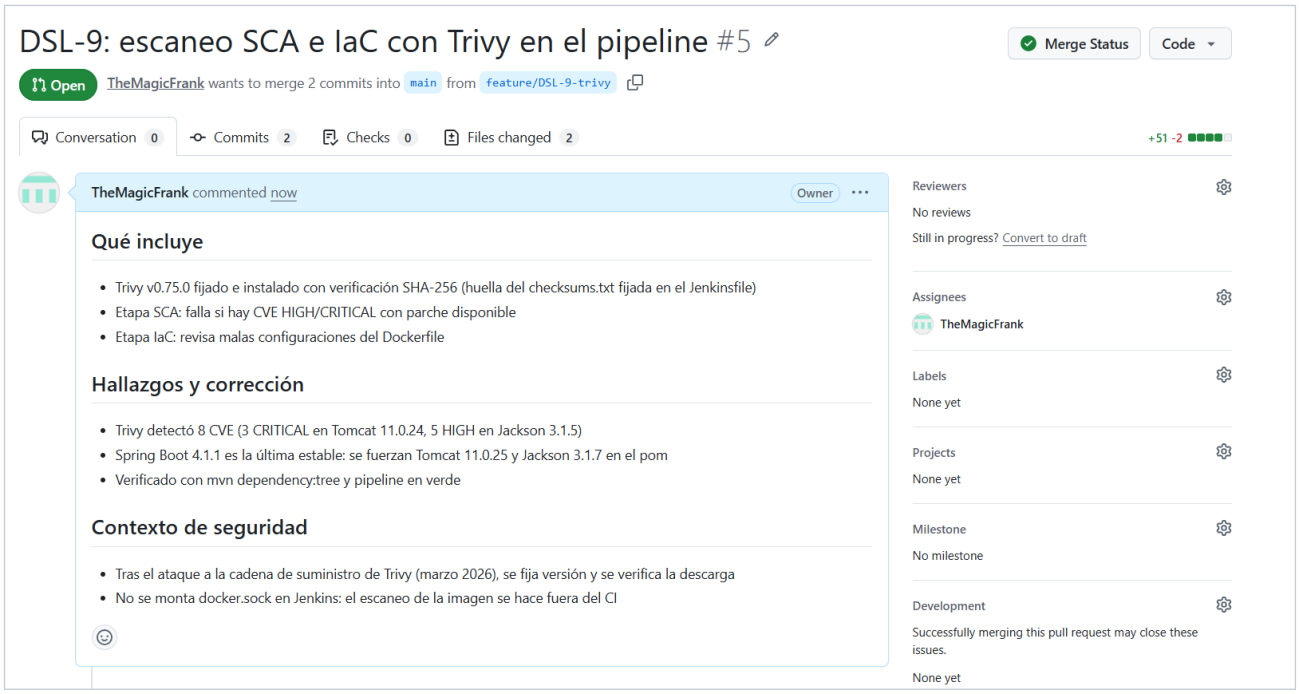


Figura 90. *Pull Request #5 abierta.*

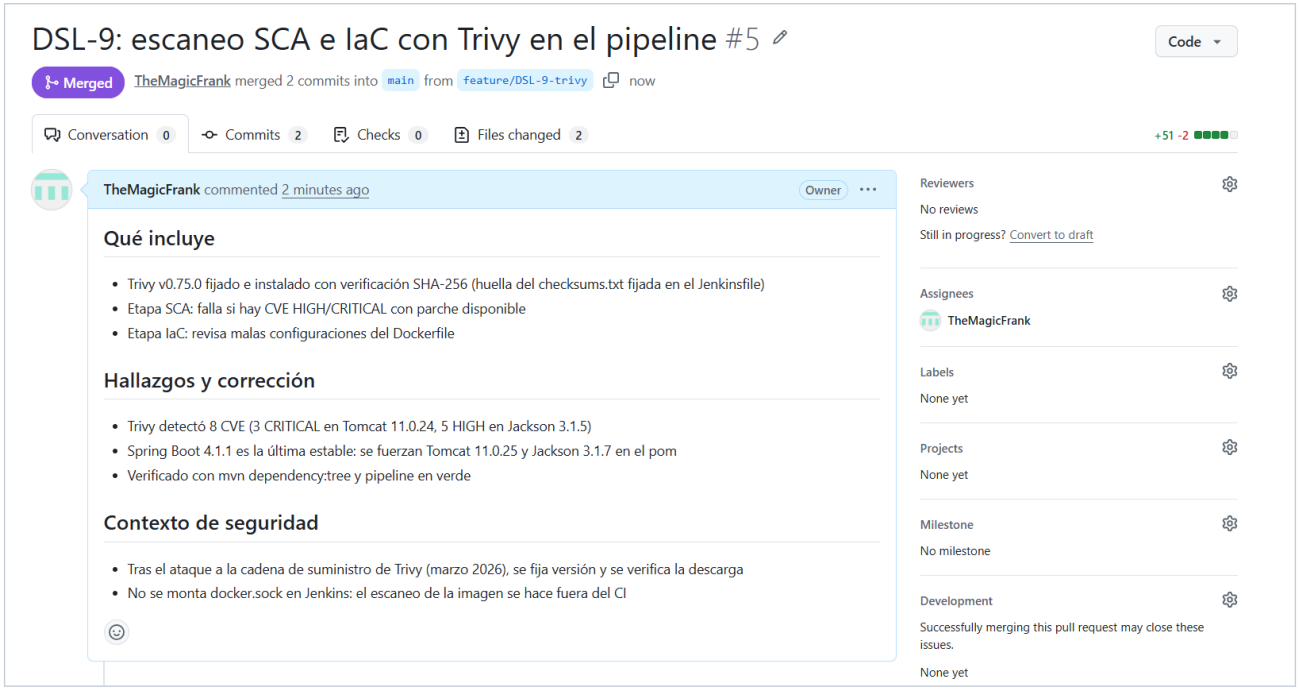


Figura 91. *PR #5 fusionada.*

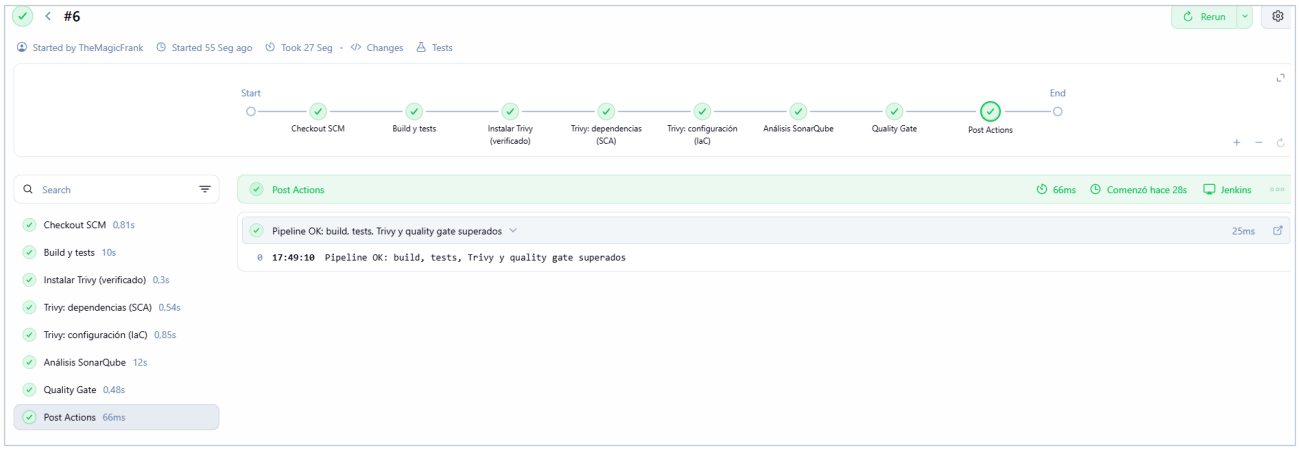


Figura 92. *Pipeline en verde desde main.*

Escaneo de la imagen e imágenes fijadas por digest

No se monta el socket de Docker en Jenkins, porque daría control total del host al contenedor. El escaneo de la imagen se hace en local y, más adelante, en GitHub Actions. Las cinco imágenes base (Maven, Temurin JRE, Jenkins, SonarQube y PostgreSQL) se fijan por **etiqueta@sha256**: una etiqueta puede cambiar, un digest identifica una imagen exacta.

```
PS> New-Item -ItemType Directory -Force $HOME\tools\trivy-0.75.0 | Out-Null
PS> cd $HOME\tools\trivy-0.75.0
PS> $v = "0.75.0"
PS> $base = "https://github.com/aquasecurity/trivy/releases/download/v$v"
PS> curl.exe -fsSLO "$base/trivy_${v}_checksums.txt"
PS> (Get-FileHash "trivy_${v}_checksums.txt" -Algorithm SHA256).Hash -eq "43A87485D39B60DAE55
CB1A14725DDDD612040C0B1EC4CAA88948C07D6C06219"
True
PS> Select-String -Path "trivy_${v}_checksums.txt" -Pattern "windows"

trivy_0.75.0_checksums.txt:22:4e43bd71a30f51aee39525f60f2b47043af77eb8df8fe082aae4372b69c666
0f  trivy_0.75.0_windows-64bit.zip


PS> $zip = "trivy_${v}_windows-64bit.zip"
PS> curl.exe -fsSLO "$base/$zip"
PS> $esperado = ((Select-String -Path "trivy_${v}_checksums.txt" -Pattern " $zip$").Line -spl
it '\s+')[0]
PS> (Get-FileHash $zip -Algorithm SHA256).Hash -eq $esperado
True
PS> Expand-Archive $zip -DestinationPath . -Force
PS> .\trivy.exe --version
Version: 0.75.0
PS>
```

Figura 93. *Trivy para Windows instalado con la misma verificación SHA-256.*

```
Report Summary

┌──────────────────────────────────┬──────┬─────────────────┬─────────┐
│              Target              │ Type │ Vulnerabilities │ Secrets │
├──────────────────────────────────┼──────┼─────────────────┼─────────┤
│ devsecops-lab-app:0.3 (alpine 3.24.2) │ alpine │        0        │    -    │
├──────────────────────────────────┼──────┼─────────────────┼─────────┤
│ app/app.jar                      │ jar  │        0        │    -    │
└──────────────────────────────────┴──────┴─────────────────┴─────────┘
Legend:
- '-': Not scanned
- '0': Clean (no security findings detected)
```

Figura 94. *Imagen 0.3: 0 vulnerabilidades HIGH/CRITICAL en Alpine y en el jar.*

```
=> [build 1/6] FROM docker.io/library/maven:3.9-eclipse-temurin-21@sha256:99e61abcff9  0.0s
=> => resolve docker.io/library/maven:3.9-eclipse-temurin-21@sha256:99e61abcff91a9b13  0.0s
=> [stage-1 1/4] FROM docker.io/library/eclipse-temurin:21-jre-alpine@sha256:51ab5e33  0.0s
=> => resolve docker.io/library/eclipse-temurin:21-jre-alpine@sha256:51ab5e3302e7141c  0.0s
```

Figura 95. *Build con las bases fijadas: todas las capas en caché, son exactamente las mismas imágenes.*

```
Report Summary

┌──────────────────────────────────┬──────┬─────────────────┬─────────┐
│              Target              │ Type │ Vulnerabilities │ Secrets │
├──────────────────────────────────┼──────┼─────────────────┼─────────┤
│ devsecops-lab-app:0.4 (alpine 3.24.2) │ alpine │        0        │    -    │
├──────────────────────────────────┼──────┼─────────────────┼─────────┤
│ app/app.jar                      │ jar  │        0        │    -    │
└──────────────────────────────────┴──────┴─────────────────┴─────────┘
Legend:
- '-': Not scanned
- '0': Clean (no security findings detected)
```

Figura 96. *Imagen 0.4 con digests: sigue en 0 y 0.*

```
PS> docker compose ps
NAME           IMAGE
                    COMMAND                  SERVICE     CREATED          STATUS
         PORTS
jenkins        jenkins/jenkins:lts-jdk21@sha256:a660310e39ade10631f774bacd5219de767dfd08947ea
5c32a739b5e5bb382c1   "/usr/bin/tini -- /u…"   jenkins     18 seconds ago   Up 13 seconds
         127.0.0.1:8081->8080/tcp
sonarqube      sonarqube:community@sha256:2e378f7852b67d43507d3d383ee8fb5a522b8b0f920be9803f7
4f72952fce8f1         "/opt/sonarqube/dock…"   sonarqube   17 seconds ago   Up 2 seconds
         127.0.0.1:9000->9000/tcp
sonarqube-db   postgres:16-alpine@sha256:721873c34ceb9f8d8fc265984940dc982404c105f19ad51be9fd
c5970a6080ea          "docker-entrypoint.s…"   db          18 seconds ago   Up 13 seconds (he
althy)   5432/tcp
PS>
```

Figura 97. *Los tres servicios levantados con sus digests, sin perder datos.*

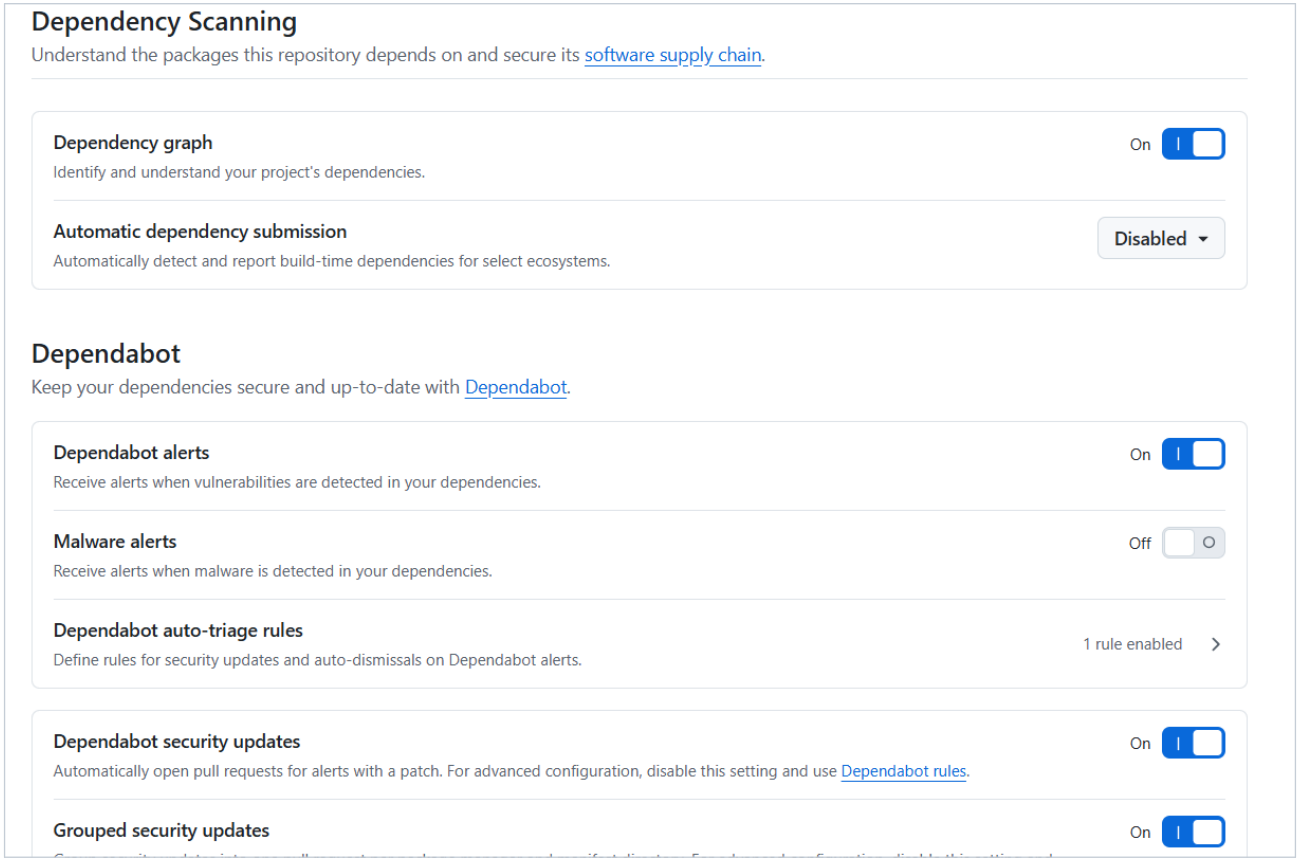


Figura 98. *Dependabot activado. Malware alerts se deja desactivado porque solo cubre paquetes npm y este proyecto no usa npm.*

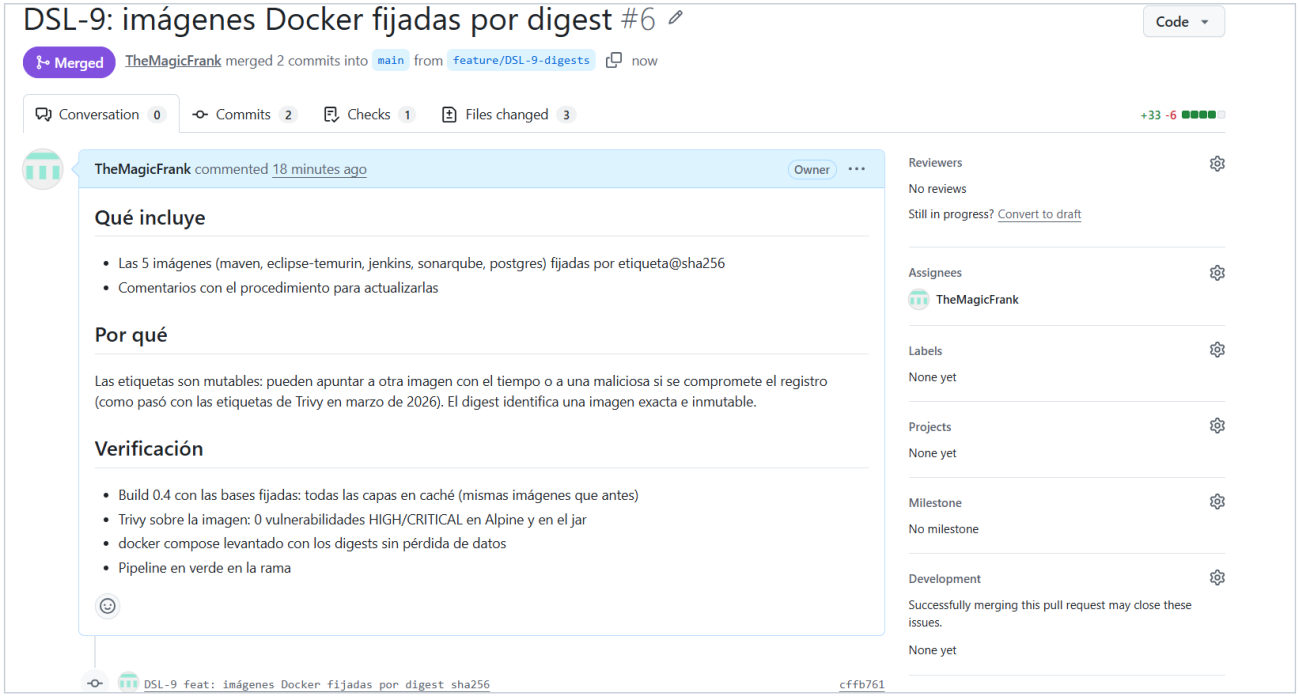


Figura 99. *Pull Request #6 fusionada: imágenes fijadas por digest y Dependabot semanal para Maven, Dockerfile y docker-compose.*

9. DSL-10: detección de secretos con Gitleaks

Gitleaks 8.30.1 se instala con la misma política que Trivy: versión fijada y verificación SHA-256. Analiza **todo el historial** de Git, no solo el estado actual, y se usa con `--redact` para que los secretos nunca aparezcan en los logs.

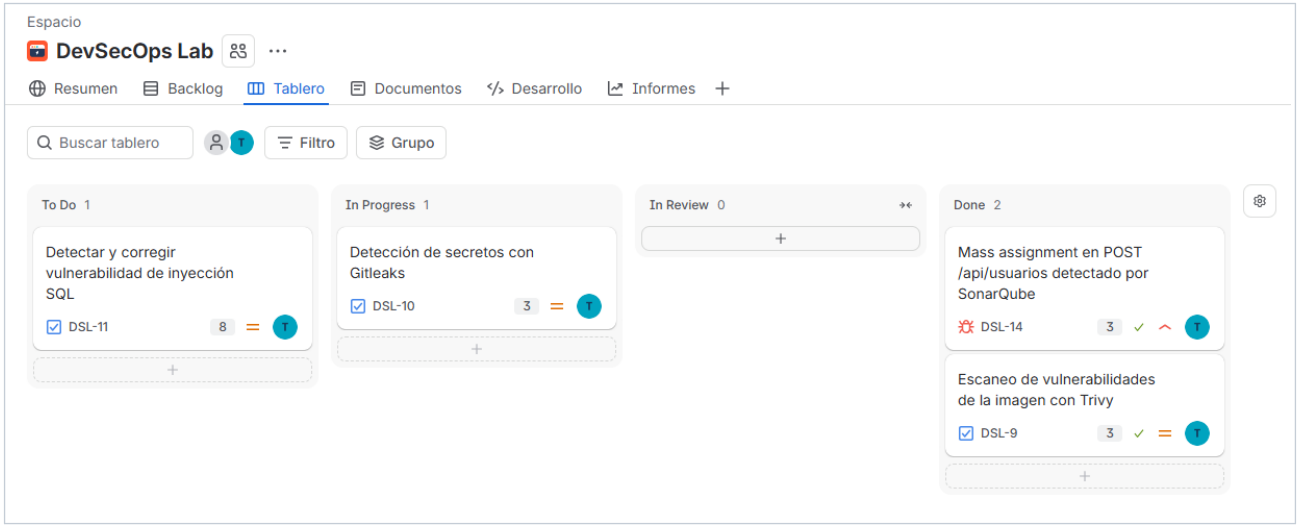


Figura 100. *DSL-10 en curso.*



Figura 101. *Huella del archivo de checksums en la release oficial de Gitleaks.*

```
PS> cd $HOME\tools\gitleaks-8.30.1
PS> $v = "8.30.1"
PS> $base = "https://github.com/gitleaks/gitleaks/releases/download/v$v"
PS> curl.exe -fsSLO "$base/gitleaks_${v}_checksums.txt"
PS> (Get-FileHash "gitleaks_${v}_checksums.txt" -Algorithm SHA256).Hash -eq "061476C21ADAF544
1516F96F185C1A4706A83CD6329B9B38762271B3D4A52FAE"
True
PS> Select-String -Path "gitleaks_${v}_checksums.txt" -Pattern "windows"

gitleaks_8.30.1_checksums.txt:8:b95f5e4f5c425cedca7ee203d9afd29597e692c4924a12ed42f970537c72
cc0f  gitleaks_8.30.1_windows_arm64.zip
gitleaks_8.30.1_checksums.txt:9:190ad53db301eec3e90afe3a1a75270768b8ebf89e731345e19421c32c1a
e1a1  gitleaks_8.30.1_windows_x32.zip
gitleaks_8.30.1_checksums.txt:10:d29144deff3a68aa93ced33dddf84b7fdc26070add4aa0f4513094c8332
afc4e  gitleaks_8.30.1_windows_x64.zip


PS> $zip = "gitleaks_${v}_windows_x64.zip"
PS> curl.exe -fsSLO "$base/$zip"
PS> $esperado = ((Select-String -Path "gitleaks_${v}_checksums.txt" -Pattern " $zip$").Line -
split '\s+')[0]
PS> (Get-FileHash $zip -Algorithm SHA256).Hash -eq $esperado
True
PS> Expand-Archive $zip -DestinationPath . -Force
PS> .\gitleaks.exe version
8.30.1
PS>
```

Figura 102. *Binario de Windows verificado.*

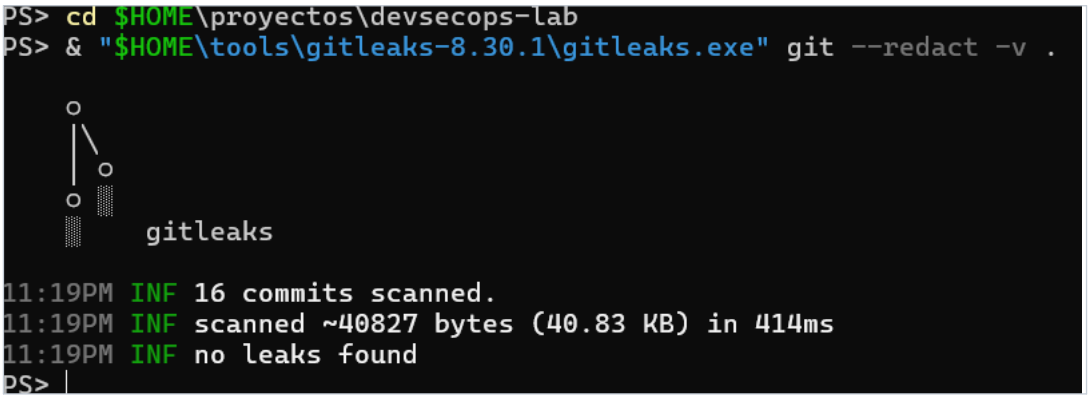


Figura 103. *Historial real del repositorio: 16 commits, sin secretos.*

> **Demostración.** En una rama local que nunca se subió, se commiteó un token falso y se borró en el commit siguiente. Gitleaks lo encontró igualmente en el historial. Lección: borrar un secreto en un commit posterior no lo elimina; si se filtra uno real, hay que revocarlo y rotarlo de inmediato.

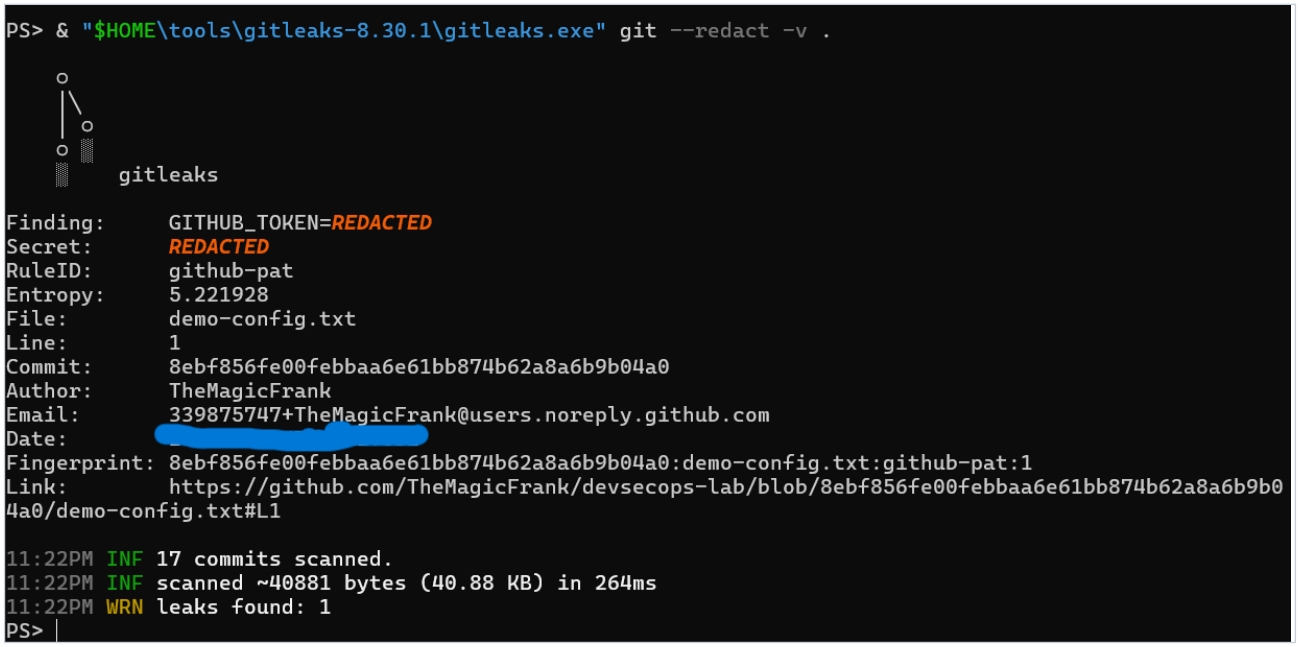


Figura 104. *Token falso detectado en el historial (regla github-pat), con el valor oculto.*

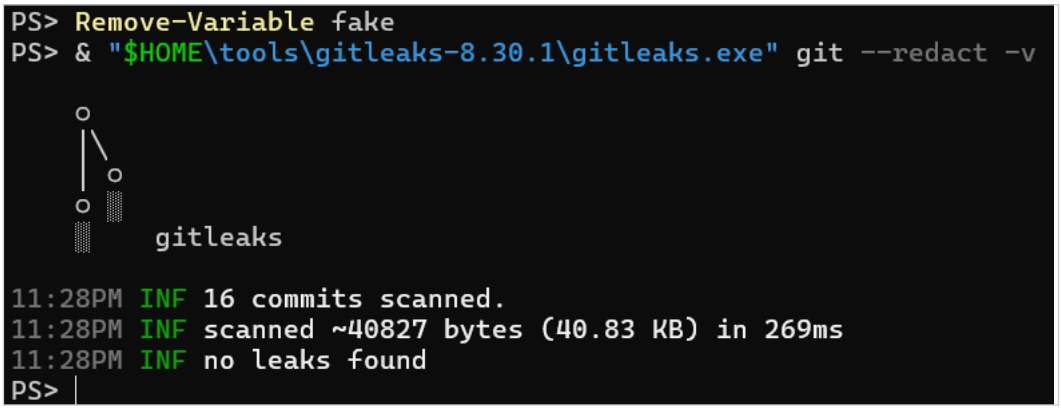


Figura 105. *Tras borrar la rama de la demo: 16 commits y ningún secreto.*

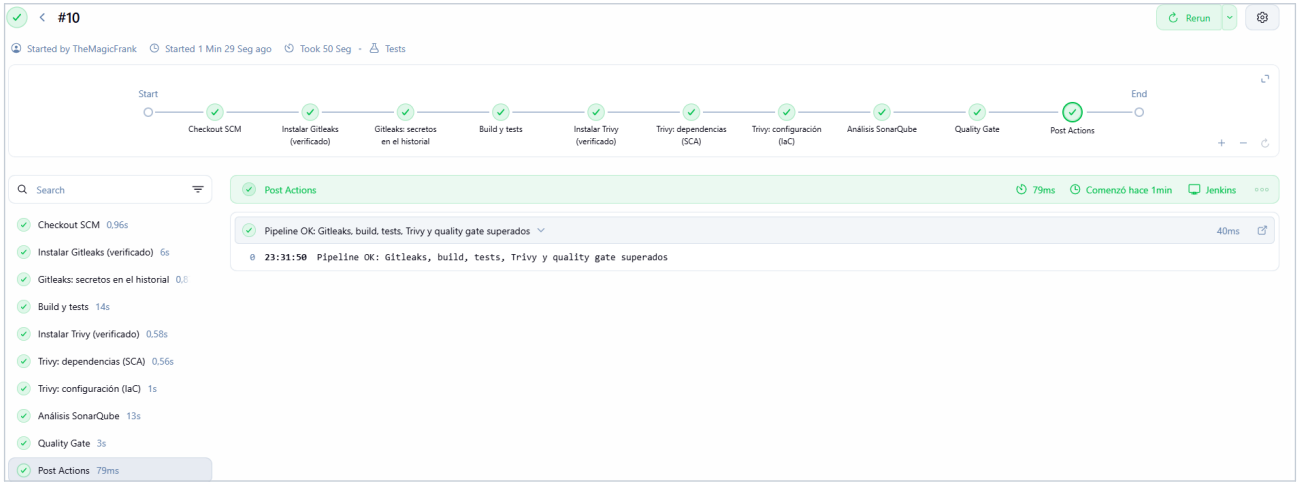


Figura 106. *Gitleaks como primera barrera del pipeline: si hay un secreto, no se llega ni a compilar.*

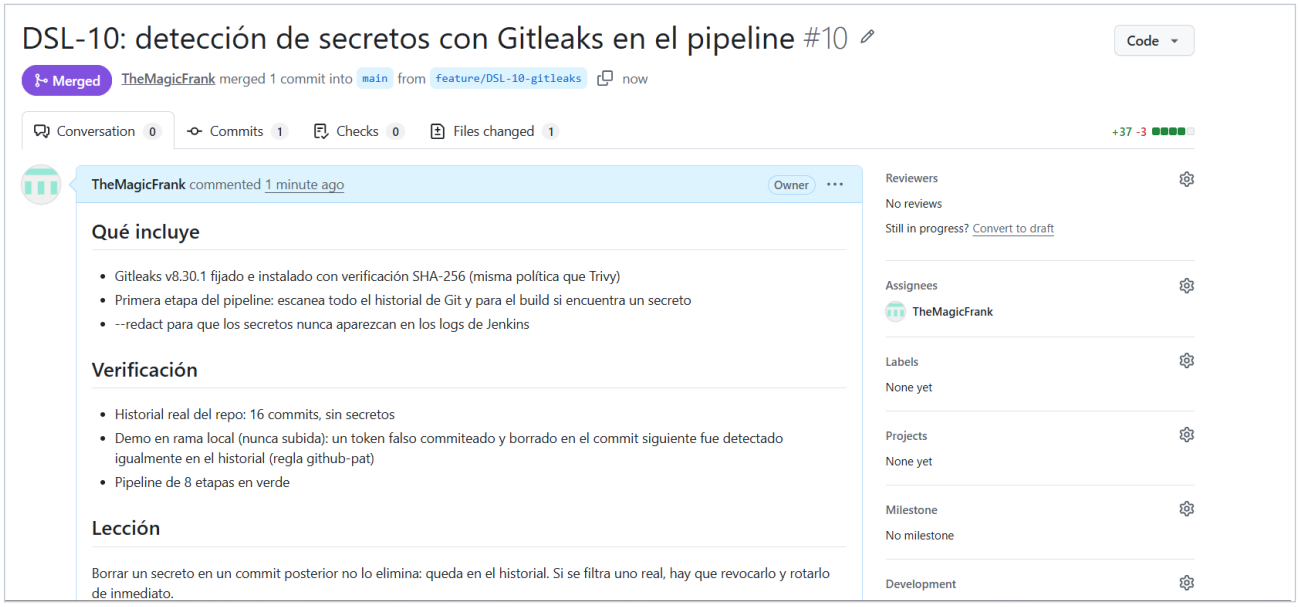


Figura 107. *Pull Request #10 fusionada.*

10. DSL-11: inyección SQL introducida, detectada y corregida

Para probar el pipeline se añadió a propósito un endpoint vulnerable, GET /api/usuarios/buscar, que construía la consulta SQL concatenando la entrada del usuario.

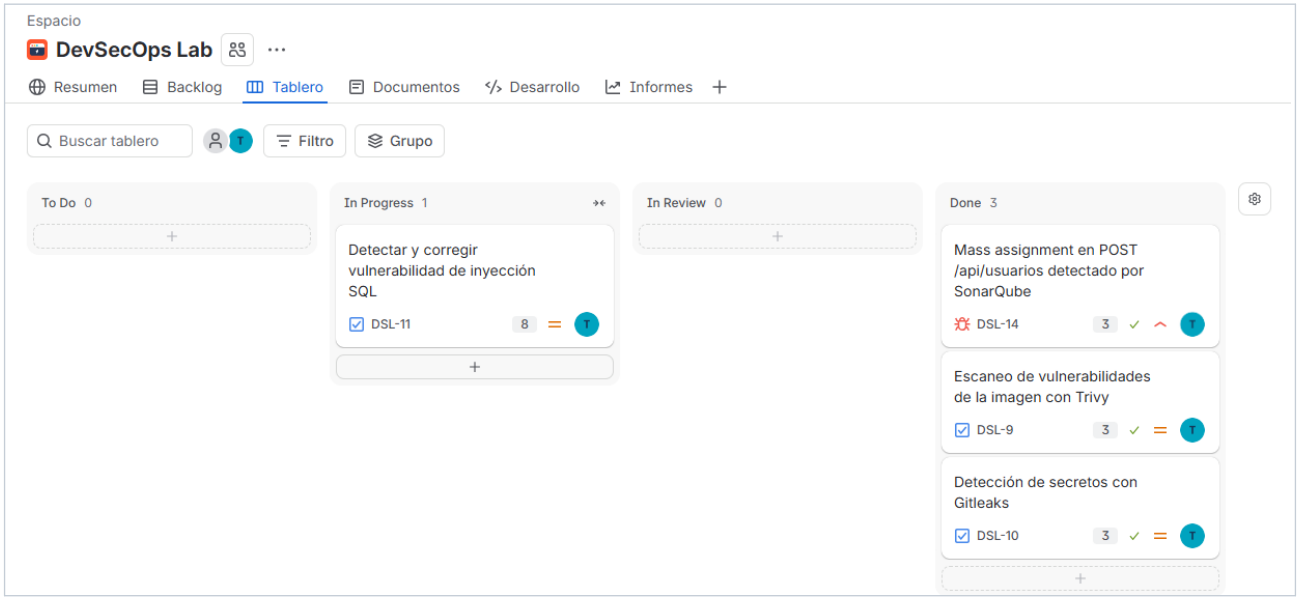


Figura 108. *DSL-11 en curso.*

```
PS> Invoke-RestMethod "http://localhost:8080/api/usuarios/buscar?nombre=Ana"

nombre email           id
------ -----           --
Ana    ana@example.com  1


PS> $payload = [uri]::EscapeDataString("x' OR '1'='1")
PS> Invoke-RestMethod "http://localhost:8080/api/usuarios/buscar?nombre=$payload"

nombre email           id
------ -----           --
Ana    ana@example.com   1
Luis   luis@example.com  2


PS>
```

Figura 109. *Exploit: con el payload x' OR '1'='1 la búsqueda devuelve todos los usuarios.*

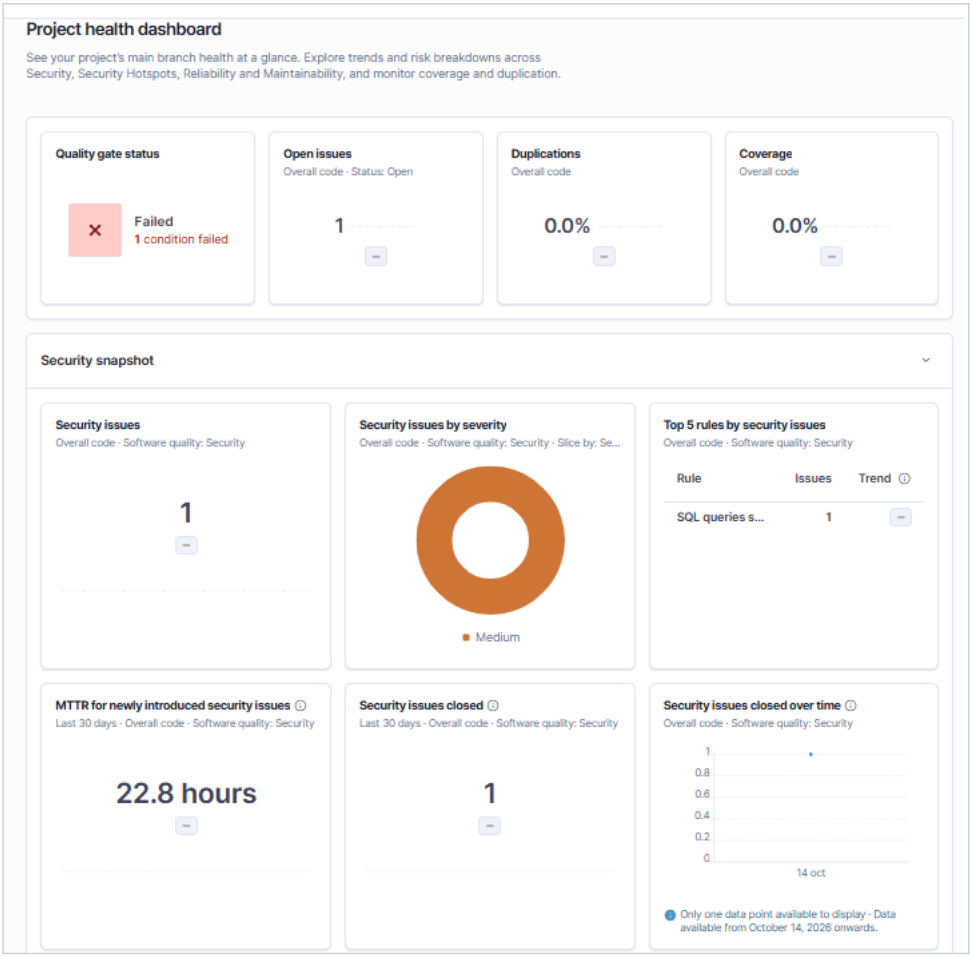


Figura 110. *SonarQube detecta la consulta construida dinámicamente y el quality gate falla.*

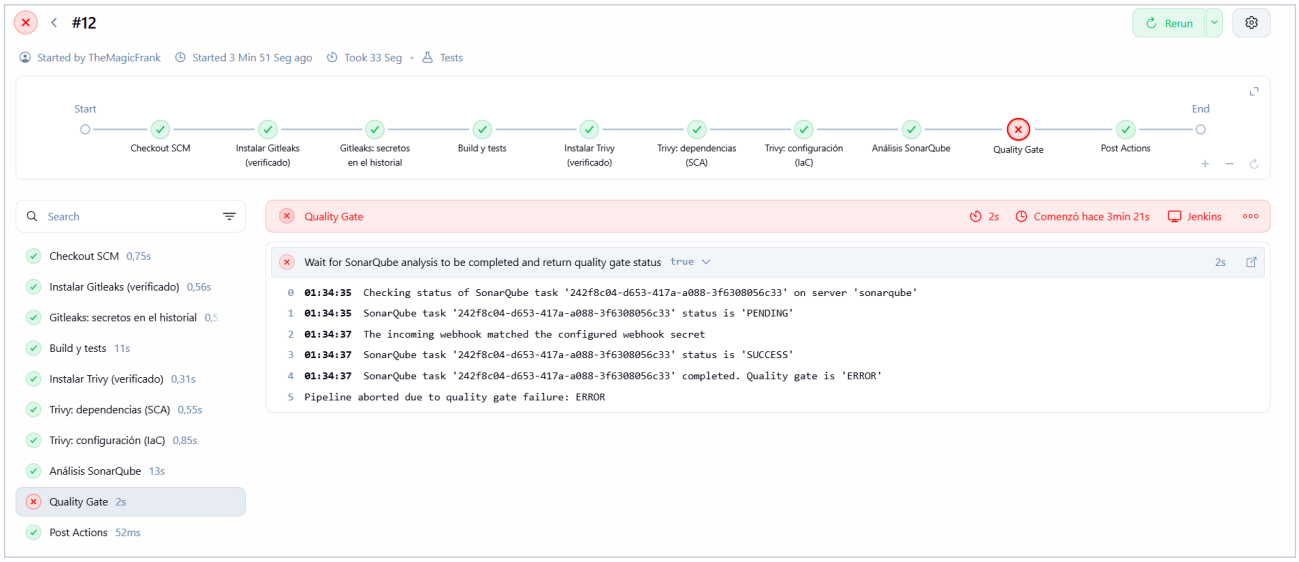


Figura 111. *El pipeline se para en rojo en el Quality Gate: el código vulnerable no pasa.*

La corrección usa una **consulta parametrizada** (createQuery con setParameter), de forma que la entrada se trata siempre como dato y nunca como parte de la consulta. Se añadieron dos tests de regresión.

```
strapper : Found @SpringBootConfiguration com.themagicfrank.app.AppApplication for test class com.them
agicfrank.app.usuario.UsuarioSqlInjectionTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.080 s -- in com.themagicfrank
.app.usuario.UsuarioSqlInjectionTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  16.048 s
```

Figura 112. *Tests de regresión de la inyección SQL: 8 tests en verde.*

```
PS> Invoke-RestMethod -Method Post -Uri http://localhost:8080/api/usuarios -ContentType "application/j
son" -Body '{"nombre":"Ana","email":"ana@example.com"}'

nombre email           id
------ -----           --
Ana    ana@example.com  1


PS> Invoke-RestMethod -Method Post -Uri http://localhost:8080/api/usuarios -ContentType "application/j
son" -Body '{"nombre":"Luis","email":"luis@example.com"}'

nombre email           id
------ -----           --
Luis   luis@example.com  2


PS> $payload = [uri]::EscapeDataString("x' OR '1'='1")
PS> Invoke-RestMethod "http://localhost:8080/api/usuarios/buscar?nombre=$payload"
PS>
```

Figura 113. *El mismo payload tras la corrección ya no devuelve ningún usuario.*

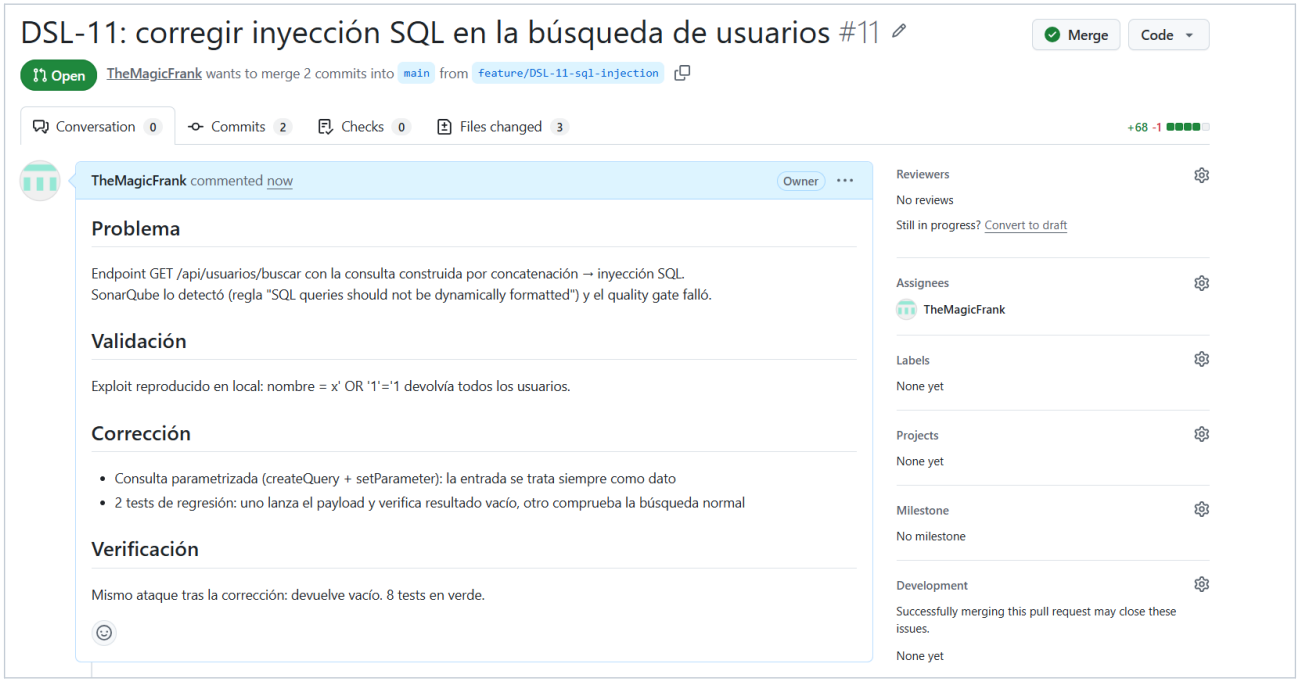


Figura 114. *Pull Request #11 abierta.*

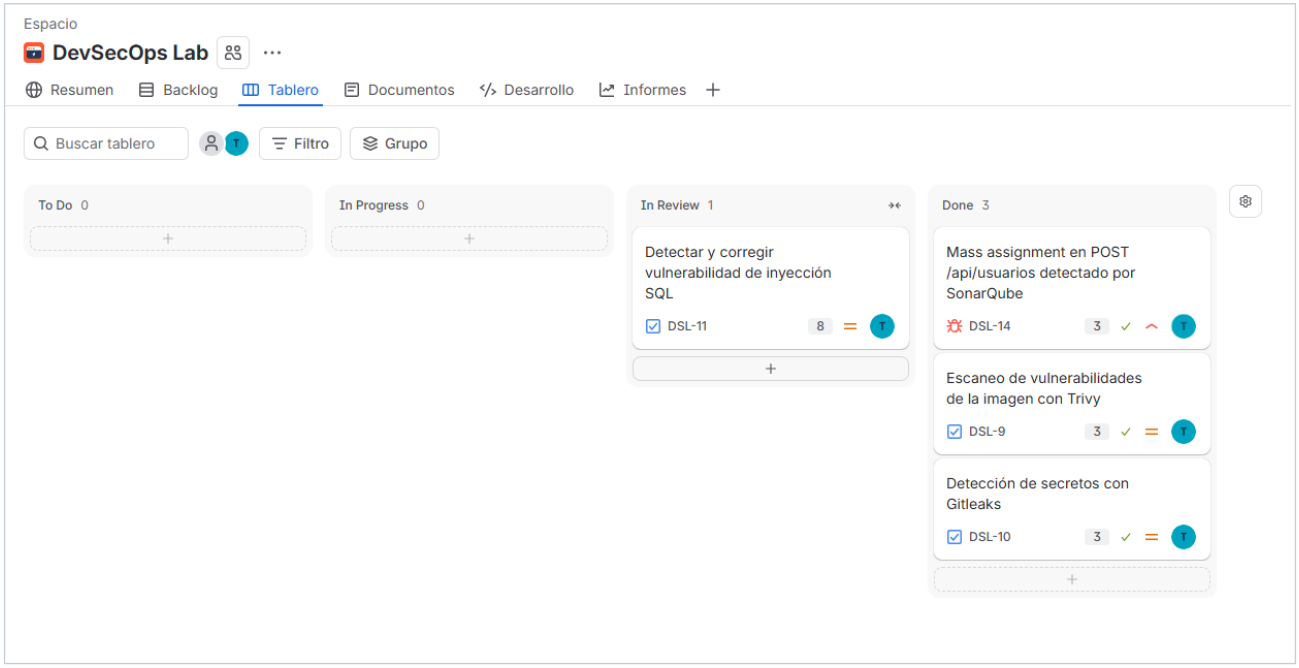


Figura 115. *DSL-11 en In Review.*

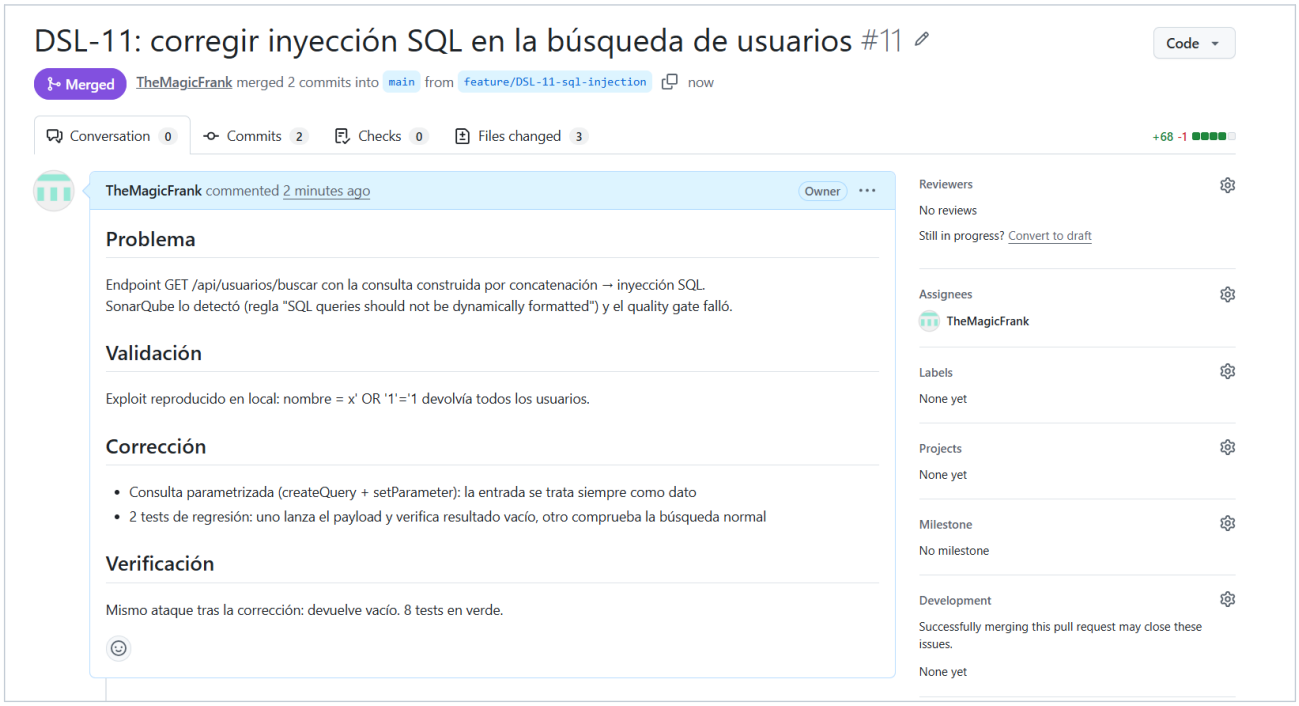


Figura 116. *PR #11 fusionada.*

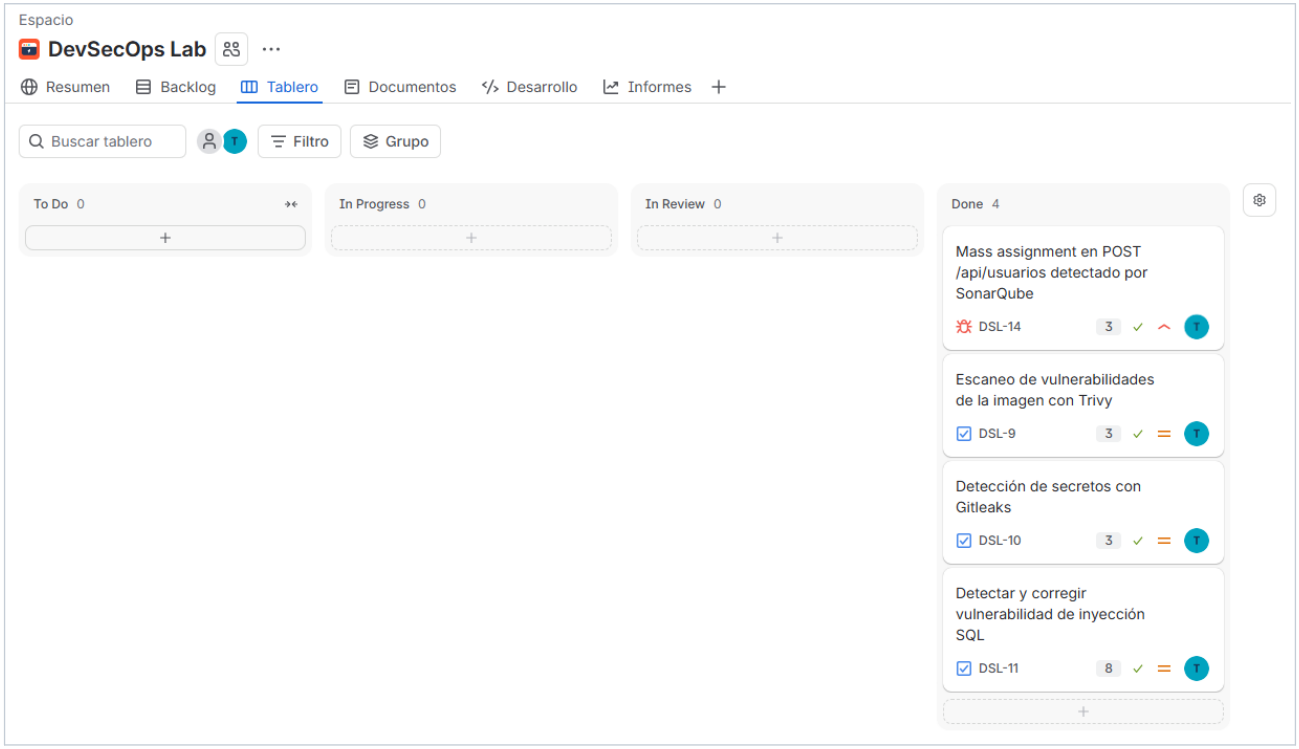


Figura 117. *Sprint 3 completo: DSL-14, DSL-9, DSL-10 y DSL-11 en Done.*

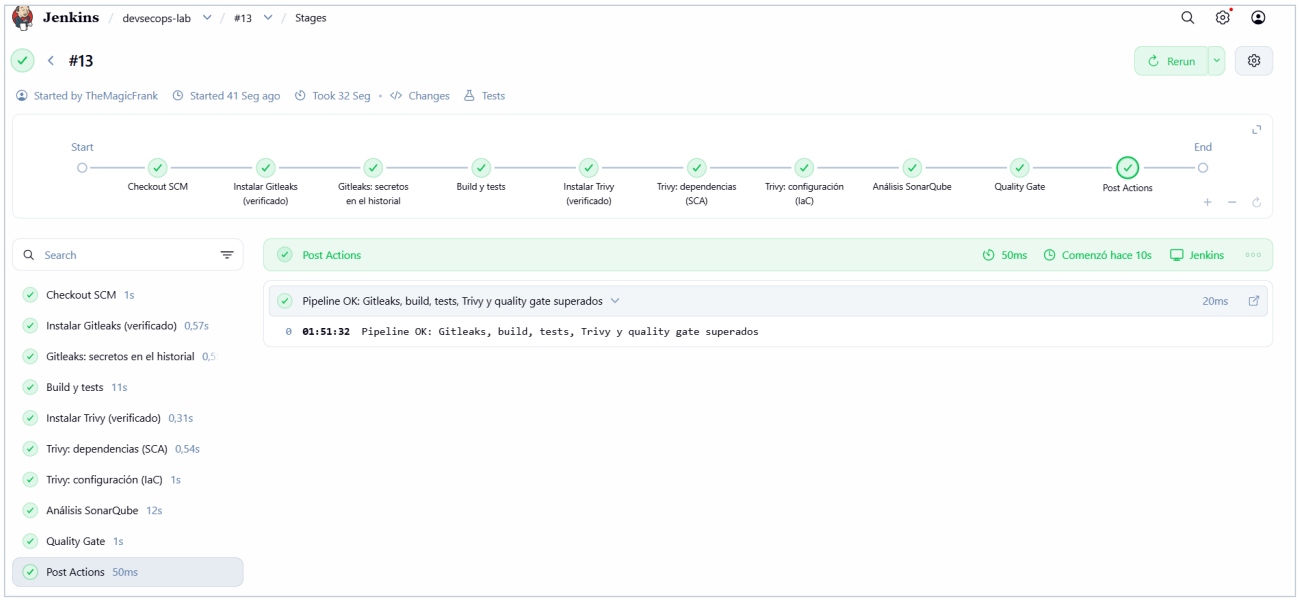


Figura 118. *Pipeline completo en verde desde main.*

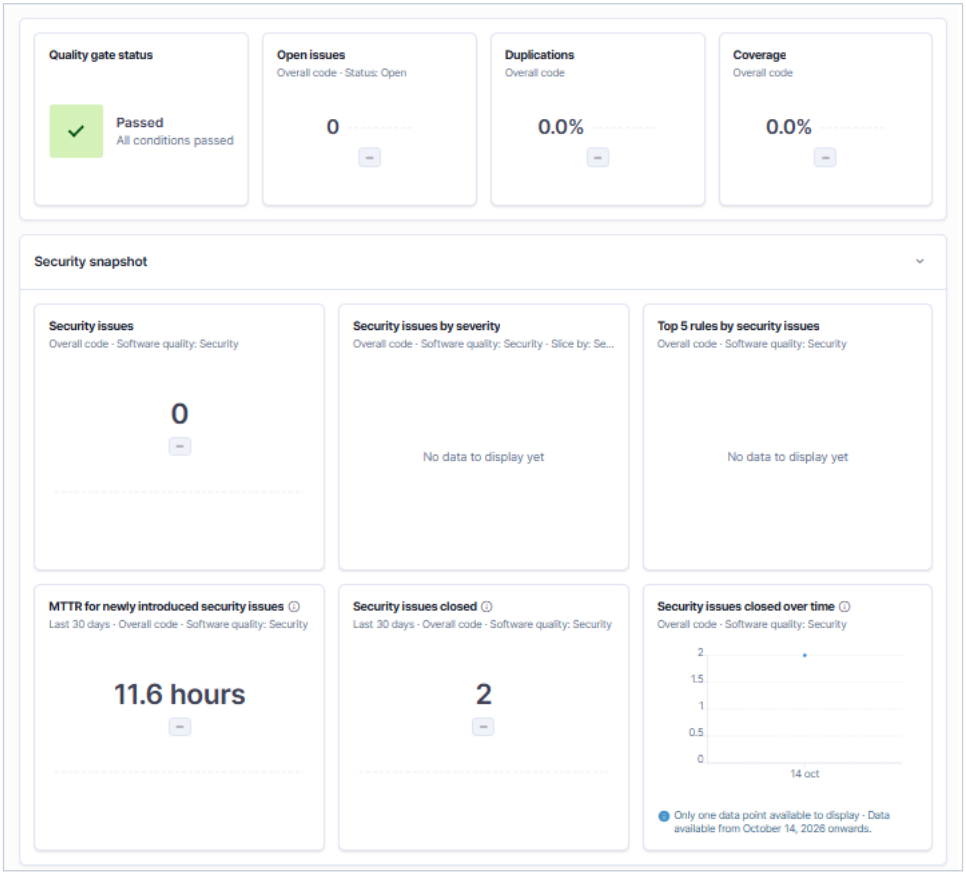


Figura 119. *SonarQube: quality gate superado, 0 issues de seguridad y 2 vulnerabilidades cerradas.*

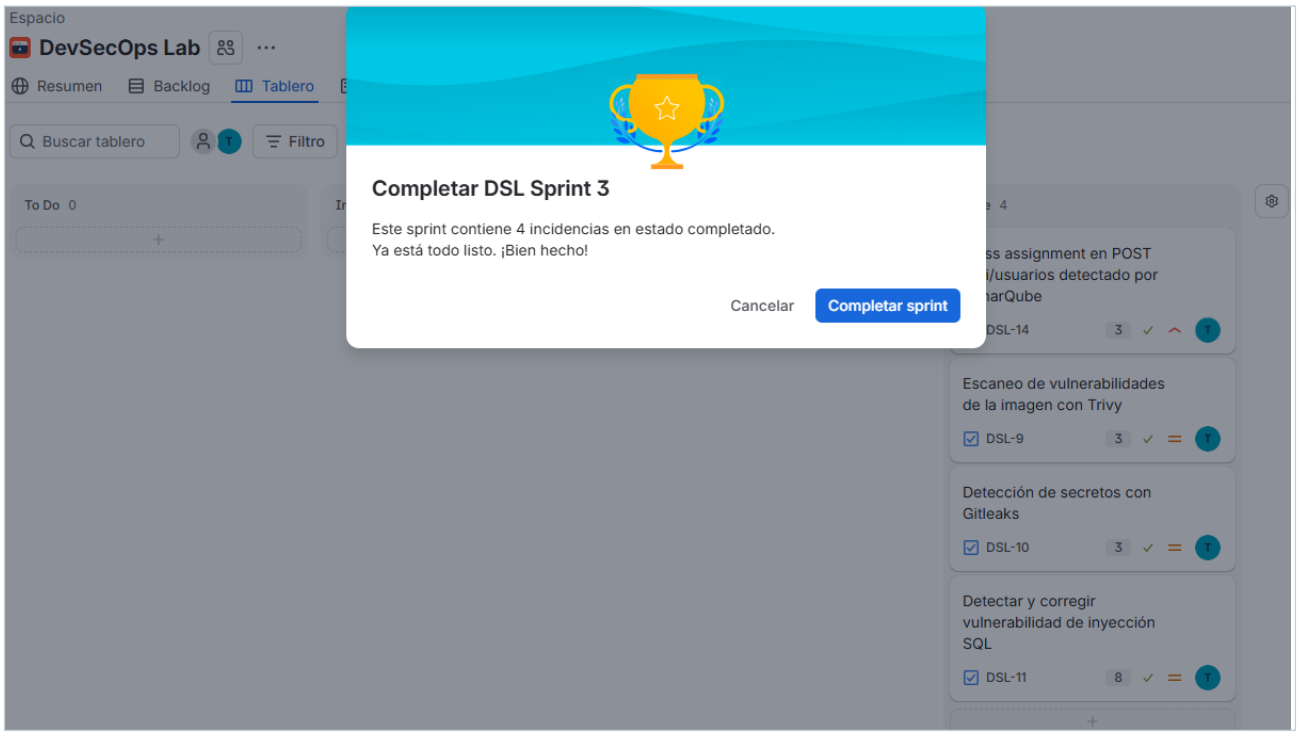


Figura 120. *Cierre del Sprint 3.*

11. Sprint 4 · DSL-12: el mismo pipeline en GitHub Actions

El workflow de GitHub Actions replica el orden de Jenkins (Gitleaks, build y tests, Trivy y análisis con quality gate) y añade el **escaneo de la imagen Docker**. El token de GitHub tiene solo `contents: read`, actions/checkout está fijada por SHA de commit y Gitleaks y Trivy se instalan con la misma verificación SHA-256, sin Actions de terceros. El workflow se dispara con `pull_request` y no con `pull_request_target`, así que las PR que lleguen desde forks no reciben los secrets del repositorio, y `SONAR_TOKEN` solo existe como variable de entorno en el paso de análisis, no en el resto del job. El análisis se hace en SonarQube Cloud.

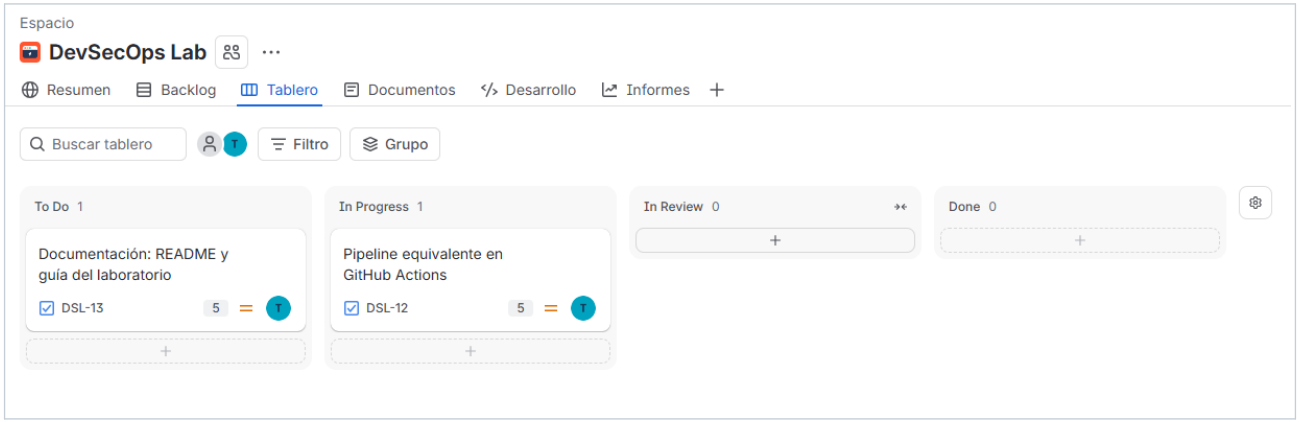


Figura 121. *Sprint 4 con DSL-12 en curso.*

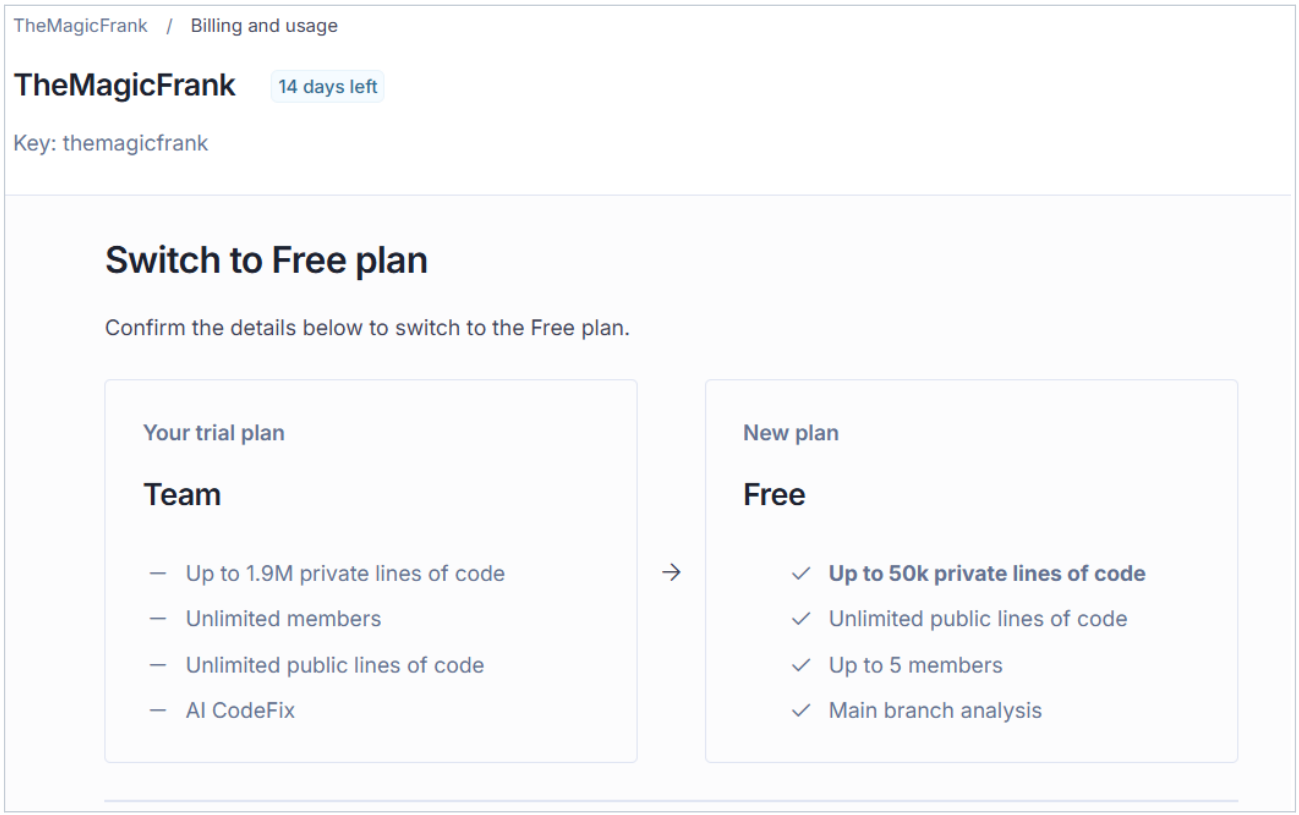


Figura 122. *Organización de SonarQube Cloud pasada al plan Free.*

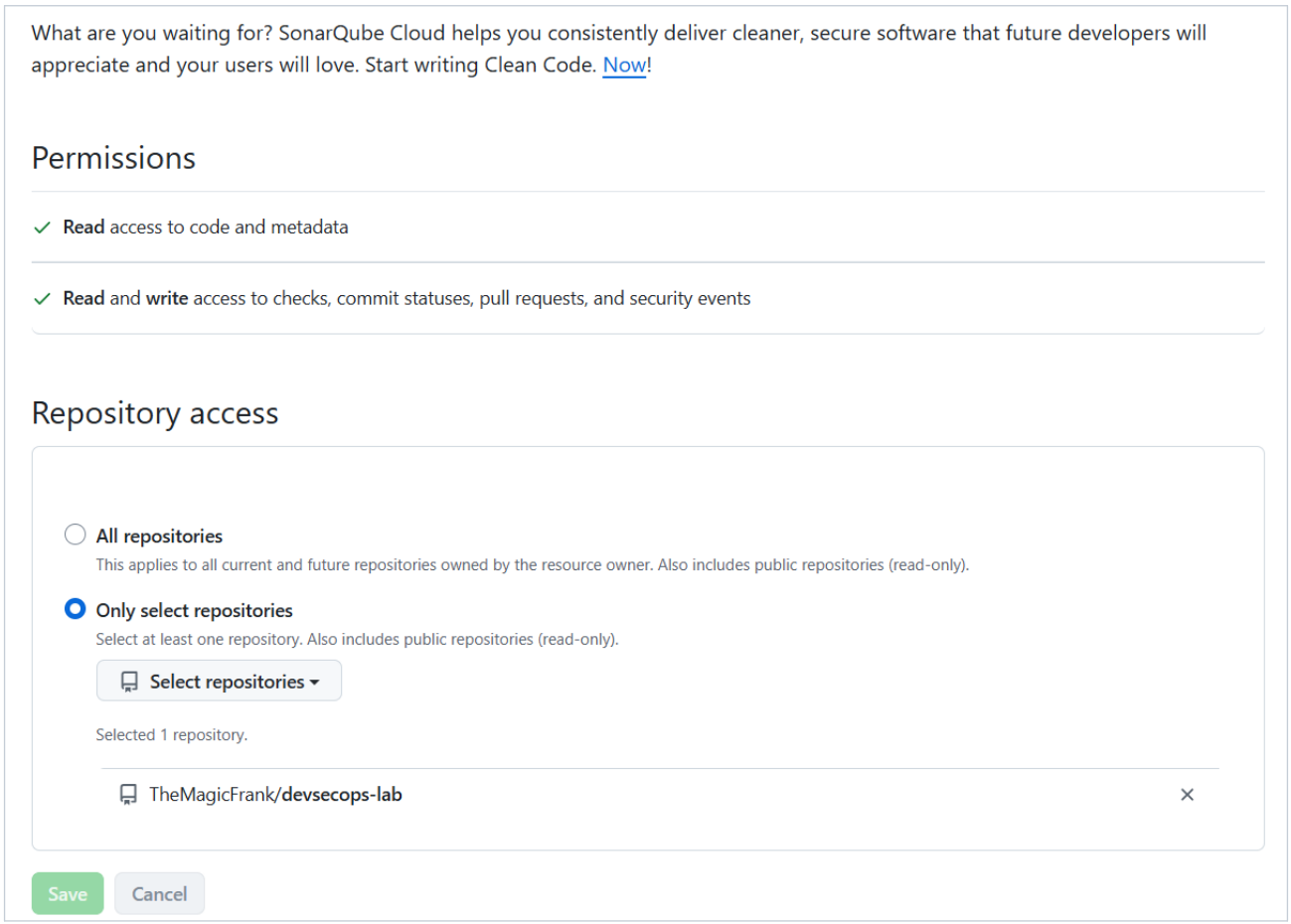


Figura 123. *App de GitHub instalada solo en el repositorio devsecops-lab, tras revisar sus permisos.*

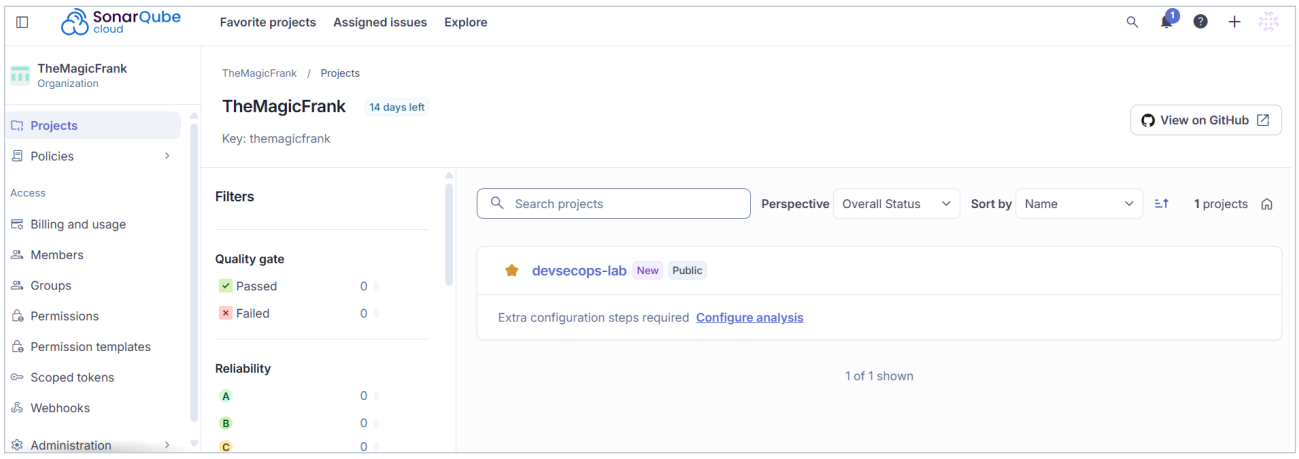


Figura 124. *Proyecto creado en SonarQube Cloud.*

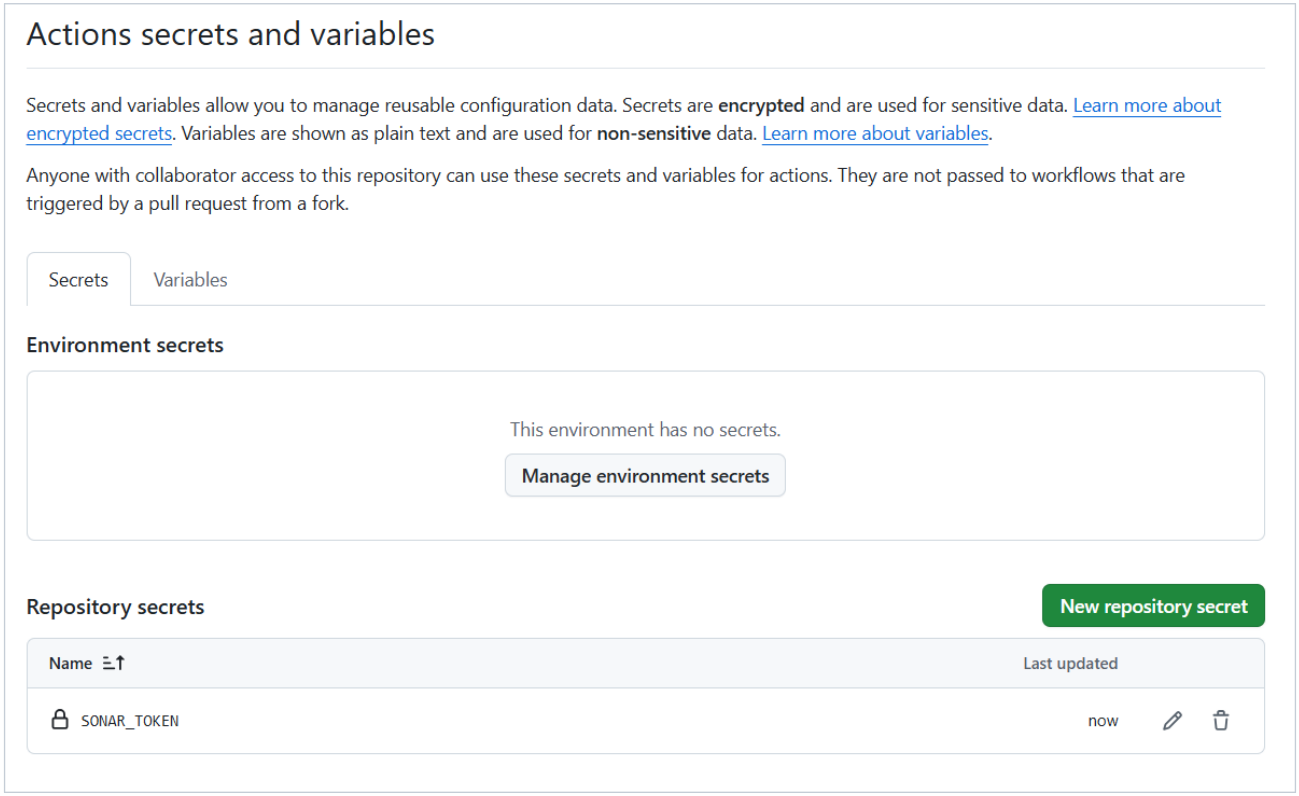


Figura 125. *Token guardado como secret cifrado SONAR_TOKEN.*

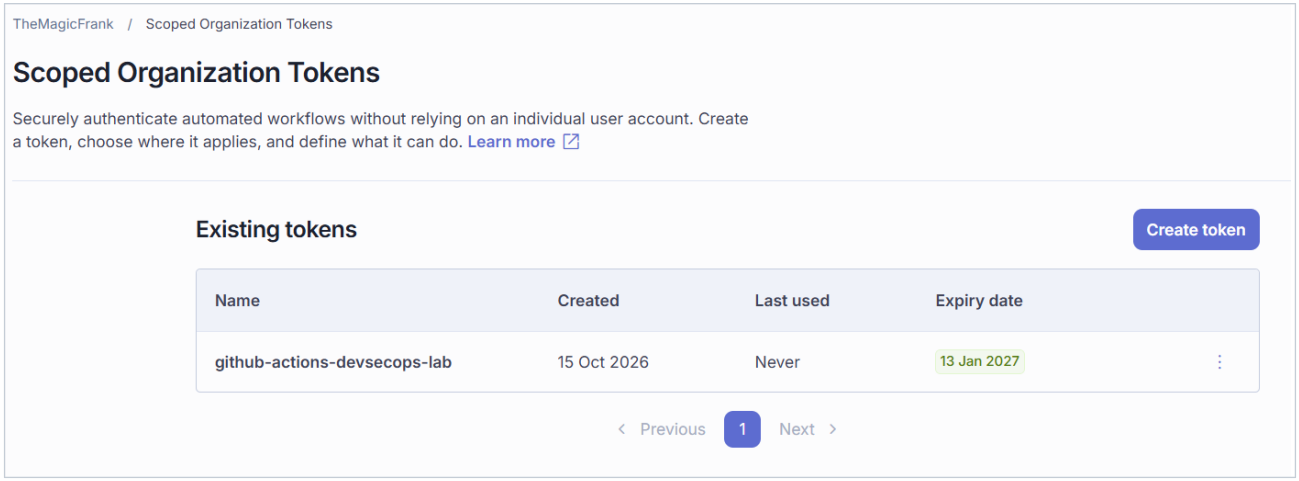


Figura 126. *Scoped Organization Token: solo para este proyecto, solo análisis y con caducidad.*

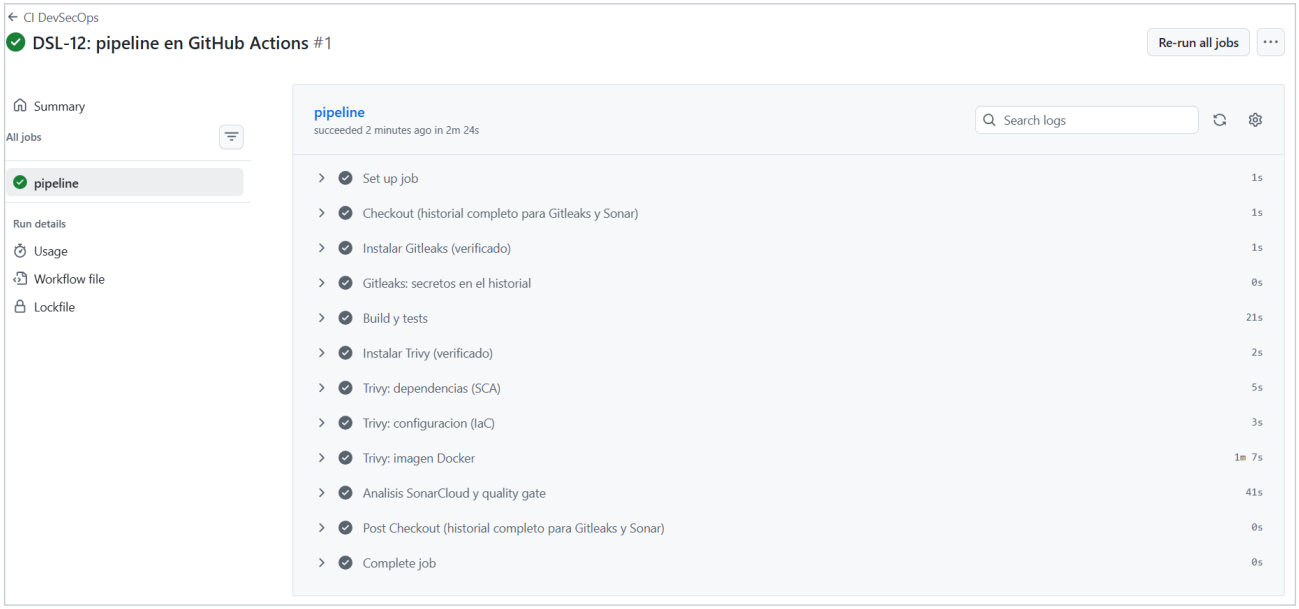


Figura 127. *Primera ejecución del workflow en verde.*

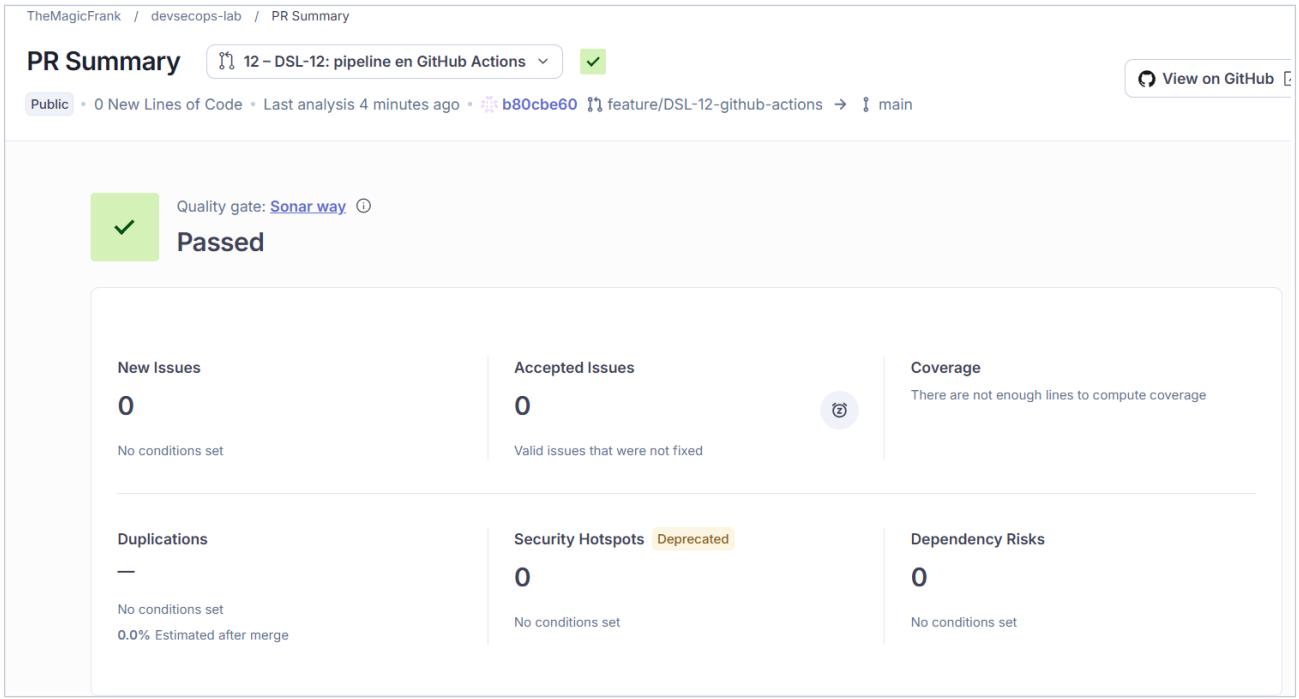


Figura 128. *Quality gate de SonarQube Cloud superado en la PR.*

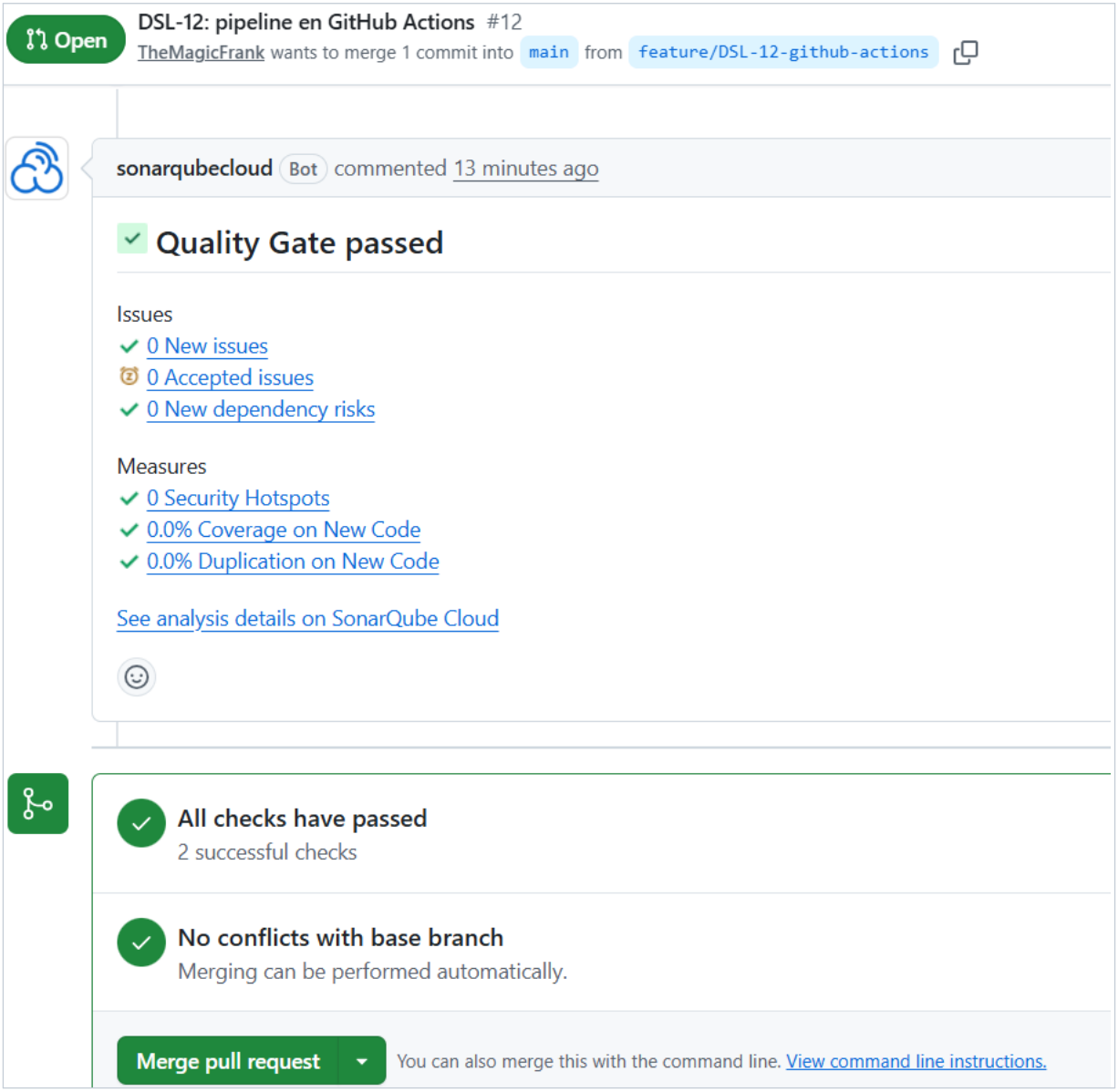


Figura 129. *Pull Request #12 con el comentario de SonarQube Cloud y los checks en verde.*

> **Revisión de Dependabot.** Dependabot abrió tres PR de salto de versión mayor: PostgreSQL 16 a 18, que requiere migrar los datos, y Java 21 a 25/26, cuando se mantiene la LTS 21. Se cerraron sin fusionar y se configuró Dependabot para ignorar las versiones mayores y vigilar también las GitHub Actions.

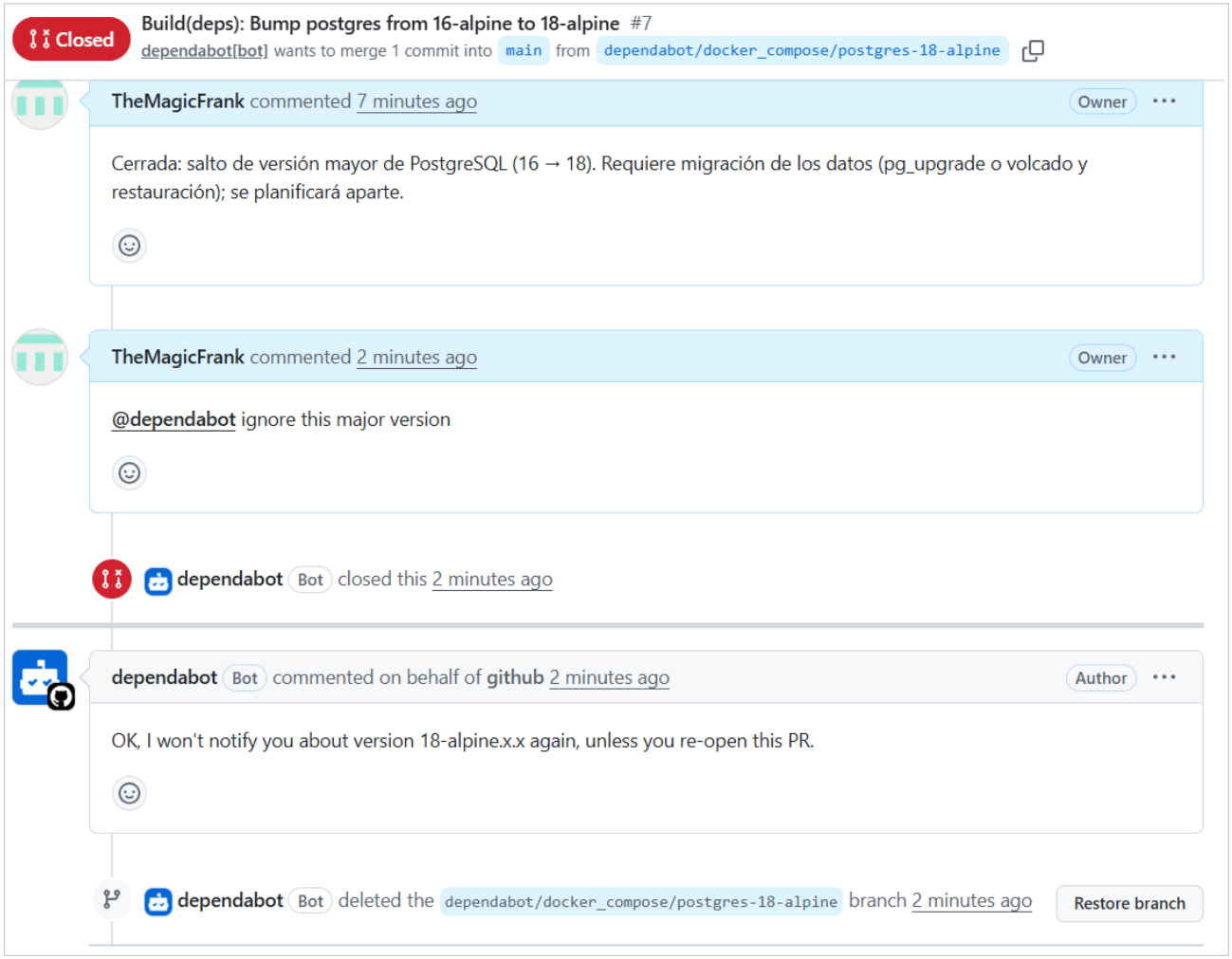


Figura 130. *PR de Dependabot cerrada con justificación.*

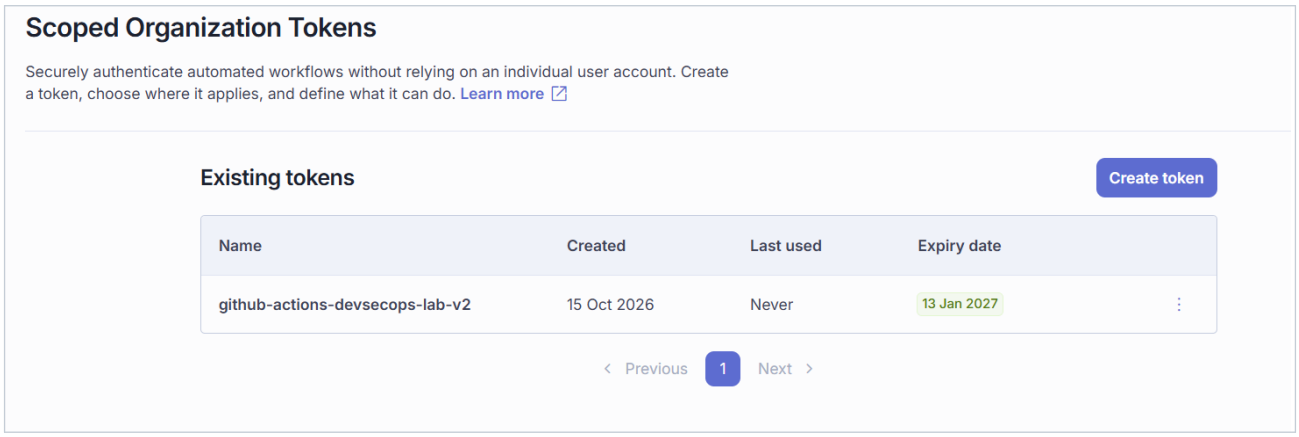


Figura 131. *Token de SonarQube Cloud rotado (v2).*

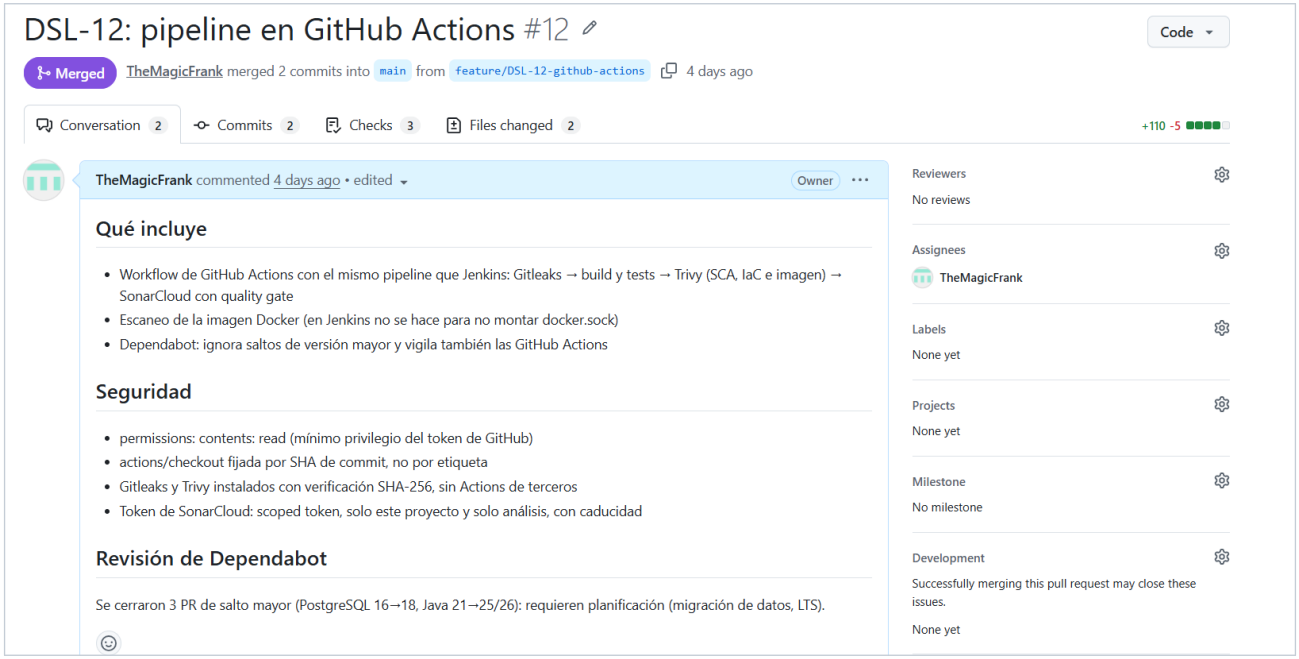


Figura 132. *PR #12 fusionada.*

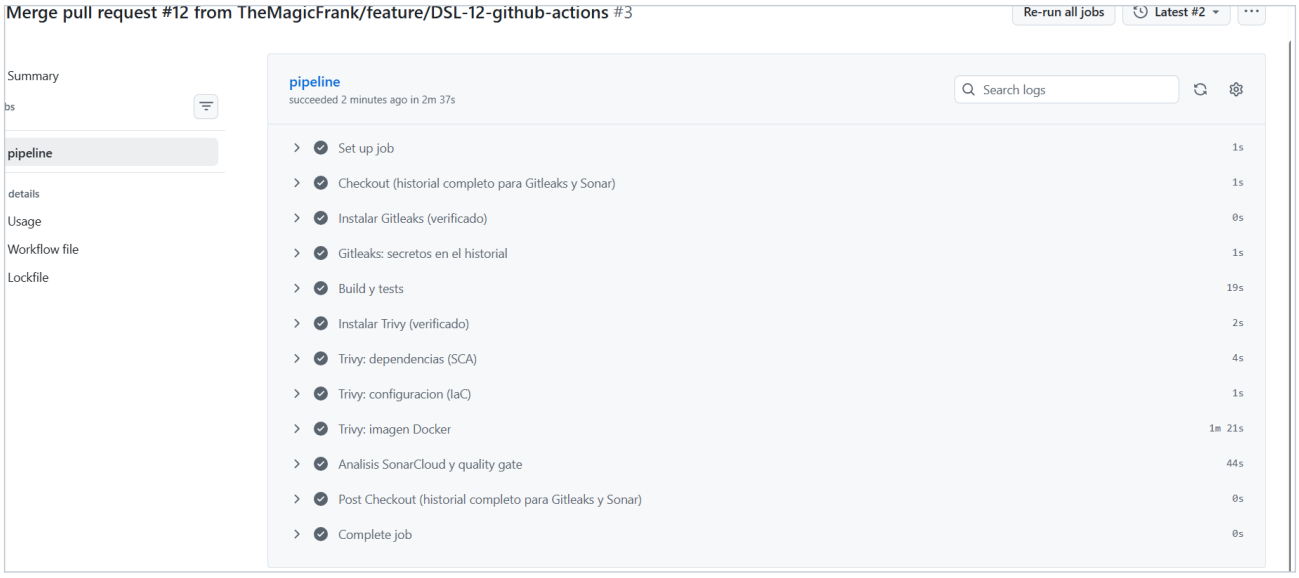


Figura 133. *Workflow en verde sobre main.*

> **Riesgos aceptados con justificación.** El análisis SCA de SonarQube Cloud detectó dos CVE en dependencias transitivas que Trivy no marcó. Ninguna es aplicable: la API solo recibe JSON y no usa SiftingAppender. Se aceptaron dejando la justificación escrita y una tarea en el backlog para revisarlas al actualizar Spring Boot.

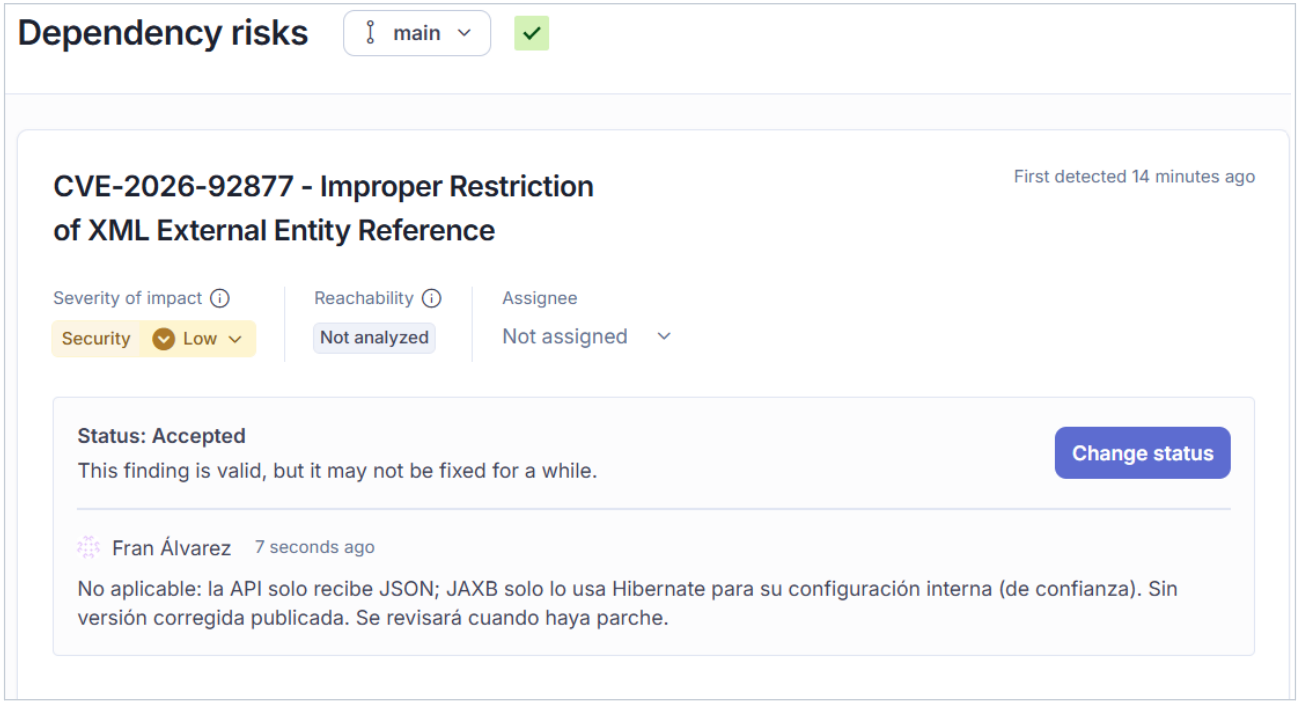


Figura 134. *CVE de JAXB (XXE) aceptada como no aplicable.*

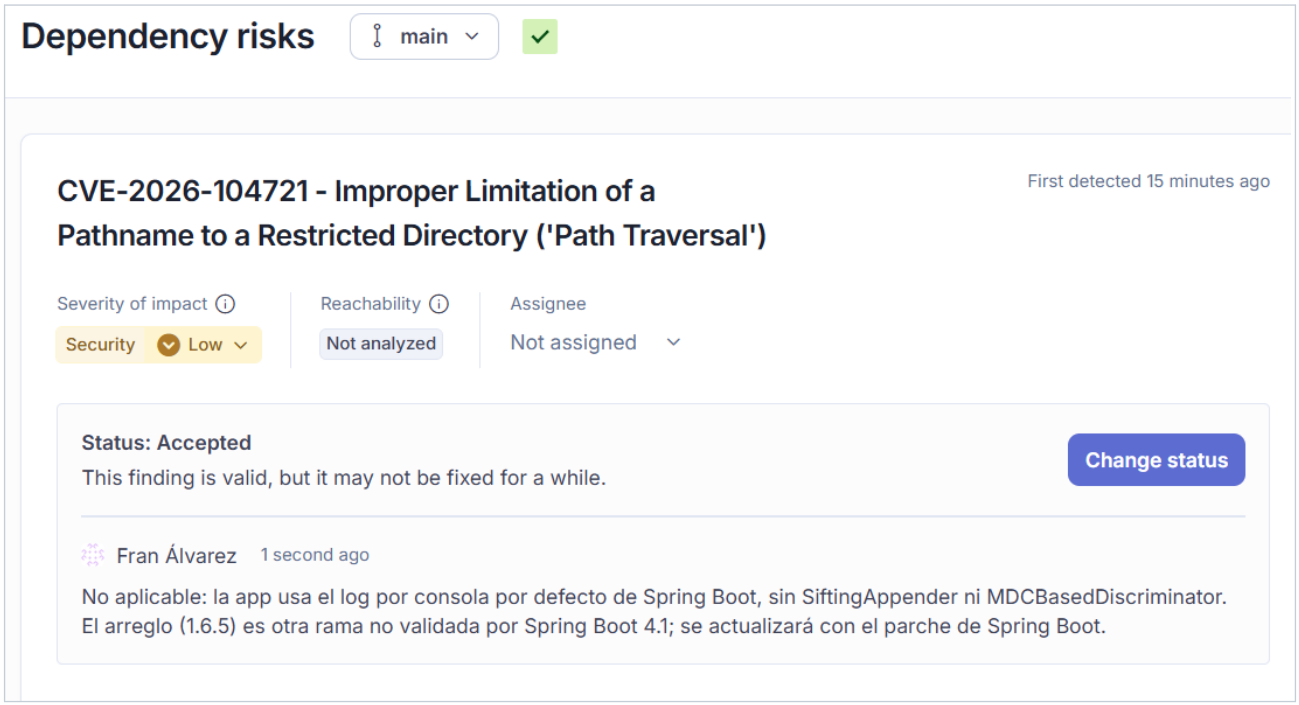


Figura 135. *CVE de Logback (path traversal) aceptada como no aplicable.*



Figura 136. *Plugin de Sonar fijado a la versión 5.8.0.7211: desaparece el aviso de versión sin especificar.*

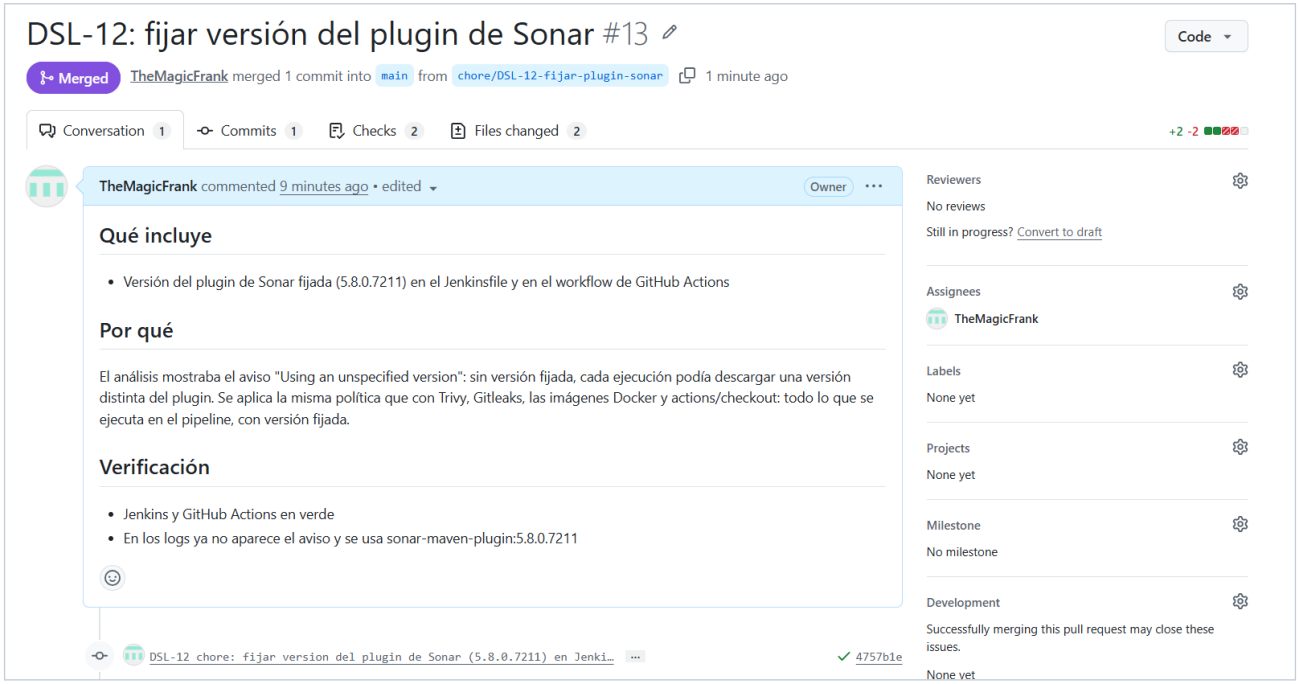


Figura 137. *Pull Request #13 fusionada.*

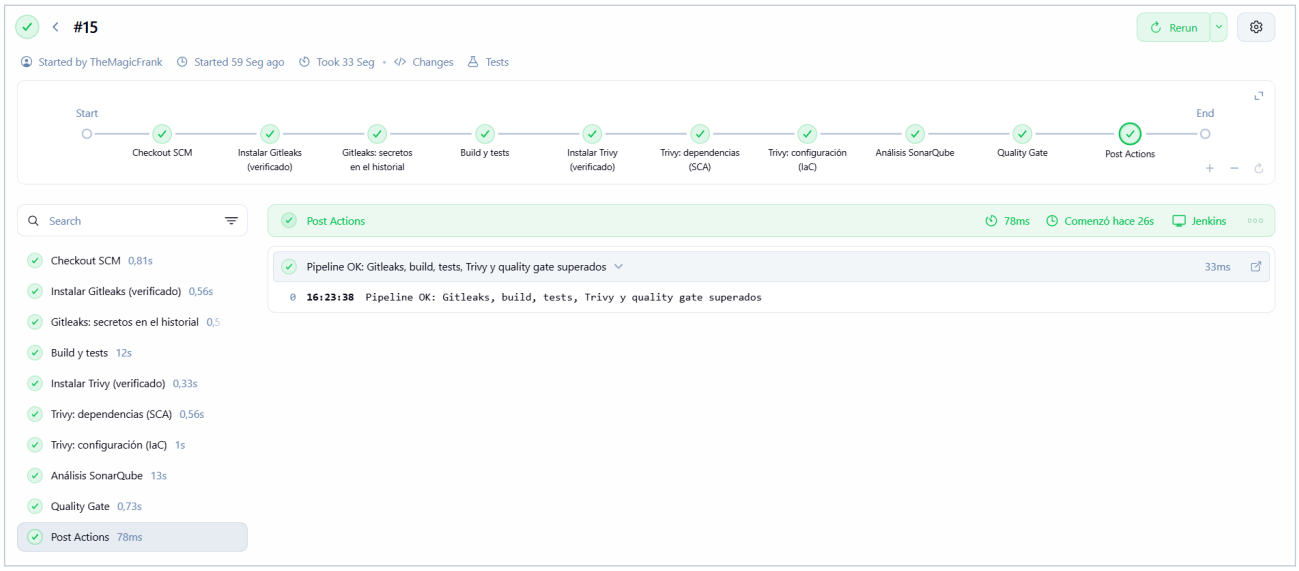


Figura 138. *Pipeline final de Jenkins en verde: Gitleaks, build y tests, Trivy SCA e IaC, SonarQube y quality gate.*

12. DSL-13: documentación

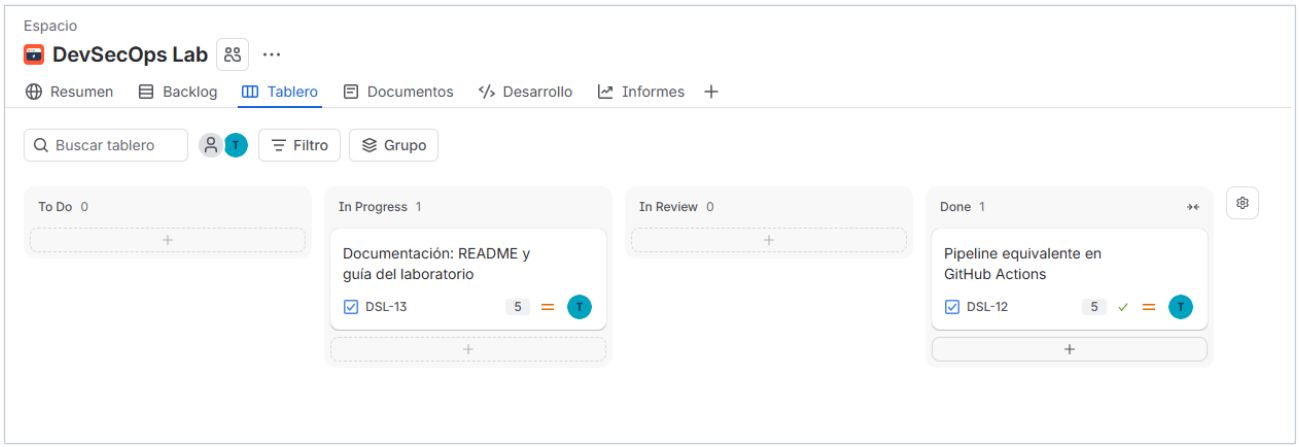


Figura 139. *DSL-13 en curso: README y esta guía.*

13. Riesgos conocidos y mejoras pendientes

Un laboratorio honesto deja por escrito lo que no está resuelto:

- Jenkins ejecuta los trabajos en el nodo principal. En producción se usarían agentes separados.
- Jenkins corre sobre Java 21, con fin de soporte en septiembre de 2027. La Content Security Policy de Jenkins está desactivada por defecto.
- La validación anti-SSRF de los webhooks de SonarQube está desactivada y se compensa con el secreto HMAC.
- Cobertura de tests al 0 % en SonarQube: falta añadir la medición de cobertura.
- Mass assignment: falta un test de regresión a nivel HTTP, además del test del servicio.
- Las versiones de Tomcat y Jackson están forzadas en el pom. Hay que quitar esas excepciones cuando Spring Boot publique una versión que las incluya.
- Trivy 0.76.0 está disponible. Actualizar es una decisión deliberada, con la misma verificación de checksums.
- Dos CVE transitivas aceptadas como no aplicables (JAXB y Logback), pendientes de revisión al actualizar Spring Boot.
- El token de SonarQube Cloud caduca el 13 de enero de 2027 y hay que rotarlo.
- La contraseña de PostgreSQL del .env se generó con Get-Random, que no es criptográficamente seguro. Es suficiente para un laboratorio local; en un entorno real se usaría System.Security.Cryptography.RandomNumberGenerator.